HyperDex Reference Manual v1.6.dev

Robert Escriva, Bernard Wong, and Emin Gün Sirer

January 12, 2015

Contents

1 Introduction **1**
- 1.1 Why HyperDex? . . . 1
- 1.2 About this Book . . . 1
- 1.3 Help and Support . . . 2
  - 1.3.1 Online and Free Resources . . . 2
  - 1.3.2 Commercial Support . . . 2

2 Installing HyperDex **3**
- 2.1 Quick Installation with Docker . . . 3
- 2.2 Installing on CentOS/RedHat Using Binary Packages . . . 4
- 2.3 Installing on Debian Using Binary Packages . . . 4
- 2.4 Installing on Ubuntu Using Binary Packages . . . 4
  - 2.4.1 Ubuntu 12.04 Precise Pangolin . . . 4
  - 2.4.2 Ubuntu 14.04 Trusty Tahr . . . 5
- 2.5 Installing on Fedora Using Binary Packages . . . 5
- 2.6 Installing on Linux Using Universal Binaries . . . 5
- 2.7 Installing on OS X Using Homebrew . . . 5
- 2.8 Installing From Source . . . 5
  - 2.8.1 Troubleshooting . . . 6
- 2.9 Verifying Installation . . . 7
- 2.10 Upgrading to 1.6 . . . 7
- 2.11 Upgrading to 1.5 . . . 7
- 2.12 Upgrading to 1.4 . . . 7
- 2.13 Upgrading to 1.3 . . . 7
- 2.14 Upgrading to 1.2 . . . 8

I For Developers **9**

3 Quick Start **10**
- 3.1 High-Level Overview . . . 10
- 3.2 Starting the Coordinator . . . 10
- 3.3 Starting HyperDex Daemons . . . 10
- 3.4 Creating a new Space . . . 11
- 3.5 Interacting with the “phonebook” Space . . . 12
- 3.6 Cleaning Up . . . 15

4 Data Types **16**
- 4.1 Setup . . . 16
- 4.2 Strings . . . 17
- 4.3 Integers . . . 18
- 4.4 Floats . . . 18
- 4.5 Lists . . . 18
- 4.6 Sets . . . 18

4.7 Maps . . . 19
4.8 Timestamps . . . 20
4.8.1 Timestamp Intervals . . . 21

5 Asynchronous Operations 23
5.1 Setup . . . 23
5.2 Asynchronous Operations . . . 24
5.3 Potential Pitfalls . . . 25
5.4 A Common Window Pattern . . . 26

6 Atomic Operations 27
6.1 Setup . . . 27
6.2 Atomic Operations . . . 28
6.3 A Comprehensive Walk . . . 29
6.3.1 Atomic Operations on Documents . . . 33

7 Document Storage 35
7.1 Setup . . . 35
7.2 Working with Documents . . . 35
7.3 Indexing Documents . . . 36

8 Authorization 37
8.1 Setup . . . 37
8.2 Using Macaroons . . . 38
8.3 Advanced Caveats . . . 39
8.4 Efficiency Considerations . . . 41

9 MongoDB Compatibility 42
9.1 Setup . . . 42
9.2 Switching to HyperDex . . . 42
9.3 The Document Type . . . 43
9.4 The Document API . . . 43
9.4.1 Operations . . . 43
9.4.2 Update Operators . . . 43
9.4.3 Predicates . . . 45

10 Transactions 46
10.1 Setup . . . 46
10.2 Using Transactions . . . 47
10.3 Rich API . . . 49
10.4 Summary . . . 49

II For Administrators 50

11 Backup and Recovery 51
11.1 Consistent versus Inconsistent Backups . . . 51
11.2 Simple Backup Strategies . . . 51
11.3 Backup Efficiency . . . 53
11.4 Advanced Backup Techniques . . . 53
11.5 Incremental Backups with Rsync . . . 54

12 Fault Tolerance **55**
12.1 Coordinator . . . 55
12.2 HyperDex Daemons . . . 55
12.3 Fault Tolerance Thresholds . . . 57
12.4 Shutting Down and Restoring a Cluster . . . 57

13 Cluster Maintenance **58**
13.1 The Daemons . . . 58
13.1.1 Scaling Up: Adding Daemons . . . 58
13.1.2 Scaling Down: Removing Daemons . . . 58
13.2 The Coordinator . . . 59
13.2.1 Listing Coordinator Servers . . . 59
13.2.2 Scaling up the Coordinator . . . 59
13.3 Spaces . . . 59
13.3.1 Adding a Space . . . 59
13.3.2 Removing a Space . . . 59
13.3.3 Renaming a Space . . . 60
13.3.4 Listing Spaces . . . 60
13.3.5 Changing a Space's Fault Tolerance . . . 60

14 Tuning HyperDex **61**
14.1 Filesystem . . . 61
14.1.1 Choosing the Right Filesystem . . . 61
14.1.2 Improving Read Performance by Disabling Access Time Updates . . . 62
14.1.3 Improving ext3/4 Write Performance with `data=writeback` . . . 62
14.1.4 Changing the Linux Disk Scheduler . . . 62
14.1.5 Linux VM Tuning . . . 62
14.2 Improving Stability by Increasing Open File Limits . . . 63
14.3 Deploying on EC2 and Other Xen Environments . . . 63

III API Reference **64**

15 C Client API **65**
15.1 Building the HyperDex C Binding . . . 65
15.2 Compiling and Linking Your Application . . . 65
15.3 Hello World . . . 65
15.4 Asynchronous Patterns . . . 67
15.5 Creating a Client . . . 67
15.6 Data Structures . . . 68
15.6.1 `enum hyperdatatype` . . . 68
15.6.2 Bytestring Format . . . 69
15.6.3 Serialization API . . . 72
15.6.4 Deserialization API . . . 75
15.6.5 Memory Management Utilties . . . 78
15.7 Attributes . . . 79
15.8 Map Attributes . . . 79
15.9 Predicates . . . 79
15.10 Error Handling . . . 80
15.11 Operations . . . 82
15.11.1 `get` . . . 83
15.11.2 `get_partial` . . . 84
15.11.3 `put` . . . 85
15.11.4 `cond_put` . . . 86

15.11.5 cond_put_or_create . . . 87
15.11.6 group_put . . . 88
15.11.7 put_if_not_exist . . . 89
15.11.8 del . . . 90
15.11.9 cond_del . . . 91
15.11.10 group_del . . . 92
15.11.11 atomic_add . . . 93
15.11.12 cond_atomic_add . . . 94
15.11.13 group_atomic_add . . . 95
15.11.14 atomic_sub . . . 96
15.11.15 cond_atomic_sub . . . 97
15.11.16 group_atomic_sub . . . 98
15.11.17 atomic_mul . . . 99
15.11.18 cond_atomic_mul . . . 100
15.11.19 group_atomic_mul . . . 101
15.11.20 atomic_div . . . 102
15.11.21 cond_atomic_div . . . 103
15.11.22 group_atomic_div . . . 104
15.11.23 atomic_mod . . . 105
15.11.24 cond_atomic_mod . . . 106
15.11.25 group_atomic_mod . . . 107
15.11.26 atomic_and . . . 108
15.11.27 cond_atomic_and . . . 109
15.11.28 group_atomic_and . . . 110
15.11.29 atomic_or . . . 111
15.11.30 cond_atomic_or . . . 112
15.11.31 group_atomic_or . . . 113
15.11.32 atomic_xor . . . 114
15.11.33 cond_atomic_xor . . . 115
15.11.34 group_atomic_xor . . . 116
15.11.35 atomic_min . . . 117
15.11.36 cond_atomic_min . . . 118
15.11.37 group_atomic_min . . . 119
15.11.38 atomic_max . . . 120
15.11.39 cond_atomic_max . . . 121
15.11.40 group_atomic_max . . . 122
15.11.41 string_prepend . . . 123
15.11.42 cond_string_prepend . . . 124
15.11.43 group_string_prepend . . . 125
15.11.44 string_append . . . 126
15.11.45 cond_string_append . . . 127
15.11.46 group_string_append . . . 128
15.11.47 string_ltrim . . . 129
15.11.48 cond_string_ltrim . . . 130
15.11.49 group_string_ltrim . . . 131
15.11.50 string_rtrim . . . 132
15.11.51 cond_string_rtrim . . . 133
15.11.52 group_string_rtrim . . . 134
15.11.53 list_lpush . . . 135
15.11.54 cond_list_lpush . . . 136
15.11.55 group_list_lpush . . . 137
15.11.56 list_rpush . . . 138
15.11.57 cond_list_rpush . . . 139
15.11.58 group_list_rpush . . . 140

15.11.59 set_add . . . 141
15.11.60 cond_set_add . . . 142
15.11.61 group_set_add . . . 143
15.11.62 set_remove . . . 144
15.11.63 cond_set_remove . . . 145
15.11.64 group_set_remove . . . 146
15.11.65 set_intersect . . . 147
15.11.66 cond_set_intersect . . . 148
15.11.67 group_set_intersect . . . 149
15.11.68 set_union . . . 150
15.11.69 cond_set_union . . . 151
15.11.70 group_set_union . . . 152
15.11.71 document_rename . . . 153
15.11.72 cond_document_rename . . . 154
15.11.73 group_document_rename . . . 155
15.11.74 document_unset . . . 156
15.11.75 cond_document_unset . . . 157
15.11.76 group_document_unset . . . 158
15.11.77 map_add . . . 159
15.11.78 cond_map_add . . . 160
15.11.79 group_map_add . . . 161
15.11.80 map_remove . . . 162
15.11.81 cond_map_remove . . . 163
15.11.82 group_map_remove . . . 164
15.11.83 map_atomic_add . . . 165
15.11.84 cond_map_atomic_add . . . 166
15.11.85 group_map_atomic_add . . . 167
15.11.86 map_atomic_sub . . . 168
15.11.87 cond_map_atomic_sub . . . 169
15.11.88 group_map_atomic_sub . . . 170
15.11.89 map_atomic_mul . . . 171
15.11.90 cond_map_atomic_mul . . . 172
15.11.91 group_map_atomic_mul . . . 173
15.11.92 map_atomic_div . . . 174
15.11.93 cond_map_atomic_div . . . 175
15.11.94 group_map_atomic_div . . . 176
15.11.95 map_atomic_mod . . . 177
15.11.96 cond_map_atomic_mod . . . 178
15.11.97 group_map_atomic_mod . . . 179
15.11.98 map_atomic_and . . . 180
15.11.99 cond_map_atomic_and . . . 181
15.11.100 group_map_atomic_and . . . 182
15.11.101 map_atomic_or . . . 183
15.11.102 cond_map_atomic_or . . . 184
15.11.103 group_map_atomic_or . . . 185
15.11.104 map_atomic_xor . . . 186
15.11.105 cond_map_atomic_xor . . . 187
15.11.106 group_map_atomic_xor . . . 188
15.11.107 map_string_prepend . . . 189
15.11.108 cond_map_string_prepend . . . 190
15.11.109 group_map_string_prepend . . . 191
15.11.110 map_string_append . . . 192
15.11.111 cond_map_string_append . . . 193
15.11.112 group_map_string_append . . . 194

15.11.13 map_atomic_min . . . 195
15.11.14 cond_map_atomic_min . . . 196
15.11.15 group_map_atomic_min . . . 197
15.11.16 map_atomic_max . . . 198
15.11.17 cond_map_atomic_max . . . 199
15.11.18 group_map_atomic_max . . . 200
15.11.19 search . . . 201
15.11.20 sorted_search . . . 202
15.11.21 count . . . 203
15.12 Working with Signals . . . 204
15.13 Working with Threads . . . 204

16 C Admin API **205**
16.1 Operations . . . 205
16.1.1 read_only . . . 206
16.1.2 wait_until_stable . . . 207
16.1.3 fault_tolerance . . . 208
16.1.4 validate_space . . . 209
16.1.5 add_space . . . 210
16.1.6 rm_space . . . 211
16.1.7 mv_space . . . 212
16.1.8 list_spaces . . . 213
16.1.9 list_indices . . . 214
16.1.10 list_subspaces . . . 215
16.1.11 add_index . . . 216
16.1.12 rm_index . . . 217
16.1.13 server_register . . . 218
16.1.14 server_online . . . 219
16.1.15 server_offline . . . 220
16.1.16 server_forget . . . 221
16.1.17 server_kill . . . 222
16.1.18 backup . . . 223
16.1.19 enable_perf_counters . . . 224
16.1.20 disable_perf_counters . . . 225

17 Go Client API **226**
17.0.1 Using Go Within Your Application . . . 226
17.0.2 Hello World . . . 226
17.0.3 Data Structures . . . 227
17.0.4 Attributes . . . 227
17.0.5 Map Attributes . . . 228
17.0.6 Predicates . . . 228
17.0.7 Error Handling . . . 228
17.0.8 Operations . . . 228

18 Java API **350**
18.1 Client Library . . . 350
18.1.1 Building the HyperDex Java Binding . . . 350
18.1.2 Using Java Within Your Application . . . 350
18.1.3 Hello World . . . 350
18.1.4 Asynchronous Operations . . . 351
18.1.5 Data Structures . . . 352
18.1.6 Attributes . . . 353
18.1.7 Map Attributes . . . 353

18.1.8 Predicates . . . 353
18.1.9 Error Handling . . . 354
18.1.10 Operations . . . 354
18.1.11 Working with Signals . . . 597
18.1.12 Working with Threads . . . 597

19 Node API **598**
19.1 Client Library . . . 598
19.1.1 Building the HyperDex Node.js Binding . . . 598
19.1.2 Using Node.js Within Your Application . . . 598
19.1.3 Hello World . . . 598
19.1.4 Asynchronous Operations . . . 599
19.1.5 Data Structures . . . 599
19.1.6 Attributes . . . 600
19.1.7 Operations . . . 600
19.1.8 Working with Signals . . . 722
19.1.9 Working with Events . . . 722

20 Python Client API **723**
20.0.1 Building the HyperDex Python Binding . . . 723
20.0.2 Using Python Within Your Application . . . 723
20.0.3 Hello World . . . 723
20.0.4 Asynchronous Operations . . . 724
20.0.5 Data Structures . . . 724
20.0.6 Attributes . . . 725
20.0.7 Map Attributes . . . 725
20.0.8 Predicates . . . 725
20.0.9 Error Handling . . . 726
20.0.10 Operations . . . 726
20.0.11 Working with Signals . . . 967
20.0.12 Working with Threads . . . 967

21 Ruby API **968**
21.1 Client Library . . . 968
21.1.1 Building the HyperDex Ruby Binding . . . 968
21.1.2 Using Ruby Within Your Application . . . 968
21.1.3 Hello World . . . 968
21.1.4 Asynchronous Operations . . . 969
21.1.5 Data Structures . . . 969
21.1.6 Attributes . . . 970
21.1.7 Map Attributes . . . 970
21.1.8 Predicates . . . 970
21.1.9 Error Handling . . . 971
21.1.10 Operations . . . 971
21.1.11 Working with Signals . . . 1212
21.1.12 Working with Threads . . . 1212

Index **1213**

Chapter 1

Introduction

1.1 Why HyperDex?

HyperDex is the next-generation key-value store, designed from the ground up to provide a convenient user interface without sacrificing reliability, robustness, or performance. First and foremost, HyperDex enables developers to create new applications quickly and with strong correctness guarantees. HyperDex features:

Performance: HyperDex achieves lower latency and higher throughput than many other key-value stores. Support your application with fewer machines and save on operational costs.

Consistency: HyperDex provides strongly-consistent reads, writes, and atomic operations, making strong guarantees to your application. Other key-value stores promise `eventual` consistency, leaving your application to pick up the slack. HyperDex solves the tough problems, enabling you to write correct applications quickly with less effort.

Fault Tolerance: HyperDex employs a unique, built-in fault-tolerance mechanism to safe-guard your data. Simply tell HyperDex how many failures your cluster is likely to experience, and HyperDex ensures that your data is replicated to withstand at least that many failures. Where other systems require you to fully understand the replication internals to bring fault-tolerance your data, HyperDex's fault-tolerance mechanism is simple to understand and straightforward to use.

Scalability: HyperDex automatically scales as more machines are added to the system. Add additional servers to your HyperDex cluster to increase storage capacity and achievable throughput.

Easy-to-Use: HyperDex is easy to use in both development and production environments. Set up a HyperDex cluster quickly, and scale out to multiple machines without any application-level changes. Other systems may require that your application know about internals of the deployment, such as cluster membership, replication options, and sharding configurations. HyperDex internally and automatically manages your cluster so your application doesn't have to.

1.2 About this Book

This book is the definitive HyperDex reference manual for users, administrators, and developers. The HyperDex developers update this manual in tandem with each release to ensure that the book serves as the best reference for HyperDex.

This book is divided into three parts, each of which is directed to a different audience. Part I, "For Developers", provides an introduction to HyperDex suitable for developers looking to build applications on top of HyperDex. The "For Developers" part of this book is written as a series of tutorials that acquaint you with HyperDex's API and describe many of the features available within HyperDex.

Part II, "For Administrators", describes the day-to-day maintenance of a HyperDex cluster. The "For Administrators" portion of this book is written as a series of small, isolated recipes that provide instruction for maintaining and monitoring HyperDex clusters.

Finally, Part III provides an API reference for HyperDex. This section serves primarily as a reference of the developer-visible, and operations-visible, components of HyperDex and should probably be used for reference rather than read in its entirety.

This book is a continuous work in progress, and newer versions are available at the HyperDex website. The book's sources are maintained alongside the HyperDex source code as the de-facto documentation for HyperDex. Patches, suggestions, and feedback about this book are welcome and may be submitted using the contact information in the next section.

1.3 Help and Support

If you run into trouble while working with HyperDex, don't panic! The HyperDex team have put together many support avenues to assist you. Many of the resources are available online for free. When you need more support than that, the HyperDex team is on standby with commercial support options that ensure that you're never left in the dark.

1.3.1 Online and Free Resources

The HyperDex team have assembled several free online resources to help you make the most of HyperDex:

- HyperDex Reference Manual: This document is the best avenue for support and assistance. It serves as the canonical documentation of HyperDex and should be your first stop with any questions or problems.
- HyperDex Mailing List: This open mailing list is the first resource most users turn to when solving problems. Messages to the list are archived which makes it easy to search for solutions to your problem. If you don't find a solution in the archives, start a new thread with your question and the HyperDex developers will try to help answer your questions.
- HyperDex IRC Channel: The `#hyperdex` IRC channel on Freenode is a great place to ask questions and interact with the HyperDex developers in real-time.
- HyperDex Bug Tracker: If you've encountered an error in the code that you think may be a bug, check the bug tracker to see if other users have reported the same problem. Consider reporting the problem yourself if it looks like the problem has not yet been reported.
- HyperDex FAQ: If you've got a question, check out the FAQ for frequently asked (and answered) questions. The FAQ contains pointers to other resources to help solve your problems.
- HyperDex Papers: If you're interested in developing or changing the internals of HyperDex, we've published papers that detail the internal architecture and design principles of HyperDex.

1.3.2 Commercial Support

Should you require more support, the HyperDex team is always on standby with commercial support options to ensure you're never left in the dark. If you're in need of support, you can contact the HyperDex developers directly at support@hyperdex.org.

Chapter 2

Installing HyperDex

HyperDex provides multiple installation methods for a variety of environments. Those looking to play around with HyperDex and go through the tutorials may find it easiest to start with the quickstart Docker container. For Linux users, the easiest and most convenient way to install HyperDex is to use pre-compiled binary packages. OS X users have the option to install using Homebrew packages. Finally, HyperDex is also distributed as a set of tarballs so that power users may install HyperDex in a custom manner on their platform of choice.

2.1 Quick Installation with Docker

HyperDex supports running within Docker containers for easy deployment. With Docker, an application is isolated in its own container, unable to change the workstation outside the container. This makes it easy to rapidly deploy new applications without worrying about having to cleanup or revert any changes they may make to the OS. This sample environment is fully feature-complete, and applications developed against such an environment will work with distributed and sharded HyperDex clusters with zero application level changes.

For HyperDex, we can leverage containers to setup short-lived clusters that may be used for one-off tasks, rapid prototyping, or for going through the HyperDex tutorial—all without any commitment or modification to our base OS. The `hyperdex/quickstart` container includes everything necessary to run a single-node HyperDex cluster, including the coordinator, daemon, and Python bindings.

Running the HyperDex quickstart container is a simple three-step process:

1. We'll need Docker installed to use this container, which may be done by following the Docker installation instructions.
2. We then fetch the HyperDex with the command `docker pull hyperdex/quickstart`.
3. To start the cluster, run `docker run --net=host -t -i hyperdex/quickstart`. When all goes well, you'll see the following message indicating your cluster is ready for use:

```
...
daemon: I1230 19:59:50.547586    21 daemon.cc:492] reconfiguration complete...
The transient HyperDex cluster is now online.

This is a transient HyperDex cluster.
You can connect to this cluster at address=172.17.3.27, port=1982.
```

With this quickstart cluster, both the coordinator and daemon logs will be displayed on stdout. You can connect to the cluster at the address and port listed and can stop the cluster at any time using the command displayed on your console. When you stop the cluster, the HyperDex cluster will be destroyed, and taken offline automatically. To fully clean up any disk space in use by the cluster, run `docker rm b6ae7c60c275`, where b6ae7c60c275 is the ID of your quickstart container.

2.2 Installing on CentOS/RedHat Using Binary Packages

The following code snippet will install HyperDex on any 64-bit CentOS 6 or 7 system. The script configures the HyperDex repository, imports the HyperDex signing key, and installs the `hyperdex` package. This script is available in the HyperDex tarball or repository at `doc/install/centos-packages.sh`.

```
cat >> /etc/yum.conf << EOF
[hyperdex]
name=hyperdex
baseurl=http://centos.hyperdex.org/base/\$basearch/\$releasever
enabled=1
gpgcheck=1
EOF
rpm --import http://centos.hyperdex.org/hyperdex.gpg.key
yum install -y epel-release
yum install -y hyperdex
```

Note: Users installing on Red Hat Enterprise Linux may have to follow the instructions to install Extra Packages for Enterprise Linux [1] prior to running the above code.

2.3 Installing on Debian Using Binary Packages

The following code snippet will install HyperDex on any 64-bit Debian 7 (Wheezy) system. The script configures the HyperDex repository, imports the HyperDex signing key, and installs the `HyperDex` package. This script is available in the HyperDex tarball or repository at `doc/install/debian7-packages.sh`.

```
wget -O - http://debian.hyperdex.org/hyperdex.gpg.key | apt-key add -
cat >> /etc/apt/sources.list.d/hyperdex.list << EOF
deb [arch=amd64] http://debian.hyperdex.org wheezy main
EOF
apt-get update
apt-get install -y hyperdex
```

2.4 Installing on Ubuntu Using Binary Packages

The following code snippets will install HyperDex on 64-bit Ubuntu systems. Pick the script that corresponds to the distribution you are running. Each script configures the HyperDex repository, imports the HyperDex signing key, and installs the `HyperDex` package. To figure out which version of Ubuntu you are using, `cat /etc/os-release` and look for the corresponding version and codename.

2.4.1 Ubuntu 12.04 Precise Pangolin

This script is available in the HyperDex tarball or repository at `doc/install/ubuntu12.04-packages.sh`.

```
wget -O - http://ubuntu.hyperdex.org/hyperdex.gpg.key | apt-key add -
cat >> /etc/apt/sources.list.d/hyperdex.list << EOF
deb [arch=amd64] http://ubuntu.hyperdex.org precise main
EOF
apt-get update
apt-get install -y hyperdex
```

[1] https://fedoraproject.org/wiki/EPEL

2.4.2 Ubuntu 14.04 Trusty Tahr

This script is available in the HyperDex tarball or repository at `doc/install/ubuntu14.04-packages.sh`.

```
wget -O - http://ubuntu.hyperdex.org/hyperdex.gpg.key | apt-key add -
cat >> /etc/apt/sources.list.d/hyperdex.list << EOF
deb [arch=amd64] http://ubuntu.hyperdex.org trusty main
EOF
apt-get update
apt-get install -y hyperdex
```

2.5 Installing on Fedora Using Binary Packages

The following code snippet will install HyperDex on any 64-bit Fedora 20 system. The script configures the HyperDex repository, imports the HyperDex signing key, and installs the `hyperdex` package. This script is available in the HyperDex tarball or repository at `doc/install/fedora-packages.sh`.

```
cat >> /etc/yum.conf << EOF
[hyperdex]
name=hyperdex
baseurl=http://fedora.hyperdex.org/base/\$basearch/\$releasever
enabled=1
gpgcheck=1
EOF
rpm --import http://fedora.hyperdex.org/hyperdex.gpg.key
yum install -y hyperdex
```

2.6 Installing on Linux Using Universal Binaries

HyperDex is also available as a self-contained universal binary that works on most glibc-based Linux distributions. To use this installation, first download the requisite tarball, and then you can install HyperDex like this:

```
tar xzvf hyperdex-1.8.dev-linux-amd64.tar.gz -C /usr/local
export PATH=/usr/local/hyperdex/bin:${PATH}
hyperdex --version
```

2.7 Installing on OS X Using Homebrew

HyperDex is available on Mac OS X using the Homebrew package manager. The following script, available in the HyperDex tarball or repository at `doc/install/homebrew.sh` may be used to install HyperDex on OS X:

```
brew tap HyperDex/hyperdex && brew install hyperdex
```

2.8 Installing From Source

Installing from source tarballs is a straightforward process that works on most any recent Linux distribution. Using the source tarballs provides power users with control over the installation process and the opportunity to customize the HyperDex installation.

To install from source, we must first prepare the system by installing a compiler, and dependencies that are not managed by the HyperDex team. This process will be highly system specific, but should be familiar

to most users who install from source. The HyperDex team and community are always happy to provide assistance to users installing from source.

On systems like Ubuntu, we can prepare the system for the from-source install with these commands:

```
sudo apt-get install -y build-essential libgoogle-glog-dev \
    libpopt-dev make pkg-config wget

wget https://github.com/jedisct1/libsodium/releases/download/1.0.1/libsodium-1.0.1.tar.gz
tar xzvf libsodium-1.0.1.tar.gz
cd libsodium-1.0.1
./configure
make
sudo make install
```

With the prereqs installed, installing HyperDex from tarballs may be accomplished with this script:

```
TARBALL_PO6=libpo6-0.9.dev
TARBALL_E=libe-0.12.dev
TARBALL_BUSYBEE=busybee-0.7.dev
TARBALL_LEVELDB=hyperleveldb-1.2.2
TARBALL_REPLICANT=replicant-0.8.dev
TARBALL_MACAROONS=libmacaroons-0.3.0
TARBALL_TREADSTONE=libtreadstone-0.3.dev
TARBALL_HYPERDEX=hyperdex-1.8.dev

for tarball in \
    ${TARBALL_PO6} ${TARBALL_E} ${TARBALL_BUSYBEE} \
    ${TARBALL_LEVELDB} ${TARBALL_REPLICANT} ${TARBALL_MACAROONS} \
    ${TARBALL_TREADSTONE} ${TARBALL_HYPERDEX}
do
    tar xzvf ${tarball}.tar.gz
    pushd ${tarball}
    ./configure
    make
    make install
    popd
done
```

2.8.0.1 Changing the Installation Directory

By default, HyperDex installs files in `/usr/local`. If you'd like to install HyperDex elsewhere, you can specify the installation prefix at configure time. For example, to install HyperDex in `/opt/hyperdex`:

```
export PKG_CONFIG_PATH=/opt/hyperdex/lib/pkgconfig
./configure --prefix=/opt/hyperdex
```

Check the `--help` option to configure for more ways to tune where HyperDex places files.

2.8.1 Troubleshooting

This section walks through some common installation problems that might be encountered while installing HyperDex from source.

2.8.1.1 Missing `libtool`

If your system is missing `libtool`, you'll see an error message like the following:

```
configure.ac:6: installing `./install-sh'
configure.ac:6: installing `./missing'
Makefile.am:48: Libtool library used but `LIBTOOL' is undefined
Makefile.am:48:   The usual way to define `LIBTOOL' is to add `LT_INIT'
Makefile.am:48:   to `configure.ac' and run `aclocal' and `autoconf' again.
Makefile.am:48:   If `LT_INIT' is in `configure.ac', make sure
Makefile.am:48:   its definition is in aclocal's search path.
Makefile.am: installing `./depcomp'
autoreconf: automake failed with exit status: 1
```

To correct this issue, install the `libtool` package using your distribution's package manager.

2.8.1.2 Missing `pkgconfig`

If your system is missing `pkgconfig`, you'll see an error message like the following:

```
./configure: line 18348: syntax error near unexpected token `PO6,'
./configure: line 18348: `PKG_CHECK_MODULES(PO6, libpo6 >= 0.3.1)'
```

To correct this issue, install the `pkg-config` package (sometimes called `pkgconfig`) using your distribution's package manager.

2.9 Verifying Installation

Once you have HyperDex installed, you should be able to view the built-in help. If the following commands provide meaningful output, then it is very likely that HyperDex is installed correctly and ready for use.

```
hyperdex help
hyperdex daemon --help
```

2.10 Upgrading to 1.6

The 1.6 release introduces new features that are backwards incompatible with previous releases, most notably a binary document format. Before upgrading to 1.6, backup your data. Once 1.6 is installed, copy your data into a newly-deployed 1.6 cluster.

2.11 Upgrading to 1.5

HyperDex 1.5 is backwards compatible with the 1.4 on-disk format. To upgrade from 1.4 to 1.5, take a backup and restart the cluster from the backup.

2.12 Upgrading to 1.4

Upgrading to 1.4 is a two step process. First, take a full backup of your cluster using the provided backup tools. Start the 1.4 cluster from the backup. This will upgrade the coordinator to 1.4, and the daemons will work with the old data directory.

Alternatively take a backup, and restart the coordinator from the backed-up coordinator state on the same address. Then do a rolling restart of the daemons.

The 1.4 release's on-disk format is backwards incompatible with the 1.3 and prior releases. HyperDex daemons will automatically upgrade to the 1.4 format when first launched.

2.13 Upgrading to 1.3

Upgrading to 1.3 is a two step process. First, take a full backup of your cluster using the provided backup tools. Start the 1.3 cluster from the backup. This will upgrade the coordinator to 1.3, and the daemons will work with the old data directory.

Alternatively take a backup, and restart the coordinator from the backed-up coordinator state on the same address. Then do a rolling restart of the daemons.

2.14 Upgrading to 1.2

The 1.2 release introduces new features that are backwards incompatible with previous releases. Before upgrading to 1.2, backup your data. Once 1.2 is installed, copy your data into a newly-deployed 1.2 cluster.

Part I

For Developers

Chapter 3

Quick Start

This chapter is designed to give you a whirlwind tour of HyperDex. The first section gives a high-level overview of a typical HyperDex deployment and application. Subsequent sections provide step-by-step instructions for how HyperDex could be used to build store data for a hypothetical phone book application.

3.1 High-Level Overview

A HyperDex cluster consists of three types of components: clients, servers, and coordinators.

Applications built on top of HyperDex are referred to as clients. and access HyperDex through various bindings for popular languages (e.g. C, C++, Java, and Python) to issue operations such as `get`, `put`, `search`, `delete`, etc.

The HyperDex servers are responsible for storing the data in the system. You can have a cluster with as few as just a single server, though typical installations will have dozens to hundreds of servers. These servers store the data in memory and on disk, and they can crash at any time. HyperDex can be configured to tolerate as many failed nodes as desired.

The HyperDex coordinator maintains the entire cluster. This involves making sure that servers are up, as well as detecting, removing, and replacing failed or slow nodes. The coordinator maintains a critical data structure, the hyperspace map, that establishes the mapping of data to servers. Clients use this map to locate the servers they need to contact, while servers use it to perform object propagation and replication to achieve the application's desired goals.

In this tutorial, we'll discuss how to get a HyperDex cluster up and running. In particular, we'll create a simple space, insert objects into it, retrieve those objects, and then perform queries over these objects.

3.2 Starting the Coordinator

First, we need to start up a coordinator that will oversee the organization of the hyperspace.

The following command starts and initializes the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

The "coordinator" command spawns a new HyperDex coordinator process listening on on the specified address. All other servers are introduced to the HyperDex cluster via this address, so it is important that it be accessible to all servers. For the purposes of this tutorial, binding to the loopback address will suffice, but you will probably want to use a public IP address for a real deployment

At this point, the coordinator is up and running, and we're ready to start up additional nodes in our cluster.

3.3 Starting HyperDex Daemons

The HyperDex daemons are the workhorse processes that actually house the data in the data store and respond to client requests. Let's start a daemon on the the same machine as the coordinator by running this in another terminal window:

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data
```

This command runs HyperDex in the foreground, listens for incoming connections on address 127.0.0.1:2012 and connects to the coordinator that we started earlier. This enables the daemon to announce its presence, which in turn enables the coordinator to assign partitions of data to the newly arrived daemon. For example, the coordinator can direct the daemon to take over some of the partitions from an existing daemon, participate in the data replication protocol, and start handling client requests. But since we have no data yet and this is our first node, this particular daemons does not have much work to do.

Note that the second argument (127.0.0.1) is the IP address to which we want to bind the daemon. Once again, we are using a loopback address here for the purposes of the tutorial, while in real life you will undoubtedly want to use one of the IP addresses bound to your internal network. If no address is specified, HyperDex will attempt to autodetect an external address.

The last argument is a pointer to a directory where the daemon will store all data. Each HyperDex daemon must have its own data directory.

It doesn't hurt to start a few more daemon instances at this point (although it is not required to continue with the tutorial).

You now have a functional HyperDex cluster. It's time to do something with it.

3.4 Creating a new Space

Let's imagine that we are building a phone book application. Such a phonebook would end up keeping track of people's first name, last name, and phone number. And to distinguish unique users, it might assign a user id to each user. In HyperDex, each such collection of data is called a space and is conceptually similar to tables in traditional databases. Let's see how we can instruct HyperDex to create a suitable space for holding such objects.

First, let's open a Python shell and connect to HyperDex via the Admin interface:

```
>>> import hyperdex.admin
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
```

This line instructs the admin bindings to talk to the coordinator and get the current cluster configuration. There is no need for static configuration files. Clients always receive the most up-to-date configuration, and if the configuration changes, say, due to failures, the coordinator will detect that a client is operating with an out-of-date configuration and instruct it to retry with a fresh config. The only thing a client must know to connect to a cluster is the address of the cluster coordinator.

Let's programmatically create a new space using the Python bindings:

```
>>> a.add_space('''
... space phonebook
... key username
... attributes first, last, int phone
... subspace first, last, phone
... create 8 partitions
... tolerate 2 failures
... ''')
True
```

This command creates a new space called "phonebook" that has a key of "username", and has attributes "first", "last", and "phone". By default, HyperDex treats every attribute as an opaque bytestring, but provides other types as well. Here, we specify that the phone number be treated as an integer. The available datatypes are discussed in Chapter 4.

Even though the objects we specify will have many attributes, HyperDex will not necessarily create one giant hyperdimensional space spanning as many dimensions as attributes. Doing so would not be desirable, as the space would be to large to efficiently map onto a cluster. Instead, we will typically want to create subspaces consisting of smaller numbers of dimensions. The lower number of dimensions enable the mapping from points in space to nodes in the cluster to be more efficient; in particular, fewer nodes need to be contacted during search operations. In this simple example, we create a 3-dimensional subspace for the first, last and phone attributes. HyperDex always implicitly creates a 1-dimensional subspace for the key of objects.

In other NoSQL systems, objects can only be retrieved efficiently by the key under which they were inserted. So an object ¡jsmith, John, Smith, 555-1234¿ can only be retrieved by its key "jsmith". Subspaces enable HyperDex to retrieve all "John" or "Smith" objects or, even, reverse lookups by phone number. The key serves as an object identifier so that objects may be retrieved or stored efficiently. Internally, the key is used to sequence updates and ensure consistency.

Even though we've only deployed one server in this example, we may want to leave room for future growth of our HyperDex cluster. The "create 8 partitions" line specifies that HyperDex will partition the resulting space into 8 regions. As a general rule, the number of partitions should be greater than the number of daemons that will ever join the cluster. It's perfectly acceptable to omit this line, and HyperDex will partition the cluster into 256 regions.

Since large scale cloud-computing deployments are sure to encounter failures, we will want to safeguard the data in our key-value store by replicating for fault tolerance. The "phonebook" space is configured to tolerate up to two concurrent failures ("tolerate 2 failures"). This instructs HyperDex to protect against up to two failures by creating three replicas of each object. Even if two daemons holding an object fail, there will still be one copy of the object remaining. HyperDex automatically repairs from this one remaining copy.

Finally, it's possible to create spaces using the command-line tools that ship with HyperDex. One could have created the "phonebook" space using the command-line:

```
hyperdex add-space -h 127.0.0.1 -p 1982 << EOF
   space phonebook
   key username
   attributes first, last, int phone
   subspace first, last, phone
   create 8 partitions
   tolerate 2 failures
EOF
```

3.5 Interacting with the "phonebook" Space

Now that we have our hyperspace defined and ready to go, it's time to insert some information into our phonebook.

Continuing in the same Python session, lets open a new Client connection to the cluster. HyperDex separates the metadata manipulation provided by the Admin library from the data manipulation provided by the Client library.

```
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
```

We can create an object by doing the following:

```
>>> c.put('phonebook', 'jsmith1', {'first': 'John', 'last': 'Smith',
...                                'phone': 6075551024})
True
```

This operation will determine the right spot in the hyperspace for this object and issue the `put` operation to all replicas. The operation will only return once the object has been committed at all requisite nodes.

We can easily retrieve the same "jsmith" object by using a standard `get`.

```
>>> c.get('phonebook', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'phone': 6075551024}
```

Yay! We inserted an object under the key "jsmith1" and retrieved it using the same key. This looks exactly like every other NoSQL store out there, but there are a few differences.

First, it's blazingly fast. You can look at our latest performance graphs for the precise comparisons, but typically, HyperDex is just way faster than other key-value stores.

Second, it's fault-tolerant. When we performed the `put`, our operation was sent through a value-dependent chain of daemons assigned to the object. The client received an acknowledgment only when the object was

replicated on every single server in the chain. Unlike NoSQL stores that optimistically assume that an update was committed before reaching all servers, HyperDex responds only when all daemons have been updated. And we can pick the replication level to achieve any level of fault tolerance we desire.

Finally, it's consistent. If we had multiple concurrent `put` operations being issued by multiple clients at the same time, we would never see an inconsistent state. What is an inconsistent state? It's what you get when you settle for eventual consistency. For instance, we would not want a prescription tracking system to say that we dispensed a drug, then to say we did not, only to settle on (say) having dispensed it. Yet this is precisely what might happen with an eventually consistent NoSQL key-value store. Eventual consistency is no consistency at all. In contrast, HyperDex provides linearizability. Time will never roll backwards from the point of any client.

And it gets better. Not only can we retrieve objects by their key, but we can also retrieve them when we don't know their key. Here are some examples:

```
>>> [x for x in c.search('phonebook', {'first': 'John'})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
>>> [x for x in c.search('phonebook', {'last': 'Smith'})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
```

Let's do that reverse phone number search:

```
>>> [x for x in c.search('phonebook', {'phone': 6075551024})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
```

Here's a fully-qualified search. Hyperspace hashing makes this nearly as fast as a key-based lookup:

```
>>> [x for x in c.search('phonebook',
...  {'first': 'John', 'last': 'Smith', 'phone': 6075551024})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
```

Let's add another user named "John Doe":

```
>>> c.put('phonebook', 'jd', {'first': 'John', 'last': 'Doe', 'phone': 6075557878})
True
>>> [x for x in c.search('phonebook',
...  {'first': 'John', 'last': 'Smith', 'phone': 6075551024})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
>>> [x for x in c.search('phonebook', {'first': 'John'})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'},
 {'first': 'John',
  'last': 'Doe',
  'phone': 6075557878,
  'username': 'jd'}]
>>> [x for x in c.search('phonebook', {'last': 'Smith'})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
>>> [x for x in c.search('phonebook', {'last': 'Doe'})]
[{'first': 'John',
  'last': 'Doe',
  'phone': 6075557878,
  'username': 'jd'}]
```

Should John Doe decide he no longer wants to be listed in the phonebook, it's trivial to remove his listing:

```
>>> c.delete('phonebook', 'jd')
True
>>> [x for x in c.search('phonebook', {'first': 'John'})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075551024,
  'username': 'jsmith1'}]
```

Suppose John Smith needs to change his phone number. This is easily accomplished by specifying just the key for the object and the changed attribute. All other attributes will be preserved.

```
>>> c.put('phonebook', 'jsmith1', {'phone': 6075552048})
True
>>> c.get('phonebook', 'jsmith1')
{'first': 'John',
  'last': 'Smith',
  'phone': 6075552048}
```

Smith is a popular name. Let's say there was "John Smith" from Rochester (area code 585):

```
>>> c.put('phonebook', 'jsmith2',
...           {'first': 'John', 'last': 'Smith', 'phone': 5855552048})
True
>>> c.get('phonebook', 'jsmith2')
{'first': 'John',
  'last': 'Smith',
  'phone': 5855552048}
```

Suppose we want to locate everyone named "John Smith" from Ithaca (area code 607). We can do this with a range query in HyperDex.

```
>>> [x for x in c.search('phonebook',
...  {'last': 'Smith', 'phone': (6070000000, 6080000000)})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075552048,
  'username': 'jsmith1'}]
```

Or perhaps we want to search for everyone whose name falls between "'Jack'" and "'Joseph'":

```
>>> [x for x in c.search('phonebook',
...  {'first': ('Jack', 'Joseph')})]
[{'first': 'John',
  'last': 'Smith',
  'phone': 6075552048,
  'username': 'jsmith1'},
 {'first': 'John',
  'last': 'Smith',
  'phone': 5855552048,
  'username': 'jsmith2'}]
```

3.6 Cleaning Up

When we're done with the "phonebook" space, we can delete the space and all the items in it directly from Python:

```
>>> a.rm_space('phonebook')
True
```

Chapter 4

Data Types

As we saw in the previous chapter, HyperDex offers support for the basic `get`, `put`, and `search` operations on strings and integers. In this chapter, we explore HyperDex's richer datatypes and atomic operations on lists, sets, and maps. We will see how providing efficient atomic operations on these rich, native datastructures greatly simplifies design for applications with complicated data layout requirements.

By the end of this chapter you'll be familiar with all datatypes provided by HyperDex, and a number of atomic operations.

4.1 Setup

As in the previous chapter, the first step is to deploy the cluster and connect a client. First we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a few daemon processes to store data. Execute the following commands (note that each instance binds to a different port and has a different `/path/to/data`):

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data1
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2013 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data2
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2014 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data3
```

This brings up three different daemons ready to serve in the HyperDex cluster. Finally, we create a space which makes use of all three systems in the cluster. In this example, let's create a space that may be suitable for storing profiles in a social network:

```
>>> import hyperdex.admin
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
>>> a.add_space('''
... space profiles
... key username
... attributes
...    string first,
...    string last,
...    int profile_views,
...    list(string) pending_requests,
...    set(string) hobbies,
...    map(string, string) unread_messages,
...    map(string, int) upvotes
... subspace first, last
... subspace profile_views
... ''')
True
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
```

Finally, let's create a profile for John Smith that we can use in the rest of this chapter.

```
>>> c.put('profiles', 'jsmith1', {'first': 'John', 'last': 'Smith'})
True
>>> c.get('profiles', 'jsmith1')
{'first': 'John', 'last': 'Smith',
 'profile_views': 0,
 'pending_requests': [],
 'hobbies': set([]),
 'unread_messages': {},
 'upvotes': {}}
```

4.2 Strings

The basic datatype in HyperDex is a byte string. If you don't specify the type of an attribute when creating a space, it is automatically treated as an 8-bit bytestring. This means that you'll have to encode and decode unicode strings as appropriate. For example, if John wanted to add an accent to his name on his social network page, the code could look like:

```
>>> c.put('profiles', 'jsmith1', {'first': u'Jóhn'.encode('utf8')})
True
```

This encodes the string to raw bytes using UTF-8. When fetching his profile it is necessary to decode the UTF-8:

```
>>> c.get('profiles', 'jsmith1')['first']
'J\xc3\x83\xc2\xb3hn'
>>> c.get('profiles', 'jsmith1')['first'].decode('utf8')
u'Jóhn'
>>> c.put('profiles', 'jsmith1', {'first': 'John', 'last': 'Smith'})
True
```

Of course, it's always possible to change John's name back to its unaccented form:

```
>>> c.put('profiles', 'jsmith1', {'first': 'John', 'last': 'Smith'})
True
>>> c.get('profiles', 'jsmith1')['first']
'John'
```

HyperDex knows nothing about encodings, so it is up to the application to encode or decode data appropriately.

4.3 Integers

As we've already seen, HyperDex supports `get` and `put` operations on integers. In addition to these basic operations, HyperDex provides atomic operations to manipulate integers using basic math operations. This is useful when implementing features such as page-view counters. Let's add support for tracking the profile views of a page by incrementing the counter:

```
>>> c.atomic_add('profiles', 'jsmith1', {'profile_views': 1})
True
>>> c.get('profiles', 'jsmith1')
{'first': 'John', 'last': 'Smith',
 'profile_views': 1,
 'pending_requests': [],
 'hobbies': set([]),
 'unread_messages': {},
 'upvotes': {}}
```

Note that this change required just one request to HyperDex. The server atomically examines the current value, and changes it by the amount specified. In this case, the `profile_views` attribute is incremented by one.

HyperDex supports a full range of basic operations including:

- `atomic_add`
- `atomic_sub`
- `atomic_mul`
- `atomic_div`
- `atomic_mod`
- `atomic_and`
- `atomic_or`
- `atomic_xor`

4.4 Floats

HyperDex also supports double-precision floating point types. Like integers, floats support range searches and atomic operations.

4.5 Lists

Let's add support for friend requests using HyperDex lists as the basis of the feature. For this we'll use the `pending_requests` attribute in the `profiles` space.

Imagine that shortly after joining, John Smith receives a friend request from his friend Brian Jones. Behind the scenes, this could be implemented with a simple list operation, pushing the friend request onto John's `pending_requests`:

```
>>> c.list_rpush('profiles', 'jsmith1', {'pending_requests': 'bjones1'})
True
>>> c.get('profiles', 'jsmith1')['pending_requests']
['bjones1']
```

The operation `list_rpush` is guaranteed to be performed atomically, and will be applied consistently with respect to all other operations on the same list.

4.6 Sets

Our social networking app captures the notion of hobbies. A set of strings is a natural representation for a user's hobbies, as each hobby is represented just once and may be added or removed.

Let's add some hobbies to John's profile:

```
>>> hobbies = set(['hockey', 'basket weaving', 'hacking',
...                'air guitar rocking'])
>>> c.set_union('profiles', 'jsmith1', {'hobbies': hobbies})
True
>>> c.set_add('profiles', 'jsmith1', {'hobbies': 'gaming'})
True
>>> c.get('profiles', 'jsmith1')['hobbies']
set(['hacking', 'air guitar rocking', 'hockey', 'gaming', 'basket weaving'])
```

If John Smith decides that his life's dream is to just write code, he may decide to join a group on the social network filled with like-minded individuals. We can use HyperDex's intersect primitive to narrow down his interests:

```
>>> c.set_intersect('profiles', 'jsmith1',
...                 {'hobbies': set(['hacking', 'programming'])})
True
>>> c.get('profiles', 'jsmith1')['hobbies']
set(['hacking'])
```

Notice how John's hobbies become the intersection of his previous hobbies and the ones named in the operation.

Overall, HyperDex supports simple set assignment (using the `put` interface), adding and removing elements with `set_add` and `set_remove`, taking the union of a set with `set_union` and storing the intersection of a set with `set_intersect`.

4.7 Maps

Lastly, our social networking system needs a means for allowing users to exchange messages. Let's demonstrate how we can accomplish this with the `unread_messages` attribute. In this contrived example, the `unread_messages` attribute is a map of usernames to unread messages from that user. For example:

```
>>> c.map_add('profiles', 'jsmith1',
...           {'unread_messages' : {'bjones1' : 'Hi John'}})
True
>>> c.map_add('profiles', 'jsmith1',
...           {'unread_messages' : {'timmy' : 'Lunch?'}})
True
>>> c.get('profiles', 'jsmith1')['unread_messages']
{'timmy': 'Lunch?', 'bjones1': 'Hi John'}
```

HyperDex enables map contents to be modified in-place within the map. For example, if Brian sent another message to John, we could separate the messages with "|" and just append the new message:

```
>>> c.map_string_append('profiles', 'jsmith1',
...                     {'unread_messages' : {'bjones1' : ' | Want to hang out?'}})
True
>>> c.get('profiles', 'jsmith1')['unread_messages']
{'timmy': 'Lunch?', 'bjones1': 'Hi John | Want to hang out?'}
```

Note that maps may have strings or integers as values, and every atomic operation available for strings and integers is also available in map form, operating on the values of the map.

For the sake of illustrating maps involving integers, let's imagine that we will use a map to keep track of the like/dislike's on John's status updates.

First, let's create some counters that will keep the net count of up and down votes corresponding to John's link posts, with ids "http://url1.com" and "http://url2.com".

```
>>> url1 = "http://url1.com"
>>> url2 = "http://url2.com"
>>> c.map_add('profiles', 'jsmith1',
...           {'upvotes' : {url1 : 1, url2: 1}})
True
```

So, John's posts start out with a counter set to 1, as shown above.

Imagine that two other users, Jane and Elaine, upvote John's first link post, we would implement it like this:

```
>>> c.map_atomic_add('profiles', 'jsmith1', {'upvotes' : {url1: 1}})
True
>>> c.map_atomic_add('profiles', 'jsmith1', {'upvotes' : {url1: 1}})
True
```

Charlie, sworn enemy of John, can downvote both of John's urls like this:

```
>>> c.map_atomic_add('profiles', 'jsmith1', {'upvotes' : {url1: -1, url2: -1}})
True
```

This shows that any map operation can operate atomically on a group of map attributes at the same time. This is fully transactional; all such operations will be ordered in exactly the same way on all replicas, and there is no opportunity for divergence, even through failures.

Checking where we stand:

```
>>> c.get('profiles', 'jsmith1')['upvotes']
{'http://url1.com': 2, 'http://url2.com': 0}
```

All of the preceding operations could have been issued concurrently – the results will be the same because they commute with each other and are executed atomically.

4.8 Timestamps

HyperDex has a special set of data types for storing timestamps. While one could store timestamps as a simple integer, perhaps seconds since the unix epoch, such a strategy would not work well if the timestamps were close relative to the range of an integer. HyperDex's timestamp types use a special hash function designed for timestamps to spread hash values evenly throughout the hash space, while retaining a customizable degree of locality in the data.

```
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
>>> a.add_space('space events_by_second key timestamp(second) when attribute body')
True
```

In this space, the key-space will be broken into one second intervals, and timestamps that fall within the same interval will be hashed to nearby to each other, while timestamps that map to different intervals will be hashed further apart in the key space. Each object is stored under the `when` attribute.

The HyperDex timestamp type appears to our application as a normal Python `datetime.datetime` object, and will be converted to and from this type by the Python API. Here's some example events that we can insert into our space:

```
>>> import datetime
>>> c.put('events_by_second', datetime.datetime(2015, 1, 7, 21, 40, 49),
...       {'body': 'the first event'})
...
True
>>> c.put('events_by_second', datetime.datetime(2015, 1, 7, 21, 40, 49, 1000),
...       {'body': 'this event happens 1ms after the first'})
...
True
>>> c.put('events_by_second', datetime.datetime(2014, 1, 7, 21, 40, 49),
...       {'body': 'this event happens a year earlier than the first'})
...
True
>>> c.put('events_by_second', datetime.datetime(2015, 1, 7, 21, 40, 51),
...       {'body': 'yet another event in our system, 2s after the first'})
...
True
```

In this sample data, our first two events happen exactly 1ms apart, but in the same second, so they will hash very closely in the key space—likely to the exact same server. The other events will be far apart from the first, and will likely reside on different servers in a cluster with more than a few servers.

Once our data is loaded, we can easily search using standard HyperDex comparison predicates like these:

```
>>> from hyperdex.client import * # for the predicates below
>>> # Search for everything in 2014
>>> [x for x in c.search('events_by_second',
...  {'when': [GreaterEqual(datetime.datetime(2014, 1, 1)),
...            LessThan(datetime.datetime(2015, 1, 1))]})]
...
[{'when': datetime.datetime(2014, 1, 7, 21, 40, 49),
  'body': 'this event happens a year earlier than the first'}]
>>> # Search for everything in 2015
>>> [x for x in c.sorted_search('events_by_second',
...  {'when': [GreaterEqual(datetime.datetime(2015, 1, 1)),
...            LessThan(datetime.datetime(2016, 1, 1))]},
...   'when', 10, 'min')]
...
[{'when': datetime.datetime(2015, 1, 7, 21, 40, 49),
  'body': 'the first event'},
 {'when': datetime.datetime(2015, 1, 7, 21, 40, 49, 1000),
  'body': 'this event happens 1ms after the first'},
 {'when': datetime.datetime(2015, 1, 7, 21, 40, 51),
  'body': 'yet another event in our system, 2s after the first'}]
```

4.8.1 Timestamp Intervals

When we created our space, we declared that we wanted the timestamp to be of type `timestamp(second)`. HyperDex supports six different timestamp types that spread the data across servers in different ways. The six types are:

- `timestamp(second)`: Intervals are one second in size.
- `timestamp(minute)`: Intervals are one minute in size.
- `timestamp(hour)`: Intervals are one hour in size.
- `timestamp(day)`: Intervals are one day in size.
- `timestamp(week)`: Intervals are 7 days in size.
- `timestamp(month)`: Intervals are 28 days in size.

Each of these types will roughly map timestamps that fall within the same time interval (e.g., second, month, etc.) nearby to each other in the key space. When deciding which timestamp type to use for our data, we should consider the data we'll store, and the queries we'll perform. In general, we wish to pick a timestamp type large enough such that range queries will fall within one unit of the interval. For example, if we'll be searching for objects that fall within a 24-hour window, it doesn't make sense to pick the second, minute, or hour types because timestamps from throughout the 24-hour window will be spread throughout the cluster. The day, week, and month timestamps make much more sense for this query workload.

Picking too large of a timestamp interval, however, can cause problems when inserting timestamps in real time. For example, if we were to always pick the `timestamp(month)` type, and insert one event every millisecond, the approximately 2.5 billion events inserted would map to the same server, and all writes would be directed to a different server each 28-day interval. If, instead, we picked the `timestamp(second)` type, writes would be directed to a different server each second, evenly consuming disk space across the cluster.

Chapter 5

Asynchronous Operations

Thus far, all HyperDex operations we've seen are synchronous. That is, the application calls the HyperDex operation and blocks until the operation is complete. HyperDex is very fast. But even so, clients may spend a non-trivial amount of time waiting on HyperDex operations. For each operation in HyperDex, there is an asynchronous variant that accomplishes the same task, while allowing the client to perform other work during operation completion.

In this chapter, we'll look at the asynchronous operations HyperDex supports, and see different ways to use them to increase concurrency in the system.

5.1 Setup

As in the previous chapters, the first step is to deploy the cluster and connect a client. First we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a few daemon processes to store data. Execute the following commands (note that each instance binds to a different port and has a different `/path/to/data`):

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data1
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2013 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data2
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2014 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data3
```

We now have three different daemons ready to serve in the HyperDex cluster. Finally, we create a space which makes use of all three systems in the cluster. In this example, let's create a space that may be suitable for storing friend lists in a social network:

```
>>> import hyperdex.admin
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
>>> a.add_space('''
... space friendlists
... key username
... attributes
...    string first,
...    string last,
...    set(string) friends
... ''')
True
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
```

Finally, our object for John Smith and some others

```
>>> c.put('friendlists', 'jsmith1', {'first': 'John', 'last': 'Smith'})
True
>>> c.put('friendlists', 'jd', {'first': 'John', 'last': 'Doe'})
True
>>> c.put('friendlists', 'bjones1', {'first': 'Brian', 'last': 'Jones'})
True
```

5.2 Asynchronous Operations

Asynchronous operations permit applications to retrieve or modify multiple objects simultaneously, and to perform local computation while doing the same. In previous chapters, we submitted synchronous operations to the key-value store, where each client had just a single outstanding request, and waited patiently for that request to complete. In high-throughput applications, clients may have a batch of operations that may be performed simultaneously. The standard practice in such cases is to issue *asynchronous* operations, where the client does not immediately wait for each individual operation to complete. HyperDex has a very versatile interface for supporting this use case.

Asynchronous operations separate the request and response portions of a single operation into two separate parts. Each asynchronous operation returns a small token that identifies the outstanding operation, which can then be used by the client, if and when needed, to wait for the completion of the selected operation.

Every API method covered in the tutorials so far (e.g. `get`) has a corresponding asynchronous version, usually prefixed with `async_` (e.g. `async_get`), for performing asynchronous operations. Those without an `async_` prefix are natively synchronous. The basic pattern of usage for asynchronous operations is:

* Initiate the asynchronous operation * Do some work and perhaps issue more operations, async or otherwise, * Wait for selected asynchronous operations to complete

This enables the application to continue doing other work while HyperDex performs the requested operations. Here's how we could insert an object for user John Jackson asynchronously:

```
>>> d = c.async_put('friendlists', 'jj', {'first': 'John', 'last': 'Jackson'})
>>> d
<hyperdex.client.Deferred object at ...>
>>> # do some work
>>> d.wait()
True
>>> d = c.async_get('friendlists', 'jj')
>>> d.wait()
{'first': 'John', 'last': 'Jackson', 'friends': set([])}
```

Notice that the return value of the first `d.wait()` is `True`. This is the same return value that would have come from performing `c.put(...)`, except the client was free to do other computations while HyperDex servers were processing the `put` request. Similarly, the second asynchronous operation, `async_get`, queues up the request on the servers, frees the client to perform other work, and yields its results only when `wait` is called. In fact, the Python bindings implement all synchronous operations using their asynchronous equivalents. For example, here's a sample definition of `get`:

```
>>> def get(client, space, key):
...     return client.async_get(space, key).wait()
...
>>> get(c, 'friendlists', 'jj')
{'first': 'John', 'last': 'Jackson', 'friends': set([])}
```

By itself, an asynchronous operation is not very useful if it is waited on right away because that makes it equivalent to a synchronous operation. The true power comes from requesting multiple concurrent operations. For example, to establish a bidirectional friendship between John Smith and John Jackson:

```
>>> d1 = c.async_set_add('friendlists', 'jj', {'friends': 'jsmith1'})
>>> d2 = c.async_set_add('friendlists', 'jsmith1', {'friends': 'jj'})
>>> d1.wait()
True
>>> d2.wait()
True
```

Note that the order in which operations are waited on does not matter. We could just as easily execute them in a different order, and still get the desired effect. Similarly, we could concurrently add multiple friends for John Smith:

```
>>> d1 = c.async_set_add('friendlists', 'jsmith1', {'friends': 'bjones1'})
>>> d2 = c.async_set_add('friendlists', 'bjones1', {'friends': 'jsmith1'})
>>> d3 = c.async_set_add('friendlists', 'jsmith1', {'friends': 'jd'})
>>> d4 = c.async_set_add('friendlists', 'jd', {'friends': 'jsmith1'})
>>> d1.wait()
True
>>> d2.wait()
True
>>> d3.wait()
True
>>> d4.wait()
True
```

This allows for powerful applications. Going a step further, HyperDex allows a client to wait for the next operation to complete, without specifying an order among asynchronous operations. For instance, fetching the first names of every friend of John can be done in parallel:

```
>>> friends_usernames = c.get('friendlists', 'jsmith1')['friends']
>>> outstanding = set()
>>> for username in friends_usernames:
...     outstanding.add(c.async_get('friendlists', username))
...
>>> friends = []
>>> while outstanding:
...     d = c.loop()
...     outstanding.remove(d)
...     friend = d.wait()['first']
...     friends.append(friend)
...
>>> sorted(friends)
['Brian', 'John', 'John']
```

Using the `loop()` method, it is possible to issue thousands of requests and then wait for each one in turn without having to serialize the round trips to the server. Each call to loop returns a `Deferred` object represents one outstanding operation.

5.3 Potential Pitfalls

Keep in mind that HyperDex may (and will!) maximize performance by executing concurrent asynchronous operations in any order. If a client wants to ensure that an operation B is performed only after asynchronous operation A is committed to the data store, that client needs to call wait() on A. When wait() returns success, the client is guaranteed that HyperDex has committed the data to sufficiently many replicas to tolerate the number of failures in the space definition.

On a related note, a client that has not explicitly performed a wait() on an outstanding asynchronous operation should not assume anything about the disposition of those operations. Specifically, the queued

asynchronous operations may not have made it to any of the servers, and therefore may not be committed anywhere. A client that simply issues asynchronous requests and terminates without calling wait() on them is not guaranteed to have any of those asynchronous operations execute. Unless the system explicitly tells a client that the data is committed, it is not safe to assume that it will be committed behind the scenes. (The flip-side of this is that, when HyperDex says the data is committed, it really is committed to sufficiently many replicas to withstand the number of simultaneous failures in the space specification).

A more subtle pitfall with asynchronous operations is that the order of concurrent asynchronous operations is not guaranteed, even if they are issued by the same client. If a client requires that operations be ordered in a particular manner, it must do so itself by ensuring that the dependent operation is not issued until all its dependencies complete successfully.

5.4 A Common Window Pattern

The last point necessitates a common pattern, where a client will want to keep a fixed-size window of outstanding requests so as to not issue too many operations concurrently, and will appropriately wait for all asynchronous operations to complete. Such code will look, approximately, like this:

```
c = hyperclient.Client(...)
outstanding = 0
for line in file:
    while outstanding >= 1024: # 1024 is window size
        d = c.loop()
        d.wait()
        outstanding -= 1
    parse line and issue put
    outstanding += 1
# flush remaining
while outstanding > 0:
    d = c.loop()
    d.wait()
    outstanding -= 1
```

Chapter 6

Atomic Operations

The operations we have discussed until now have consisted of lone read or write operations (though we did examine both synchronous and asynchronous versions of both). While such lone reads and write are very common, some applications require read-modify-write operations that execute atomically.

For instance, take an application that wants to count the number of times each word appears as it crawls the web. There will undoubtedly be many concurrent threads fetching web documents, checking the datastore for the current count of each word, inserting a "1" if that word has not been encountered before, and otherwise reading the count, incrementing it and storing the updated value.

It's easy to see that implementing such an application with lone reads and writes would immediately lead to an incorrect application. One possible scenario is that two crawlers will encounter the word at approximately the same time in two different documents. They will both check whether the word exists, note that it is not currently in the data store, and both crawlers will store a "1", whereas the correct count should be 2. Another possible scenario is that one crawler will encounter a word, read the current count, say 17, and then get delayed because of VM scheduling. In the meantime, a fast crawler goes through millions of documents, updating the count for that particular word to, say, 124,232. When the slow crawler resumes, it will blithely write an "18" for that word, losing more than 124,000 of the occurrences it should have been tracking.

Atomic read-modify-write operations provide a solution to this problem. Such operations are guaranteed to execute without interference from other operations. Atomic operations ensure that the read-modify-write sequence that comprises an operation is executed in a manner that cannot be interrupted by or interleaved with any other operation. The entire block is one atomic unit.

Let's see how they're used. The setup below is identical to the previous chapter, so if you have that space already set up, you can skip it.

6.1 Setup

As in the previous chapters, the first step is to deploy the cluster and connect a client. First we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a few daemon processes to store data. Execute the following commands (note that each instance binds to a different port and has a different `/path/to/data`):

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data1
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2013 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data2
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2014 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data3
```

We now have three different daemons ready to serve in the HyperDex cluster. Finally, we create a space which makes use of all three systems in the cluster. In this example, let's create a space that may be suitable for storing friend lists in a social network:

```
>>> import hyperdex.admin
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
>>> a.add_space('''
... space friendlists
... key username
... attributes
...    string first,
...    string last,
...    set(string) friends
... ''')
True
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
```

Finally, our object for John Smith and some others

```
>>> c.put('friendlists', 'jsmith1', {'first': 'John', 'last': 'Smith',
...                                  'friends': set(['bjones1', 'jd', 'jj'])})
True
>>> c.put('friendlists', 'jd', {'first': 'John', 'last': 'Doe'})
True
>>> c.put('friendlists', 'bjones1', {'first': 'Brian', 'last': 'Jones'})
True
```

6.2 Atomic Operations

HyperDex supports a few different types of atomic operations. Perhaps the most general one is the `cond_put`. A `cond_put` performs an `put` if and only if the value being updated matches a condition specified along with the new values to be inserted.

In our example, let's say the application wants to change John Smith's name to Jon Smith only if his name is unchanged, but wants the application to fail if the application has changed his name since it was last read:

```
>>> c.get('friendlists', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'friends': set(['bjones1', 'jd', 'jj'])}
>>> c.cond_put('friendlists', 'jsmith1',
...            {'first': 'John', 'last': 'Smith'},
...            {'first': 'Jon'})
True
>>> c.get('friendlists', 'jsmith1')
{'first': 'Jon', 'last': 'Smith', 'friends': set(['bjones1', 'jd', 'jj'])}
```

Here we told HyperDex to update John's name if and only if it is currently equal to "John". The third argument is a set of attributes that must match the object for the update to succeed. The fourth argument takes the same form as a typical `put`.

Not surprisingly, this request succeeded, as John's name matched the specified values. Let's try issuing the same operation again.

```
>>> c.cond_put('friendlists', 'jsmith1',
...            {'first': 'John', 'last': 'Smith'},
...            {'first': 'Jon'})
False
```

Notice that `cond_put` failed because the value of the first name field is no longer "John".

Note that the last argument has the same generality as the arguments to a regular `put` operation. So there is no requirement that a `cond_put` check and update the same field. The following is a perfectly legitimate operation that updates the first name field of Jon's profile if and only if his set of friends has not changed:

```
>>> c.cond_put('friendlists', 'jsmith1',
...            {'friends': set(['bjones1', 'jd', 'jj'])},
...            {'first': 'John'})
True
>>> c.get('friendlists', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'friends': set(['bjones1', 'jd', 'jj'])}
```

The great thing about HyperDex is that `cond_put` operations are fast. In fact, their performance is indistinguishable from a normal `put`, all else being equal. Thus, you can rely heavily upon `cond_put` operations to avoid race conditions without sacrificing performance. Going a step further, it's possible to use `async_cond_put` operations to achieve even more concurrency without losing correctness.

Keep in mind that `cond_put` operations can and will fail, as intended, if there are interceding operations that update the object fields that must match. In these cases, the client will typically want to re-fetch the object, re-perform its updates, and re-submit the conditional operation.

Another useful atomic primitive HyperDex provides is the `put_if_not_exist` operation. This operation succeeds if and only if the object does not already exist. This can be useful to implement locking behavior. For example, a simple lock space can be used to provide per-user locking:

```
>>> a.add_space('''
... space userlocks
... key username
... ''')
True
```

With this space, we can atomically check if an object exists, creating it in the process; or fail the operation, leaving the object unchanged.

```
>>> c.put_if_not_exist('userlocks', 'jsmith1', {})
True
>>> c.get('userlocks', 'jsmith1')
{}
>>> c.put_if_not_exist('userlocks', 'jsmith1', {})
False
>>> c.delete('userlocks', 'jsmith1')
True
```

Here's an illustrative example that shows how a test-and-set mechanism can be used to implement `lock()` and `unlock()`.

```
>>> def lock(client, user):
...     while not client.put_if_not_exist('userlocks', user, {}):
...         pass
>>> def unlock(client, user):
...     client.delete('userlocks', user)
>>> lock(c, 'jsmith1')
>>> unlock(c, 'jsmith1')
```

Note that a real implementation will not want to busy-loop, and will want to make provisions for when clients fail while holding the lock.

6.3 A Comprehensive Walk

Having built an intuition for how to structure and use the atomic operations, let's go through them and illustrate the various atomic operations for each of the different types. So, let's first create a space that can allow us to do this:

```
>>> a.add_space('''
... space alldatatypes
... key k
... attributes
...    string s,
...    int i,
...    float f,
...    list(string) ls,
...    set(string) ss,
...    map(string, string) mss,
...    map(string, int) msi''')
True
```

The key-based operations `put_if_not_exist` and `cond_put` can be used to create the object if it does not exist, and to modify it only if certain fields match expected values, respectively.

```
>>> c.put_if_not_exist('alldatatypes', 'somekey', {'s': 'initial value'})
True
>>> c.put_if_not_exist('alldatatypes', 'somekey', {'s': 'initial value'})
False

>>> # cond_put.  First is predicate.  May be any valid search predicate
>>> c.cond_put('alldatatypes', 'somekey', {'s': 'initial value'}, {'s': 'some string'})
True
>>> c.cond_put('alldatatypes', 'somekey', {'s': 'initial value'}, {'s': 'some string'})
False
```

Note how the first operations succeeds, and the second one fails. Let's check to make sure that our object reflects the changes we have applied:

```
>>> c.get('alldatatypes', 'somekey')
{'f': 0.0, 'i': 0, 'mss': {}, 'ss': set([]), 's': 'some string', 'ls': [], 'msi': {}}
```

The operation `cond_put_or_create` is a combination of `put_if_not_exist` and `cond_put`. If the object does not exist, it will create the object, but if the object exists it will be modified only if certain fields match expected values.

```
>>> c.cond_put_or_create('alldatatypes', 'anotherkey', {'s': 'a'}, {'s': 'b'})
True
>>> c.cond_put_or_create('alldatatypes', 'anotherkey', {'s': 'a'}, {'s': 'b'})
False
>>> c.get('alldatatypes', 'anotherkey')
{'f': 0.0, 'i': 0, 'mss': {}, 'ss': set([]), 's': 'b', 'ls': [], 'msi': {}}
>>> c.cond_put_or_create('alldatatypes', 'anotherkey', {'s': 'b'}, {'s': 'a'})
True
>>> c.get('alldatatypes', 'anotherkey')
{'f': 0.0, 'i': 0, 'mss': {}, 'ss': set([]), 's': 'a', 'ls': [], 'msi': {}}
```

Let's now perform some atomic operations on integers and floats. These are self-explanatory, so we'll let the code do the talking. You will note that the float "f" and integer "i" fields are the ones of interest here, the rest are non-changing:

```
>>> c.atomic_add('alldatatypes', 'somekey', {'i': 1, 'f': 0.25})
True
>>> c.get('alldatatypes', 'somekey')
{'f': 0.25, 'i': 1, 'mss': {}, 'ss': set([]), 's': 'some string', 'ls': [], 'msi': {}}

>>> c.atomic_sub('alldatatypes', 'somekey', {'i': 2, 'f': 0.5})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': -1, 'mss': {}, 'ss': set([]), 's': 'some string', 'ls': [], 'msi': {}}

>>> c.atomic_mul('alldatatypes', 'somekey', {'i': 2, 'f': 4.})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -1.0, 'i': -2, 'mss': {}, 'ss': set([]), 's': 'some string', 'ls': [], 'msi': {}}

>>> c.atomic_div('alldatatypes', 'somekey', {'i': 2, 'f': 4.})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': -1, 'mss': {}, 'ss': set([]), 's': 'some string', 'ls': [], 'msi': {}}

>>> c.put('alldatatypes', 'somekey', {'i': 0xdeadbeefcafe})
True
>>> c.atomic_and('alldatatypes', 'somekey', {'i': 0xffffffff0000})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 244837814042624, 'mss': {}, 'ss': set([]), 's': 'some string', 'ls': [], 'msi': {}}
>>> print "0x%x" % (c.get('alldatatypes', 'somekey')['i'],)
0xdeadbeef0000

>>> c.atomic_or('alldatatypes', 'somekey', {'i': 0x00000000cafe})
True
>>> print "0x%x" % (c.get('alldatatypes', 'somekey')['i'],)
0xdeadbeefcafe

>>> c.atomic_xor('alldatatypes', 'somekey', {'i': 0xdea5a0403af3})
True
>>> print "0x%x" % (c.get('alldatatypes', 'somekey')['i'],)
0x81eaff00d
```

Ok, now let's perform some atomic operations on strings:

```
>>> c.string_prepend('alldatatypes', 'somekey', {'s': '->'})
True
>>> c.get('alldatatypes', 'somekey')['s']
'->some string'

>>> c.string_append('alldatatypes', 'somekey', {'s': '<-'})
True
>>> c.get('alldatatypes', 'somekey')['s']
'->some string<-'
```

Atomic operations on lists allow the client to push new items on the left or right of the list:

```
>>> c.put('alldatatypes', 'somekey', {'ls': ['B']})
True
>>> c.list_lpush('alldatatypes', 'somekey', {'ls': 'A'})
True
>>> c.get('alldatatypes', 'somekey')['ls']
['A', 'B']

>>> c.list_rpush('alldatatypes', 'somekey', {'ls': 'C'})
True
>>> c.get('alldatatypes', 'somekey')['ls']
['A', 'B', 'C']
```

Sets provide the whole range of atomic set operations:

```
>>> c.set_add('alldatatypes', 'somekey', {'ss': 'C'})
True
>>> c.get('alldatatypes', 'somekey')['ss']
set(['C'])

>>> c.set_remove('alldatatypes', 'somekey', {'ss': 'C'})
True
>>> c.get('alldatatypes', 'somekey')['ss']
set([])

>>> c.set_union('alldatatypes', 'somekey', {'ss': set(['A', 'B', 'C'])})
True
>>> c.get('alldatatypes', 'somekey')['ss']
set(['A', 'C', 'B'])

>>> c.set_intersect('alldatatypes', 'somekey', {'ss': set(['A', 'B', 'Z'])})
True
>>> c.get('alldatatypes', 'somekey')['ss']
set(['A', 'B'])
```

Finally, we have maps. Maps provide two kinds of atomic operations: the kind that atomically manipulates the map itself, and the kind that atomically manipulates an element of the map.

Let's atomically add an element to the `mss` field while atomically adding another element, with the same key, to the `msi` field.

```
>>> c.map_add('alldatatypes', 'somekey', {'mss': {'mapkey': 'mapvalue'}, 'msi': {'mapkey': 16}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some strin
```

Let's add another field to one of the maps:

```
>>> c.map_add('alldatatypes', 'somekey', {'mss': {'tmp': 'delete me'}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'tmp': 'delete me', 'mapkey': 'mapvalue'}, 'ss': set(['A', 'B'])
```

Let's now atomically delete that field. We need only specify its key, the value does not matter for the `map_remove` operation.

```
>>> c.map_remove('alldatatypes', 'somekey', {'mss': 'tmp'})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some strin
```

Now we can perform all of the preceding atomic operations on each of the elements of the maps, atomically:

```
>>> c.map_atomic_add('alldatatypes', 'somekey', {'msi': {'mapkey': 16}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_atomic_sub('alldatatypes', 'somekey', {'msi': {'mapkey': -32}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_atomic_mul('alldatatypes', 'somekey', {'msi': {'mapkey': 4}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_atomic_div('alldatatypes', 'somekey', {'msi': {'mapkey': 64}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_atomic_and('alldatatypes', 'somekey', {'msi': {'mapkey': 2}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_atomic_or('alldatatypes', 'somekey', {'msi': {'mapkey': 5}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_atomic_xor('alldatatypes', 'somekey', {'msi': {'mapkey': 7}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': 'mapvalue'}, 'ss': set(['A', 'B']), 's': '->some string

>>> c.map_string_prepend('alldatatypes', 'somekey', {'mss': {'mapkey': '->'}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': '->mapvalue'}, 'ss': set(['A', 'B']), 's': '->some stri

>>> c.map_string_append('alldatatypes', 'somekey', {'mss': {'mapkey': '<-'}})
True
>>> c.get('alldatatypes', 'somekey')
{'f': -0.25, 'i': 34874585101, 'mss': {'mapkey': '->mapvalue<-'}, 'ss': set(['A', 'B']), 's': '->some s
```

6.3.1 Atomic Operations on Documents

Note that Hyperdex also supports atomic operations on documents. To do this, we first have to create a space that has a field of the type document.

```
>>> a.add_space('''
... space people
... key k
... attributes
...    document info''')
True
```

Now, we can insert data into this table.

```
>>> Document = hyperdex.client.Document
>>> c.put('people', 'jane', {'info' : Document( {'name': 'Jane Doe', 'gender' : 'female', 'age' : 21, '
True
```

We might now want to modify this information of this person without rewriting the whole document. For example, it might be Jane Doe's birthday and we want to increment the age. Fortunately, atomic operations also work on this kind of data. HyperDex is able to increment integers within documents by providing the full path to the integer within the document.

```
>>> c.atomic_add('people', 'jane', {'info.age' : 1})
True
>>> c.get('people', 'jane')
{'info': Document({"name": "Jane Doe", "gender": "female", "age": 22, "likes": ["cornell", "python"]})}
```

HyperDex will also prevent you from adding to non-numeric fields.

```
>>> c.atomic_add('people', 'jane', {'info.gender' : 1})
False
```

You can also add new entries to the document using this method.

```
>>> c.atomic_add('people', 'jane', {'info.children' : 1})
True
>>> c.get('people', 'jane')
{'info': Document({"name": "Jane Doe", "gender": "female", "age": 22, "children": 1, "likes": ["cornell
```

Like for maps, there are other possible atomic operations. You might want to modify the name by prepending a title.

```
>>> c.string_prepend('people', 'jane', {'info.name' : 'Dr. '})
True
>>> c.get('people', 'jane')
{'info': Document({"name": "Dr. Jane Doe", "gender": "female", "age": 22, "children": 1, "likes": ["cor
```

Similarly, we can append text.

```
>>> c.string_append('people', 'jane', {'info.name' : ', Jr.'})
True
>>> c.get('people', 'jane')
{'info': Document({"name": "Dr. Jane Doe, Jr.", "gender": "female", "age": 22, "children": 1, "likes":
```

HyperDex's atomic operations are extensive and very expressive. They are guaranteed to be applied in the same order on all replicas, so the state of the object you are operating on is guaranteed to be the same, regardless of failovers.

Chapter 7

Document Storage

Thus far, we have assumed that you know the structure of your data in advance. What do you do when you don't know what your data looks like in advance? HyperDex provides a document type for just this scenario. Documents are JSON objects, stored within HyperDex, that can be updated and queried just like regular objects.

Let's see how to use documents in practice. The setup below is similar to the previous chapter, so if you have a running cluster, you can skip to the space creation step.

7.1 Setup

As in the previous chapters, the first step is to deploy the cluster and connect a client. First we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a daemon process to store data. Execute the following command:

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data
```

We now have a HyperDex cluster ready to serve our data. Finally, we create a space which makes use of the cluster. In this example, let's create a space that may be suitable for storing social network profiles.

```
>>> import hyperdex.admin
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
>>> a.add_space('''
... space profiles
... key username
... attributes
...    document profile
... ''')
True
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
```

7.2 Working with Documents

It's easy to see how documents enable a wide array of applications. Consider a social networking application that stores each user's profile as a document in HyperDex. Users which provide very little information to the social network will have a relatively sparse profile like this:

```
{"name": "John Smith"}
```

You can store this simple document almost exactly like you would store any other data in HyperDex.

```
>>> Document = hyperdex.client.Document
>>> c.put('profiles', 'jsmith1', {'profile': Document({"name": "John Smith"})})
True
```

Of course, nothing prevents users from having more-complex versions of the profile like so:

```
>>> c.put('profiles', 'jd', {'profile': Document({
...     "name": "John Doe",
...     "www": "http://example.org",
...     "email": "doe@example.org",
...     "friends": ["John Smith"]
... })})
True
```

You can search over documents just like you can search over regular HyperDex attributes. For example, to retrieve the objects for all people named John Doe, you can do a search on `profile.name` to retrieve all such objects:

```
>>> print [x for x in c.search('profiles', {'profile.name': 'John Doe'})]
[{'username': 'jd', 'profile': Document({"www": "http://example.org", "friends":
["John Smith"], "name": "John Doe", "email": "doe@example.org"})}]
```

You can even do HyperDex's more-complex queries. For example, to find everyone whose name starts with "John", you can do a regular expression search like this:

```
>>> print [x for x in c.search('profiles', {'profile.name': hyperdex.client.Regex('John')})]
[{'username': 'jsmith1', 'profile': Document({"name": "John Smith"})}, {'username': 'jd', 'profile': Do
```

7.3 Indexing Documents

Documents, by their very nature, impose no structure on the data they contain. Your application is free to store any JSON it wants as a document, and HyperDex will search it just fine. Sometimes though, your documents do have some structure, and your queries will often look at the same elements of the documents. For these situations, it is possible to construct an index on the document that significantly speeds up many popular search queries.

With our example queries above, we could create an index on the `profile.name` attribute. This index may be consulted by equality and range searches that include the `profile.name` element.

To create this index, use the `add-index` command, specifying the space "profiles" and the attribute "profile.name".

```
hyperdex add-index profiles profile.name
```

This will instruct HyperDex to add the new index, and will index all existing data using the new index. You can add and remove indices at any time as your documents change.

HyperDex will automatically make use of the new index upon its creation.

Chapter 8

Authorization

In previous chapters, we developed applications that have complete access to all data stored within HyperDex. For some applications, the stored data is sensitive and needs to be protected against unauthorized reads and writes. The compromise of a single web-facing client, for instance, should not allow the attacker to access all of the database.

HyperDex provides a fine-grained authorization mechanism that enables the system to restrict clients' access to data. Authorization in HyperDex uses macaroons, a new decentralized authorization framework developed by Google for use in distributed systems. Macaroons enable an application developer to enforce security policies per object, as opposed to per table or per database. This ensures that a client compromise does not lead to loss of the entire database. Further, macaroons are very flexible and expressive, able to capture such security policies as "allow access to this object only if the client is accessing it on behalf of an employee, and only between the hours of 9am and 5pm." Finally, macaroons scale well and can integrate external authentication mechanisms naturally.

Let's go through a quick tutorial that demonstrates the use and power of macaroons. We'll start slow, show you some familiar operations, and build up to an example towards the end where we implement a rich security policy that can only be expressed using Macaroons.

8.1 Setup

As in the previous chapters, the first step is to deploy the cluster and connect a client. The cluster setup below is similar to the previous chapters, so if you have a running cluster, you can skip to the space creation step.

First, we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a daemon process to store data:

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data
```

We now have a HyperDex cluster ready to serve our data. Now, we create a space and declare that we will be using macaroons.

```
>>> import hyperdex.admin
>>> a = hyperdex.admin.Admin('127.0.0.1', 1982)
>>> a.add_space('''
... space accounts
... key account
... attributes
...    string name,
...    int balance
... with authorization
... ''')
True
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
```

The added statement, "with authorization," indicates to HyperDex that we wish to have Macaroons enabled for this space.

Now that the space is ready, let's create some objects.

8.2 Using Macaroons

To use macaroons, every object we wish to protect needs to be protected by a secret that will act as the master key to that object. Think of this key as a shibboleth of sorts. Any client that is able to whisper this secret word is able to access the object, as if they created it. In essence, it acts as a master secret, a capability, from which all other lesser capabilities are derived.

```
>>> SECRET = 'super secret password'
>>> account = 'account number of john smith'
>>> c.put('accounts', account, {'name': 'John Smith', 'balance': 10}, secret=SECRET)
True
```

Once an object has an associated secret, attempts to retrieve that object will fail unless accompanied by this secret:

```
>>> c.get('accounts', account)
Traceback (most recent call last):
HyperDexClientException: ... it is unauthorized [HYPERDEX_CLIENT_UNAUTHORIZED]
```

To access the object, the application must present a macaroon that proves to HyperDex that the request is authorized. Such a macaroon is called a root macaroon. The root macaroon demonstrates knowledge of the master secret that protects the object.

Root macaroons are created by specifying the secret and converting them into portable tokens. Under the covers, these tokens do not actually carry the secret (for if they did, someone could reverse-engineer a macaroon and obtain unfettered access to the object), but instead carry an irreversible hash of the secret. They are typically created by the person who creates the object, and are passed onto clients that need to access it.

Let's create a root macaroon from scratch:

```
>>> import macaroons
>>> M = macaroons.create('account number', SECRET, '')
>>> token = M.serialize()
```

In this case, M is the root macaroon, and token is the serialized version of that macaroon that can be passed around easily. This macaroon provides full access to John Smith's account, and may be used to read the account information or update the account balance.

Now that we have a macaroon that proves that we possess the secret, we can use it to gain access to the object:

```
>>> c.get('accounts', account, auth=[token])
{'name': 'John Smith', 'balance': 10}
>>> c.atomic_add('accounts', account, {'balance': 5}, auth=[token])
True
>>> c.get('accounts', account, auth=[token])
{'name': 'John Smith', 'balance': 15}
```

While this basic example shows how to use macaroons, it doesn't fully exploit their power. The true power of macaroons stems from the ability to embed caveats into macaroons. A caveat is essentially a restriction on what the macaroon authorizes; it turns a full object capability into a restricted capability.

For instance, in our running example, we might want to allow the clients to read the account balance while prohibiting them from altering it. We can easily create a read-only macaroon to accomplish this:

```
>>> M = macaroons.create('account number', SECRET, '')
>>> M = M.add_first_party_caveat('op = read')
>>> token = M.serialize()
```

This new macaroon has the caveat that it is useful solely for read operations. Attempts to write with this macaroon will fail, as desired:

```
>>> c.get('accounts', account, auth=[token])
{'name': 'John Smith', 'balance': 15}
>>> c.atomic_add('accounts', account, {'balance': 5}, auth=[token])
Traceback (most recent call last):
HyperDexClientException: ... it is unauthorized [HYPERDEX_CLIENT_UNAUTHORIZED]
```

Macaroon caveats can be stacked or chained on top of each other, to create arbitrarily restricted capabilities. For instance, suppose that we want to add an expiry date to our read-only macaroon such that it is only valid for thirty seconds. We can accomplish this with the following code:

```
>>> M = macaroons.create('account number', SECRET, '')
>>> M = M.add_first_party_caveat('op = read')
>>> import time
>>> expiration = int(time.time()) + 30
>>> M = M.add_first_party_caveat('time < %d' % expiration)
>>> token = M.serialize()
>>> c.get('accounts', account, auth=[token])
{'name': 'John Smith', 'balance': 15}
>>> time.sleep(31)
>>> c.get('accounts', account, auth=[token])
Traceback (most recent call last):
HyperDexClientException: ... it is unauthorized [HYPERDEX_CLIENT_UNAUTHORIZED]
```

Macaroons are extremely efficient to construct and verify, as they rely solely on efficient hash functions and avoid public key cryptography. This means that clients may generate a new macaroon-based token for each request. Each of these tokens may have a unique expiration time very near in the future. Even if the token makes its way into the hands of a malicious user, the token can only be used for a short period of time, and subject to other caveats attached to the macaroon.

8.3 Advanced Caveats

Macaroons enable rich security policies to be enforced. Specifically, macaroons enable third-party caveats, which enable security policies that incorporate a third-party's approval for access. Such third-parties may verify any property of an application during their access control decisions, such as:

- User authentication: The third party service can authenticate the user against existing user databases (e.g., LDAP, OpenAuth, Facebook, Twitter and the like), and provide a proof that the user is the same user identified in the third party caveat.
- Auditing and logging: The third party service can log the interaction, and issue a proof that the request was logged securely in a centralized logging location.
- Usage limits: The third party can check to ensure that the user does not perform a given operation more than a desired number of times.

Adding a third-party caveat is almost as simple as adding a first-party caveat, but requires interacting with the third party to generate the caveat. The client who is adding the caveat contacts the third party service with the caveat to be enforced, and a unique secret key generated for the caveat. In response, the third party returns an opaque identifier that it can use to recall the caveat and key at a later time. The identifier is meaningless to everyone but the third party, so that only the third party can recover the caveat and key from the identifier. Possible implementations include encrypting the client-provided key and caveat with a secret known to the third party, and returning the resulting ciphertext. Other possible implementations include storing the key and caveat in a hash table (possibly HyperDex) under a random key, and returning this random key to the client.

We can use third-party caveats to implement a user authentication service for macaroons. This service provides a means of generating third party caveats, and a method for clients to authenticate themselves with macaroons. The service exposes a call to generate caveats, whose implementation looks like this:

```
>>> keys = {}
>>> def add_caveat_rpc(key, user, password):
...     r = 'a random string' # your implementation should gen a rand string
...     keys[r] = (key, user, password)
...     return r
...
```

The client can then call this method (over HTTP or some other service-like interface), and retrieve an identifier for the third-party caveat.

```
>>> key = 'a unique key for this caveat; should be random in the crypto sense'
>>> ident = add_caveat_rpc(key, 'jane.doe@example.org', "jane's password")
```

The identifier returned from the `add_caveat_rpc` call can be embedded in a macaroon as a third party caveat:

```
>>> M = macaroons.create('account number', SECRET, '')
>>> M = M.add_first_party_caveat('op = read')
>>> M = M.add_third_party_caveat('http://auth.service/', key, ident)
>>> token = M.serialize()
```

Notice that the client constructs the third-party caveat using the key it provided to the third-party, and the identifier returned from the third party. The URL "http://auth.service/" is a location-hint as to where the service for the third-party caveat resides.

When the client tries to use our new token, the request will be denied because the macaroon does not carry a full proof authorizing access to the object.

```
>>> c.get('accounts', account, auth=[token])
Traceback (most recent call last):
HyperDexClientException: ... it is unauthorized [HYPERDEX_CLIENT_UNAUTHORIZED]
```

To obtain this access, the client must go back to the third party and request a discharge macaroon that proves that the user can authenticate using Jane's email and password. The implementation within the third-party recalls the key, checks the username and password, and returns a discharge macaroon when the user authenticates successfully.

```
>>> def generate_discharge_rpc(ident, user, password):
...     if ident not in keys:
...         # unknown caveat
...         print 'fuck1'
...         return None
...     key, exp_user, exp_password = keys[ident]
...     if exp_user != user or exp_password != password:
...         # invalid user/password pair
...         print 'fuck2', repr(exp_user), repr(user), repr(exp_password), repr(password)
...         return None
...     D = macaroons.create('', key, ident)
...     expiration = int(time.time()) + 30
...     D = D.add_first_party_caveat('time < %d' % expiration)
...     return D
...
```

The application may then request a discharge macaroon from this third party service at any time, provided that it can fulfill the caveat being enforced by the third-party. For our example authentication service, we can generate a discharge macaroon as follows:

```
>>> D = generate_discharge_rpc(ident, 'jane.doe@example.org', "jane's password")
```

With the discharge macaroon in hand, we can provide both our original token, and the token for the new discharge macaroon as the auth parameter to HyperDex. When both tokens are provided together, the request is authorized, just as before:

```
>>> discharge_M = M.prepare_for_request(D)
>>> discharge_token = discharge_M.serialize()
>>> c.get('accounts', account, auth=[token, discharge_token])
{'name': 'John Smith', 'balance': 15}
```

One of the nice things about the macaroon structure is that any caveats added to discharge macaroons are also enforced by HyperDex. If we wait until the expiration time of the discharge macaroon has passed, the request will fail, just as it did before when the expiration was on the root macaroon:

```
>>> time.sleep(31)
>>> c.get('accounts', account, auth=[token, discharge_token])
Traceback (most recent call last):
HyperDexClientException: ... it is unauthorized [HYPERDEX_CLIENT_UNAUTHORIZED]
```

8.4 Efficiency Considerations

Macaroons are excellent for use in distributed systems, because they allow applications to enforce complex authorization constraints without requiring server-side modification. Applications can use existing infrastructure to generate discharge macaroons, and provide these macaroons to HyperDex. On the server-side, HyperDex uses local and fast cryptographic operations to verify that the macaroons contain a valid proof that the user is authorized to continue their request. Consequently, it is very easy to perform per-object authorization without expensive operations on the server-side fast path.

Chapter 9

MongoDB Compatibility

The previous chapters focus on HyperDex's API, and demonstrate how it may be leveraged in new applications. In this chapter, we'll focus on another portion of the HyperDex API, the Mongo Veneer, that is API compatible with its namesake. This enables applications written for MongoDB to be ported to HyperDex with little to no modification.

With the Mongo Veneer, applications may take advantage of the familiar MongoDB API, with HyperDex's strong consistency, configurable fault tolerance, and consistent backups in the backend.

Let's take a brief look at HyperDex's Mongo Veneer.

9.1 Setup

As in the previous chapters, the first step is to deploy the cluster and connect a client. The cluster setup below is similar to the previous chapters, so if you have a running cluster, you can skip to the next step:

First, we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a daemon process to store data:

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data
```

We now have a HyperDex cluster ready to serve our Mongo application. There's no need to create spaces at this time as the API will automatically handle that for us.

9.2 Switching to HyperDex

Switching from Mongo to HyperDex is trivial. Suppose you have some application code that uses MongoDB:

```
>>> import pymongo # the old import
>>> client = pymongo.MongoClient('localhost', 27017)
```

Switching to HyperDex involves simply replacing this snippet with the following:

```
>>> import hyperdex.mongo as pymongo
>>> client = pymongo.HyperDatabase('localhost', 1982)
```

The rest of the code, like the snippet below, remains unaffected and continues to work:

```
>>> # Here's the code you don't change
>>> collection = client.db.profiles
>>> collection.insert({'_id': 'jane', 'name': 'Jane Doe', 'sessioncount': 1})
'jane'
>>> collection.update({'name': 'Jane Doe'}, {'$inc': {'sessioncount': 1}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 0}
```

Just as in MongoDB, there's no need to explicitly declare schemas or hyperspaces. The Mongo veneer automatically creates and manages the necessary data spaces, and translates MongoDB queries and updates to HyperDex calls.

Let's examine the document type that this veneer supports, and delve into the operations supported on documents.

9.3 The Document Type

The Mongo Veneer builds on HyperDex's existing documents. This datatype allows us to store, retrieve and operate on JSON documents directly. HyperDex imposes few limits on the structure of documents – they can be deeply nested, contain any number of fields, and so forth; it requires only that they be valid JSON, not contain more than 200 levels of nesting, and not exceed 32MB. These limits are double what is supported by the latest release (2.6) of MongoDB.

9.4 The Document API

The HyperDex document API supports many of the operations and operators supported by MongoDB. The following code examples work identically under both MongoDB and HyperDex, and will produce the output shown. Many MongoDB tutorials can now be executed, word for word, on the HyperDex Mongo Veneer instead.

9.4.1 Operations

Of the features present in MongoDB, this latest release (1.6) supports operations like:

- **insert** Store a document into the database. For example, this will insert a document with the key "andy1":

```
>>> collection.insert({'_id': 'andy1', 'name': 'Andy', 'friends': []})
'andy1'
```

- **save** Overwrite a document, changing its value, possibly inserting it if it does not exist. For example, this overwrites the Andy object with a new one, where his name is "Andy", and he has an Age of 17:

```
>>> collection.save({'_id': 'andy1', 'name': 'Andrew', 'age': 17, 'friends': []})
'andy1'
```

- **find** This will find all documents that match a particular predicate. So we can find every user named Andrew:

```
>>> list(collection.find({'name': 'Andrew'}))
[{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andrew'}]
```

- **find_one** This will find at most one document that matches the predicate. So we can find any one user named Andrew:

```
>>> collection.find_one({'name': 'Andrew'})
{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andrew'}
```

- **update** This will update the documents that match a particular predicate. The predicate is the same form as the selection used for "find". For example, this sets value "is_andrew" to true for every user named Andrew:

```
>>> collection.update({'name': 'Andrew'}, {'$set': {'is_andrew': True}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andrew'})
{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andrew', 'is_andrew': 1}
```

9.4.2 Update Operators

The new veneer supports many of the modification operators, such as:

- **$set** This will set a field within a document to the specified value. For example, we can change Andrew's name back to Andy with this:

```
>>> collection.update({'_id': 'andy1'}, {'$set': {'name': 'Andy'}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andy', 'is_andrew': 1}
```

- **$inc** Increment a particular field. If Andy acquires ten trinkets, we can increment his trinket count by ten. Since the field doesn't exist, the field will be created and set to 0 + 10, like this:

```
>>> collection.update({'name': 'Andy'}, {'$inc': {'trinkets': 10}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andy',
 u'is_andrew': 1, 'trinkets': 10}
```

- **$mul** Multiply the existing value by the provided value. For example, to add 1% interest to his existing value, we could multiply it by 1.01 like this:

```
>>> collection.update({'name': 'Andy'}, {'$mul': {'trinkets': 1.01}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andy',
 u'is_andrew': 1, 'trinkets': 10.1}
```

- **$div** Divide is very similar to the "$mul" operator. For example, we can reduce Andy's holdings of trinkets by half like so (note: MongoDB does not provide $div):

```
>>> collection.update({'name': 'Andy'}, {'$div': {'trinkets': 2.0}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [], u'name': u'Andy',
 u'is_andrew': 1, 'trinkets': 5.05}
```

- **$push** The push operator appends to an array. We can add Buzz to the list of Andy's friends like this:

```
>>> collection.update({'name': 'Andy'}, {'$push': {'friends': 'Buzz'}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [u'Buzz'], u'name': u'Andy',
 u'is_andrew': 1, 'trinkets': 5.05}
```

- **$rename** Move a field within a document. We can change the "is_andrew" field to "is_andy" like this:

```
>>> collection.update({'name': 'Andy'}, {'$rename': {'is_andrew': 'is_andy'}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [u'Buzz'], u'name': u'Andy',
 u'is_andy': 1, 'trinkets': 5.05}
```

- **$bit** Manipulate fields in a bitwise fashion. For instance:

```
>>> collection.update({'name': 'Andy'}, {'$bit': {'perms': {'and': 488}}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [u'Buzz'], u'name': u'Andy',
 u'is_andy': 1, 'trinkets': 5.05, 'perms': 0}

>>> collection.update({'name': 'Andy'}, {'$bit': {'perms': {'or': 055}}})
{'updatedExisting': True, u'nModified': 1, u'ok': 1, u'n': 1}
>>> collection.find_one({'name': 'Andy'})
{u'age': 17, u'_id': u'andy1', u'friends': [u'Buzz'], u'name': u'Andy',
 u'is_andy': 1, 'trinkets': 5.05, 'perms': 45}
```

9.4.3 Predicates

HyperDex also supports many of the same predicates, such as:

- **$eq** Check that the values are equal. Here we find everyone named Andy:

```
>>> list(collection.find({'name': {'$eq': 'Andy'}}))
[{u'trinkets': 5.05, u'name': u'Andy', u'is_andy': 1, u'age': 17,
  u'perms': 45, u'_id': 'andy1', u'friends': [u'Buzz']}]
```

- **$lt**, **$lte**, **$gte**, **$gt**: Less than, lesser-equal, greater-equal, and greater-than. These operators will work to compare numbers like this:

```
>>> # Find people who are underage
>>> list(collection.find({'age': {'$lt': 18}}))
[{u'trinkets': 5.05, u'name': u'Andy', u'is_andy': 1, u'age': 17,
  u'perms': 45, u'_id': 'andy1', u'friends': [u'Buzz']}]
>>> # Find people who qualify for AARP
>>> list(collection.find({'age': {'$gt': 65}}))
[]
>>> # Find people in the [25-35] demographic
>>> list(collection.find({'age': {'$gte': 25, '$lte': 35}}))
[]
```

Chapter 10

Transactions

HyperDex Warp is a superset of HyperDex that provides transactions. For this chapter, you'll need to install the `hyperdex-warp` demo package available by contacting the HyperDex support team.

By the end of this chapter you'll be familiar with HyperDex Warp's transactions.

10.1 Setup

As in the previous chapter, the first step is to deploy the cluster and connect a client. First we launch and initialize the coordinator:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982
```

Next, let's launch a few daemon processes to store data. Execute the following commands (note that each instance binds to a different port and has a different `/path/to/data`):

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data1
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2013 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data2
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2014 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data3
```

This brings up three different daemons ready to serve in the HyperDex cluster. Finally, we create a space which makes use of all three systems in the cluster. In this example, let's create a space that may be suitable for storing virtual currency in a banking application:

```
>>> import hyperdex.client
>>> c = hyperdex.client.Client('127.0.0.1', 1982)
>>> c.add_space('''
... space accounts
... key id
... attribute
...    int balance
... ''')
```

Let's create some accounts with some starting balances:

```
>>> c.put('accounts', 'joe', {'balance': 100})
True
>>> c.put('accounts', 'brian', {'balance': 25})
True
```

10.2 Using Transactions

The prototypical example of where transactions come in handy is a standard bank debit/credit scenario. If Joe wanted to transfer currency to Brian, it would be bad for Joe and Brian if the money were to leave Joe's account and not find its way into Brian's. It would be bad for the bank if money were to be created out of thin air by a deposit in Brian's account without a corresponding withdrawal from Joe's – manipulating markets in such a fashion is reserved for those who are too big to fail.

In the prototypical, "Hello World," example involving transactions, we transfer $10 from Joe to Brian:

```
>>> x = c.begin_transaction()
>>> brian = x.get('accounts', 'brian')['balance']
>>> joe = x.get('accounts', 'joe')['balance']
>>> x.put('accounts', 'brian', {'balance': brian + 10})
True
>>> x.put('accounts', 'joe', {'balance': joe - 10})
True
>>> x.commit()
True
>>> print "Brian's balance =", c.get('accounts', 'brian')['balance']
Brian's balance = 35
>>> print "Joe's balance =", c.get('accounts', 'joe')['balance']
Joe's balance = 90
```

This transaction is relatively straight-forward. There are three critical properties that will distinguish this transaction. One, either both of these operations execute, or neither do. It is impossible for half of a transaction to be lost.

Two, there is no eventual consistency. All transaction results are immediately visible to all future events. There is no need to reconcile or maintain multiple versions. There is no inconsistency.

Finally, HyperDex Warp is fault tolerant. A user-configurable fault tolerance threshold, `f` allows HyperDex Warp to remain available in the presence of concurrent failures.

For a real bank application, we'd likely want to charge Joe an (excessive) overdraft fee if he didn't have the money to cover the transfer. With transactions, implementing this case is trivial:

```
>>> x = c.begin_transaction()
>>> brian = x.get('accounts', 'brian')['balance']
>>> joe = x.get('accounts', 'joe')['balance']
>>> x.put('accounts', 'brian', {'balance': brian + 10})
True
>>> if joe < 10:
...     # assess an overdraft fee on Joe
...     joe -= 35
...
>>> x.put('accounts', 'joe', {'balance': joe - 10})
True
>>> x.commit()
True
>>> print "Brian's balance =", c.get('accounts', 'brian')['balance']
Brian's balance = 45
>>> print "Joe's balance =", c.get('accounts', 'joe')['balance']
Joe's balance = 80
```

Here, we've successfully transferred money from Joe to Brian and cover the case where the bank wishes to charge Joe a fee for not having enough money in the first place.

The astute reader will notice that the example above is very different from doing the following:

```
>>> # an example of broken code
>>> c.atomic_add('accounts', 'brian', {'balance': 10})
True
>>> c.atomic_sub('accounts', 'joe', {'balance': 10})
True
>>> print "Brian's balance =", c.get('accounts', 'brian')['balance']
Brian's balance = 55
>>> print "Joe's balance =", c.get('accounts', 'joe')['balance']
Joe's balance = 70
```

This example is broken in multiple ways. First, even though each individual operation is atomic, the two operations are not indivisible, and will be interspersed with all other operations on the accounts. Second, the client is a single point of failure for this transaction. Imagine if the server executing these operations failed between the two atomic operations. No matter what the order of operations, someone would lose money.

Transactions are especially useful when multiple competing processes are executing transactions concurrently. HyperDex Warp guarantees *one-copy serializability*. Any number of transactions may execute simultaneously. The final state of the database will always be identical to executing the committed transactions sequentially without concurrency.

Let's illustrate HyperDex's behavior with concurrent transactions. In the same terminal, let's begin a new transaction to pay Joe his yearly 5% interest payment:

```
>>> x = c.begin_transaction()
>>> balance = x.get('accounts', 'joe')['balance']
>>> balance = int(balance * 1.05)
>>> x.put('accounts', 'joe', {'balance': balance})
True
```

At this point, transaction `x` has not committed. Any modifications performed within the context of a transaction are visible only within the transaction. No other clients or transactions may observe the state. Verify this for yourself with:

```
>>> print "Joe's balance =", c.get('accounts', 'joe')['balance']
Joe's balance = 70
>>> print "Joe's balance =", x.get('accounts', 'joe')['balance']
Joe's balance = 73
```

Joe's balance is still the same as before. Now, open another window and start a second, concurrent transaction where Joe deposits cash at a branch location:

```
>>> import hyperdex.client
>>> c2 = hyperdex.client.Client('127.0.0.1', 1982)
>>> y = c2.begin_transaction()
>>> balance = y.get('accounts', 'joe')['balance']
>>> balance += 386
>>> y.put('accounts', 'joe', {'balance': balance})
True
```

At this point, both `x` and `y` are ready to commit, but neither has actually altered the account balance. In fact, there is no way for `x` and `y` to commit that doesn't violate consistency. If `x` commits before `y`, the ending balance will be $466, shorting Joe of the $4 in interest he is due. If, however, `y` commits before `x`, then the ending balance of $84 will cause Joe's cash deposit of $386 to be lost into the ether. In this situation, one transaction must abort. If `y` commits first, then `x` will abort:

In the second terminal, commit `y`:

```
>>> y.commit()
True
>>> print "Joe's balance =", c2.get('accounts', 'joe')['balance']
Joe's balance = 456
```

Trying to commit x in the first terminal will result in the transaction being aborted by HyperDex Warp:

```
>>> x.commit()
Traceback (most recent call last):
HyperClientException: HyperClient(HYPERCLIENT_ABORTED, Transaction was aborted)
```

As expected, transaction x will not alter Joe's balance:

```
>>> print "Joe's balance =", c2.get('accounts', 'joe')['balance']
Joe's balance = 456
```

A transaction will only abort if it was executed simultaneously with another transaction that operates on the same data. The typical process for handling aborted transactions is to repeatedly try the transaction until it succeeds:

```
>>> committed = False
>>> while not committed:
...     try:
...         x = c.begin_transaction()
...         balance = x.get('accounts', 'joe')['balance']
...         balance = int(balance * 1.05)
...         x.put('accounts', 'joe', {'balance': balance})
...         committed = x.commit()
...     except hyperdex.client.HyperDexClientException as e:
...         if e.status != hyperdex.client.HYPERCLIENT_ABORTED:
...             raise e
...
True
```

10.3 Rich API

HyperDex Warp Transactions expose the full set of atomic and asynchronous operations from the HyperDex API. All operations performed under a transaction are fully transactional. The following example shows asynchronous atomic math operations combined with an asynchronous commit:

```
>>> t = c.begin_transaction()
>>> d1 = t.async_atomic_add('accounts', 'brian', {'balance': 10})
>>> d2 = t.async_atomic_sub('accounts', 'joe', {'balance': 10})
>>> d1.wait()
True
>>> d2.wait()
True
>>> d3 = t.async_commit()
>>> d3.wait()
True
```

All transactions operate on the most recent copy of the data. When a transaction commits, the transaction is consistently applied. HyperDex Warp really means it when it says a transaction has committed. There are no conflicting operations to reconcile later (after the system commits), and there is no eventual consistency. All effects are immediate and permanent.

10.4 Summary

HyperDex Warp provides decentralized, distributed ACID transactions. Committed transactions are one-copy serializable, making it extremely easy to reason about the concurrent operations. The rich API coupled with transactional guarantees provide a rare combination of ease-of-use, correctness, and performance that other NoSQL systems cannot provide.

Part II

For Administrators

Chapter 11

Backup and Recovery

Backups provide disaster recovery and protect against user error. HyperDex's replication provides some of this same protection, but there is no substitute for proper backup hygiene. A full blown backup protection mechanism protects the cluster correlated failures that would otherwise violate HyperDex's f-fault tolerance guarantees.

HyperDex provides one of the best backup experiences available in any NoSQL store. HyperDex's backups enable administrators to backup the on-disk state of HyperDex servers in their native on-disk state. As a result, backups are quick and efficient, because there is no need to serialize and deserialize the data during the backup process. The backup/restore process is strongly consistent, doesn't require shutting down servers, and enables incremental backup support. Further, the process of initiating a backup is quite efficient; it completes quickly, and does not consume CPU or I/O for extended periods of time.

11.1 Consistent versus Inconsistent Backups

In a distributed data store, the simple but flawed way to make backups is to take a snapshot at each of the shards independently. Needless to say, this is precisely what a number of other systems do. The problem with this approach is that backups taken without coordination among the servers are not going to take a consistent cut through the data. A portion of the data in the backup might come from a snapshot at time T0, while others portions come from T1, and yet others from different points in time. Depending on the application, these kinds of uncoordinated backups can lead to inconsistencies in the database. Restoring from a backup might take the system to a state that actually never existed. And that defeats the entire point of a backup.

In contrast, HyperDex backups are coordinated. The network pauses for slightly less than a second, the operation queues are drained, pending operations are completed, and therefore the snapshots capture a consistent, clean view of the data. A restore will take the system back to a precise point in time.

11.2 Simple Backup Strategies

The simplest strategy for taking a backup is to use the included `backup-manager` command. Of course, before you can take a backup of a cluster, the cluster has to exist. We can easily setup a simple cluster with by executing following sequence of commands in separate terminals.

```
% hyperdex coordinator -f -l 127.0.0.1 -p 1982
% hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                     --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data
```

This will start a single-node cluster on our local machine. We can take a backup of this cluster by issuing the following command:

```
% hyperdex backup-manager --backup-dir /path/to/backups
```

Our backup of the entire cluster now resides under the `/path/to/backups` directory. Behind the scenes, this command will:

1. Put the cluster into read-only mode
2. Wait for ongoing writes to quiesce

3. Take a backup of the coordinator
4. Take a backup of each individual daemon
5. Put the cluster back into read-write mode
6. Copy the coordinator state and each of the daemon's state to `/path/to/backups/YYY-mm-ddTHH:MM:SS` for the current date and time, exploiting data available in previous backups if possible. This relies on having ssh access to the listed hosts, and using rsync to copy the data.

For example, after the above backup, the directory hierarchy will look like this:

```
% find /path/to/backups
/path/to/backups
/path/to/backups/LATEST
/path/to/backups/2015-01-05T18:43:08
/path/to/backups/2015-01-05T18:43:08/coordinator.bin
/path/to/backups/2015-01-05T18:43:08/13036267341651542609
/path/to/backups/2015-01-05T18:43:08/13036267341651542609/000004.log
/path/to/backups/2015-01-05T18:43:08/13036267341651542609/000006.log
/path/to/backups/2015-01-05T18:43:08/13036267341651542609/MANIFEST-000002
/path/to/backups/2015-01-05T18:43:08/13036267341651542609/000005.sst
/path/to/backups/2015-01-05T18:43:08/13036267341651542609/CURRENT
/path/to/backups/2015-01-05T18:43:08/13036267341651542609/LOG
```

We can see our recent backup is in directory `2015-01-05T18:43:08`. Within this directory is a backup of the coordinator state, `coordinator.bin`, and one directory for the server's state. Because we only have one server, we have one directory. With more servers, we would see more directories, each identified by the server's unique token. We can get a list of the servers in our cluster with this command:

```
% hyperdex show-config | grep server
```

We can see that indeed, the only server in our cluster is `13036267341651542609`. The files stored within this directory are a copy of the on-disk data at our server taken atomically during the backup process.

Suppose the unthinkable happens (a datacenter outage, cooling failure, flood, act of God, or more likely, some human error that causes data loss) we need to restore from backup on a different set of machines. Restoring the cluster is as simple as copying the backup directory `/path/to/backups/2015-01-05T18:43:08/13036267341651542609` to a different daemon server, and then launching a new coordinator from the backup:

```
% hyperdex coordinator --restore coordinator.bin --foreground -l 127.0.0.1 -p 1983
```

We can then bring the remainder of the cluster online by using the copied data directory on an entirely different server from the one that initially existed in our cluster.

```
% hyperdex daemon -f --listen=127.0.0.1 --listen-port=2013 \
                     --coordinator=127.0.0.1 --coordinator-port=1983 --data=/path/to/copy
```

All freshly-restored clusters come online in read-only mode. Once all daemons have come online, you can make the cluster fully operational by executing:

```
% hyperdex set-read-write -h 127.0.0.1 -p 1983
```

At this point, we have two fully functional HyperDex clusters running. We have our original cluster with the coordinator on port 1982, and the daemon on 2012; and, we have the restored copy of the cluster running on on ports 1983 and 2013.

11.3 Backup Efficiency

HyperDex's backups are extremely efficient, because its architecture enables quick snapshots, which may then be copied in the background.

HyperDex uses HyperLevelDB as its storage backend, which, in turn, constructs an LSM-tree on disk. The majority of data stored within HyperLevelDB is stored within immutable `.sst` files. Once written, these files are never overwritten. The backup feature within HyperDex can extract a snapshot of these files via an instantaneous hard link in the filesystem. This state, once snapshotted so efficiently, can then be transferred elsewhere at will.

At the cluster level, backups pause writes for a short period of time while the HyperDex daemons take their individual snapshots. Because individual snapshots are efficient, the pause generally takes less than a second.

Finally, the backup tool ensures that the cluster is restored to full operation before transferring any data over the network. This transfer comprises the bulk of the backup operation, and happens only after the cluster resumes normal operation.

11.4 Advanced Backup Techniques

Backup solutions often require customization; some setups use network attached storage and back up to a centralized location, others will demand custom control over where and how the snapshots are transferred. Let's walk through a more detailed example where the snapshots are manually managed.

Every custom backup solution will start by initiating a snapshot operation within the HyperDex cluster. The command to take a snapshot is called `backup`. For example, this command will create a snapshot named `my-backup`:

```
% hyperdex backup my-backup
13036267341651542609 127.0.0.1 /path/to/data/backup-my-backup
```

When this command succeeds, it will exit zero, and output a list of the servers that hold individual snapshots. For instance, we can see that our server identified by `13036267341651542609` on 127.0.0.1 backed up its state to `13036267341651542609`.

We can then backup this state to wherever we wish to store the backup. In the backup-manager command, we transfer the state to the specified directory using rsync like this:

```
% rsync -a --delete -- 127.0.0.1:/path/to/data/backup-my-backup/ 13036267341651542609/
```

This command will create a directory named "13036267341651542609" and transfer all of the server's state to this directory. This directory is suitable for passing to the `--data` parameter of HyperDex's daemon.

In addition to the per-daemon state backups, the backup command above also creates a file in the current directory called `my-backup.coordinator.bin`. This file contains the coordinator's state, suitable for using to bootstrap a new coordinator cluster as shown above.

The backup-manager command shown above builds upon this low-level backup command, and neatly organizes the backup of each server and the coordinator into the backup directory. Of course, other possible schemes include:

- Using filesystem-level snapshots to save the backup directories. Daemons may then be launched from the snapshots in the future. While this protects against some software faults, it doesn't provide as thorough protection against disaster recovery as moving the backups offsite.
- Having the daemons directly rsync their state to an off-site storage location. This avoids any resource constraints that may limit the effectiveness of the backup-manager, and avoids the possibility for hardware failure to corrupt filesystem-level snapshots.
- Restore multiple instances of a cluster simultaneously to experiment with your production data. For example, before deploying a new application to production, backup the cluster, and restore the backup in parallel to the production cluster. Then, deploy the new application against the parallel cluster before deploying to production.

11.5 Incremental Backups with Rsync

The backup-manager command demonstrated above supports incremental backups, wherein successive calls will only transfer state that has been modified since the last backup. Each backup appears as a full backup of the cluster and shares storage resources with previous backups. Consequently, the backup manager will store several complete backups of the cluster without requiring the storage space of several complete backups.

The implementation leverages the `--link-dest` option to rsync to avoid making redundant copies of the data. When provided this option, rsync will look for files in the specified directory. If the file in the source directory is also present, unchanged, in the link-dest directory, then it will be hard-linked into place and not copied over the network.

Assuming an existing backup exists in the directory `13036267341651542609-1`, we can create an incremental backup in `13036267341651542609-2` by issuing:

```
% rsync -a --delete --link-dest=13036267341651542609-1 -- \
    127.0.0.1:/path/to/data/backup-my-backup/ \
    13036267341651542609-2
```

This will use rsync to de-duplicate the data and avoid redundant copies. Other possibilities include storing the data into a storage service like S3, with a higher level application orchestrating the de-duplication logic.

Chapter 12

Fault Tolerance

HyperDex recovers from failures automatically. When a node fails, HyperDex detects the failure, removes the failed node, and automatically reintegrates other nodes to restore the desired level of fault tolerance. The coordinator is made fault tolerant using a replicated state machine. It, too, is fault tolerant and can tolerate a configurable number of failures.

Let's see both kinds of fault tolerance in action.

12.1 Coordinator

First, let's demonstrate the coordinator's fault tolerance. Bring up a singly-fault tolerant coordinator using the same procedure as in previous chapters. Note that this single node coordinator can shutdown and restart, but is unavailable while it is shutdown for obvious reasons.

```
hyperdex coordinator -f -l 127.0.0.1 -p 1982 -D /path/to/coord1
```

A HyperDex coordinator becomes more fault tolerant as more nodes are added. A simple majority of the nodes must remain available for the coordinator to make progress. For example, a cluster with three nodes can fail one node at any point in time, while a cluster with five nodes can tolerate two simultaneous failures.

Internally, the HyperDex coordinator uses state machine replication to provide fault tolerance. Running at least five servers is recommended so that the cluster can tolerate at least two failures. To do so, bring additional coordinators online:

```
hyperdex coordinator -f -l 127.0.0.1 -p 1983 -c 127.0.0.1 -P 1982 -D /path/to/coord2
hyperdex coordinator -f -l 127.0.0.1 -p 1984 -c 127.0.0.1 -P 1982 -D /path/to/coord3
hyperdex coordinator -f -l 127.0.0.1 -p 1985 -c 127.0.0.1 -P 1982 -D /path/to/coord4
hyperdex coordinator -f -l 127.0.0.1 -p 1986 -c 127.0.0.1 -P 1982 -D /path/to/coord5
```

For each new server in the cluster, the coordinator will automatically transfer a copy of the initialized state machine and replicate further operations on those nodes as well. In its current form our cluster can tolerate two simultaneous failures or planned shutdowns. Go ahead and shutdown one or more coordinator servers by pressing Control-C. When restarted, they'll rejoin the cluster and collect any state changes that occurred in their absence.

It's even possible to completely shutdown and restart the coordinator by shutting down and restarting all nodes in the cluster.

12.2 HyperDex Daemons

HyperDex daemons are organized by the coordinator. In response to failures and shutdowns, the coordinator automatically responds by assigning a new server to take its place and initiating recovery procedures to ensure the new server maintains an up-to-date view of the data.

To see this in action, we first need to start a cluster:

```
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2012 \
                   --coordinator=127.0.0.1 --coordinator-port=1982 --data=/path/to/data1
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2013 \
                   --coordinator=127.0.0.1 --coordinator-port=1983 --data=/path/to/data2
hyperdex daemon -f --listen=127.0.0.1 --listen-port=2014 \
                   --coordinator=127.0.0.1 --coordinator-port=1984 --data=/path/to/data3
```

If you look carefully, you'll see that we've asked each daemon to connect to a different coordinator server. Each daemon connects to the coordinator in a fault-tolerant manner. Should the specified server fail after the daemon connects, the daemon will simply connect to another server in the cluster. When a failure of either a coordinator node or a daemon occurs, the coordinator ensures that all servers in the system will eventually be able to make progress.

Let's see this in action by creating some data items we care deeply about and checking what happens to our data as failures occur:

```
>>> a.add_space('''
... space phonebook
... key username
... attributes first, last, int phone
... subspace first, last, phone
... create 8 partitions
... tolerate 2 failures
... ''')
>>> c.put('phonebook', 'jsmith1', {'phone': 6075551024})
True
>>> c.get('phonebook', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'phone': 6075551024}
```

So now we have a data item we deeply care about. We certainly would not want our NoSQL store to lose this data item because of a failure. Let's create a failure by killing one of the three hyperdaemon processes we started in the setup phase of the tutorial. Feel free to use `kill -9`, there is no requirement that the nodes shut down in an orderly fashion. HyperDex is designed to handle a configurable number of crash failures.

```
>>> # kill a node at random
>>> c.get('phonebook', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'phone': 6075551024}
>>> c.put('phonebook', 'jsmith1', {'phone': 6075551025})
True
>>> c.get('phonebook', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'phone': 6075551025}
>>> c.put('phonebook', 'jsmith1', {'phone': 6075551026})
True
```

So, our data is alive and well. Not only that, but the subspace is continuing to operate as normal and handling updates at its usual rate.

Let's kill one more server.

```
>>> # kill a node at random
>>> c.get('phonebook', 'jsmith1')
Traceback (most recent call last):
File "<stdin>", line 1, in <module>
File "hyperclient.pyx", line 473, in hyperclient.Client.put ...
File "hyperclient.pyx", line 499, in hyperclient.Client.async_put ...
File "hyperclient.pyx", line 255, in hyperclient.DeferredInsert.__cinit__ ...
hyperclient.HyperClientException: Connection Failure
>>> c.get('phonebook', 'jsmith1')
{'first': 'John', 'last': 'Smith', 'phone': 6075551026}
```

Note that the HyperDex API exposes some failures to the clients at the moment, so a client may have to catch HyperClientException and retry the operation. The HyperDex library does not resubmit failed operations on behalf of clients. In this example, behind the scenes, there were two node failures in the triply-replicated space. Each failure was detected, the space was repaired by cleaving out the failed node, and normal operations resumed without data loss.

12.3 Fault Tolerance Thresholds

HyperDex daemons and coordinators each tolerate a configurable number of failures before the system fails completely. For a desired level of fault tolerance f, you need to start $f+1$ daemons and $2f+1$ coordinators. Both are able to tolerate more than f failures so long as enough nodes rejoin the cluster to bring the number of failures back under the failure threshold.

12.4 Shutting Down and Restoring a Cluster

On occasion, you might need to completely shutdown a HyperDex cluster. For example, planned power outages or kernel upgrades might require such shutdowns. Note that complete shutdowns, by definition, require every server process to be killed and therefore violate the fault tolerance threshold. Managing and recovering from such shutdowns is in general a non-trivial process, but HyperDex makes it easy to restart all processes without any data loss.

Shutting down a cluster is a three step process. First, stop all client traffic. Second, kill all the daemon processes using `SIGHUP`, `SIGINT`, or `SIGTERM`. The daemons will inform the coordinator of their departure, and the coordinator will track which daemons departed successfully. Finally, kill each coordinator process using `SIGHUP`, `SIGINT`, or `SIGTERM`.

To restore a cluster, all you need to do is first restart the coordinator nodes and then restart the daemon nodes. Because state was cleanly saved to the disk, this restart process will bring back the entire state of the database without data loss.

Chapter 13

Cluster Maintenance

This chapter walks through some basic cluster maintenance tasks one might encounter running HyperDex on a day-to-day basis. This chapter takes the form of small recipes that may be applied individually as required.

13.1 The Daemons

This section covers basic tasks for inspecting and manipulating the storage daemons in a HyperDex cluster.

13.1.1 Scaling Up: Adding Daemons

To scale the cluster up, launch additional daemons that connect to the same coordinator. Each new daemon will be automatically integrated into the cluster by the coordinator, with no need for further action. Assuming we already have a coordinator at 10.1.1.50:1982, we can launch another daemon like this:

```
$ hyperdex daemon --coordinator 10.1.1.50 --coordinator-port 1982
```

13.1.2 Scaling Down: Removing Daemons

To scale the cluster down, we remove servers from the cluster, and let the cluster rebalance when done. To remove a server from the cluster, we first identify the server we wish to remove. We can list all servers like this:

```
$ hyperdex show-config | grep '^server'
server 4094788354897669190 10.1.1.50:2012 AVAILABLE
server 5430153233975899120 10.1.1.51:2012 AVAILABLE
server 5578696023185727686 10.1.1.52:2012 AVAILABLE
server 8033285057934092439 10.1.1.53:2012 OFFLINE
server 12806351800314218469 10.1.1.54:2012 AVAILABLE
server 17738609275975365923 10.1.1.55:2012 AVAILABLE
```

Here we can see we have six daemons in the cluster, and that 5/6 of them are available, while the sixth is offline. We can permanently remove this server from the cluster—and free resources associated with it like this:

```
$ hyperdex server-kill 8033285057934092439
```

This indicates to our cluster that this daemon is permanently unavailable. It will never be allowed to reintegrate to the cluster, allowing an administrator to use the server-kill command to remove flaky hardware, or rearrange the cluster during a migration. Once a daemon is killed, we can kill the daemon on the server itself, and erase its data directory. Finally, we can totally forget the server ever existed with the server-forget command:

```
$ hyperdex server-forget 8033285057934092439
```

While it's not necessary to use the kill and forget commands to remove daemons, doing so has several distinct advantages: first, it enables the cluster to permanently reassign the hyperspace that belonged to the failed server, and, second, it prevents the failed server from consuming resources in the cluster coordinator indefinitely.

13.2 The Coordinator

HyperDex's coordinator acts as one coherent entity that shepherds the rest of the cluster through global state changes, such as when servers join or depart, and when spaces are added, removed, or modified.

The coordinator is comprised of a set of servers that act in concert to agree upon these state changes. So long as a majority of these servers are available, the coordinator as a whole is available.

Underneath the hood, the HyperDex coordinator is powered by replicant, which replicates the logic of the HyperDex coordinator.

13.2.1 Listing Coordinator Servers

To get a list of all servers that comprise the coordinator, we can use the list-servers command provided by replicant.

```
$ replicant list-servers
10.1.1.50:1982
10.1.1.51:1982
10.1.1.52:1982
10.1.1.53:1982
```

We can see that there are four servers currently active. Any one of these four may be used as the address to which a HyperDex client connects. In an advanced deployment, it's recommended to tie the listed servers into the DNS so that clients may connect to any one live server. For example, we could create a round-robin DNS entry for "hyperdex.example.org" that resolves to the above servers. The entry should be reasonably up-to-date, but once a daemon or client connects to the cluster, it will track the list of coordinator servers automatically, so introducing it to any one server is sufficient.

13.2.2 Scaling up the Coordinator

To scale up the coordinator, launch another instance of the HyperDex coordinator and tell it to connect to a pre-existing address. For example, we can add a sixth server to the cluster on 10.1.1.54:1982 by running this command on that host:

```
$ hyperdex coordinator --connect 10.1.1.52 --connect-port 1982 --listen 10.1.1.54
```

The coordinator will only ever make use of five servers at a time, so launching more than five servers at a time is unnecessary and will not improve cluster scalability or stability.

13.3 Spaces

HyperDex's spaces are meta-data that is stored in the coordinator and distributed throughout the HyperDex cluster. Spaces are relatively lightweight to construct and remove.

13.3.1 Adding a Space

In the preceding chapters, we've looked at a variety of spaces that we could create with HyperDex. The command add-space can be used to add a space to the cluster from the command line. For example, if our coordinator is on 10.1.1.50:1982, we can add a new space called "people" like this:

```
$ hyperdex add-space -h 10.1.1.50 -p 1982 << EOF
space people key name attribute document profile
EOF
```

13.3.2 Removing a Space

Removing a space can be done from the commandline with the rm-space command. For example, if our coordinator is on 10.1.1.50:1982, we can remove the space "people" like this:

```
$ hyperdex rm-space -h 10.1.1.50 -p 1982 people
```

13.3.3 Renaming a Space

The mv-space command provides a means by which the name of a space may be changed. The command changes the meta-data for the space, and does not rewrite any data stored within the space. Consequently, it is a relatively lightweight operation. For example, we can rename the space "persons" to "people" like this:

```
$ hyperdex rm-space -h 10.1.1.50 -p 1982 persons people
```

13.3.4 Listing Spaces

It is also possible to list spaces from the commandline with the list-spaces command. For example:

```
$ hyperdex list-spaces -h 10.1.1.50 -p 1982
people
```

13.3.5 Changing a Space's Fault Tolerance

To change a space's fault tolerance, use the command set-fault-tolerance like this:

```
$ hyperdex set-fault-tolerance people 2
```

This changes the fault tolerance of the space "people" from the default of 1, to a setting of 2. This means the space can tolerate two concurrent failures without any unavailability.

Chapter 14

Tuning HyperDex

Out of the box, HyperDex provides a stable system with high performance. This chapter is dedicated to those who are looking to get even more performance out of their system. In this chapter, we'll explore system settings that can impact HyperDex's performance and stability.

As we navigate the available tuning options, keep in mind that these guidelines are general, and may or may not apply to all workloads. Before tuning a cluster, it is advisable to setup an end-to-end benchmark and regularly compare the impact each individual setting has on cluster stability and performance. Generally, the best benchmarks are derived directly from a HyperDex application, but the YCSB Benchmark (Section **??**) provides a set of workloads common to many applications and will work nearly as well.

14.1 Filesystem

The filesystem is the subsystem to tune to improve performance. Because most operating systems are designed to accommodate a wide variety of workloads, they usually will not provide optimal performance for a key-value store.

14.1.1 Choosing the Right Filesystem

When deploying a new cluster, the filesystem setup and configuration is the first opportunity for optimization. HyperDex works with any POSIX-compatible filesystem because it relies on simple and portable primitives. Filesystems with complex features that extend beyond the basic POSIX feature set (such as those provided by ZFS and btrfs) can impose performance penalties if such features are not used in the deployment. Simpler filesystems such as ext3, ext4, and XFS are usually a better choice because they are commonly available on Linux distributions, have received extensive testing, and provide high performance.

To chose the best filesystem for a new deployment, start by consulting the appropriate documentation for each filesystem. The following filesystems are listed in order of preference:

- XFS: Available on Linux, and the default filesystem on Red Hat Enterprise 7, XFS is a filesystem designed for scalability and performance.
- ext3: The de facto filesystem on most Linux distributions, ext4 is great for smaller partitions. With larger filesystems (1TB+), you'll need to tune the amount of space that is reserved for inode allocation .
- UFS: UFS is the FreeBSD filesystem. Its low overhead, and soft-update support make it the ideal choice for use on FreeBSD.
- ZFS: Available on Solaris and FreeBSD, ZFS provides advanced features, such as volume management, that replace the need for traditional RAID controllers and soft RAID. For maximum performance, ZFS requires quite a bit of memory—significantly more than any of the other systems.
- btrfs: Available on Linux, btrfs is the closest thing to ZFS on Linux (besides the ZFS userspace port). btrfs is relatively new compared to other systems that have decades of experience. For that reason we recommend avoiding it for the near term.

If you still have doubts, it is best to go with XFS, ext4, or UFS, depending on their availability on your platform. Check your distribution's documentation for details.

14.1.2 Improving Read Performance by Disabling Access Time Updates

By default, most filesystems update the access time on a file each time a file is accessed. For key-value workloads, these access time updates convert each read into a meta-data write. This can turn a relatively inexpensive cached read into an expensive disk write—which will, in turn, impact the performance of reads from disk, and other writes to disk.

Because HyperDex does not rely upon access times for correctness, we can obtain higher read performance by disabling access-time updates. On XFS filesystems, this behavior has been the default since 2006, so there is no need to tune this setting. Consult the filesystem documentation for details on other filesystems. For example, on ext4, this behavior may be obtained by setting the `relatime` flag in `/etc/fstab` for the partition with HyperDex data:

```
/dev/sdb1 /hyperdex/data/dir ext4 defaults,relatime 0 2
```

14.1.3 Improving ext3/4 Write Performance with `data=writeback`

The ext3 and ext4 filesystems are journaled filesystems, which means that they will often impose an order on disk operations to improve integrity during crashes. HyperDex relies upon application-level integrity checks that render this serialization at the filesystem level unnecessary. By relaxing these ordering constraints, we can obtain higher performance on journaled ext systems, comparable to the default behavior of XFS. To do this, add the flag `data=writeback` to the `/etc/fstab`:

```
/dev/sdb1 /hyperdex/data/dir ext4 defaults,data=writeback 0 2
```

14.1.4 Changing the Linux Disk Scheduler

On Linux, the disk scheduler impacts the order and latency of disk requests. For server workloads, the default scheduler may have undesirable consequences for the 99th percentile latency. A better option is to switch to the `noop` scheduler on SSDs and the `deadline` scheduler on spinning disks. To do this switch the scheduler like so (where DEV is your block device):

```
# echo noop > /sys/block/DEV/queue/scheduler
# echo deadline > /sys/block/DEV/queue/scheduler
```

14.1.5 Linux VM Tuning

The Linux kernel buffers writes in memory before flushing them to disk in the background. On occasion, the size of the write buffer can grow to such an extent that the kernel will use HyperDex's scheduled time slots to flush data to disk instead, appearing to stall the application for periods of time. There are multiple system controls that can be tuned to avoid this situation and reduce write latency variation.

To properly tune these parameters, you'll need to know the approximate throughput at which your disk devices will service write requests. This is important, because we are simultaneously tuning the threshold at which the background flush thread will wakeup, and the threshold at which the kernel will begin to block HyperDex. In the below examples, we'll assume each server has a single SSD capable of 500 MB/s.

- `vm.dirty_background_bytes`: This setting limits the number of bytes that will be marked as dirty before the kernel's flush thread begins to write data to disk. Increasing this setting will increase the time between flushes to disk, and increase the likelihood that disk I/O will stall userspace applications. Generally this should be no more than 5% of system memory, and no more than the data written in 10s at the disk's maximum sustained write speed. In our example, we'd set it to no more than 5 GB.

- `vm.dirty_bytes`: This setting limits the number of bytes that may be dirty before the application will block. This setting should be at least twice the value of `vm.dirty_background_bytes`, and may be higher. Experimentation is necessary to select the optimum setting of this value.

For more information on tuning the Linux virtual memory subsystem, consult the Linux kernel documentation.

14.2 Improving Stability by Increasing Open File Limits

Internally, HyperDex maintains multiple open file descriptors corresponding to network sockets and data files. Most Linux systems restrict the number of open files to 1024 by default. We recommend setting this value to 65536 or higher. This can be accomplished by adding the following line to `/etc/security/limits.conf` (the exact file may vary on your distribution):

```
*       -       nofile          65536
```

14.3 Deploying on EC2 and Other Xen Environments

Occasionally, Xen-based environments such as EC2 have bugs that demanding HyperDex workloads can expose. When deploying an EC2 cluster, ensure that all nodes are deployed with the EC2 enhanced networking to avoid such bugs. In environments where enhanced networking is unavailable, the bug can be worked around with the following commands:

```
# ethtool -K eth0 rx off tx off sg off tso off ufo off gso off gro off lro off
# sysctl -w net.ipv4.route.flush=1
```

Part III

API Reference

Chapter 15

C Client API

The HyperDex Client library, `libhyperdex-client` is the de facto way to access a HyperDex cluster for storing and retrieving data. All data-store operations are provided by the this library.

15.1 Building the HyperDex C Binding

The HyperDex C Binding is automatically built and installed via the normal HyperDex build and install process. You can ensure that the client is always built by providing the `--enable-client` option to `./configure` like so:

```
% ./configure --enable-client
```

15.2 Compiling and Linking Your Application

Unless otherwise noted, all Client operations are defined in the `hyperdex/client.h` include. You can include this in your own program with:

```
#include <hyperdex/client.h>
```

To link against `libhyperdex-client`, provide the `-lhyperdex-client` option at link time:

```
% cc -o output input.c -I/path/to/hyperdex/include -L/path/to/hyperdex/lib -lhyperdex-client
```

HyperDex provides support for the automatically determining the compiler and linker flags for `libhyperdex-client`. First, ensure the `pkg-config` program is installed, and then run:

```
% pkg-config --cflags hyperdex-client -I/usr/local/include
% pkg-config --libs hyperdex-client -L/usr/local/lib -lhyperdex-client
```

The first command outputs the compiler flags necessary to include the `hyperdex/client.h` file, while the second command outputs the flags necessary for linking against `libhyperdex-client`.

To put it all together, you can compile your application with:

```
% cc -o output input.c `pkg-config --cflags --libs hyperdex-client`
```

For more information about `pkg-config`, see the pkg-config documentation.

15.3 Hello World

The following is a minimal application that stores the value "Hello World" and then immediately retrieves the value:

```
/* WARNING:  This code omits all error checking and is for illustration only */
/* HyperDex Includes */
#include <hyperdex/client.h>
int
main(int argc, const char* argv[])
{
```

```
    struct hyperdex_client* client = NULL;
    struct hyperdex_client_attribute attr;
    struct hyperdex_client_attribute* attrs;
    size_t attrs_sz = 0;
    int64_t op_id;
    enum hyperdex_client_returncode op_status;
    int64_t loop_id;
    enum hyperdex_client_returncode loop_status;
    size_t i;

    client = hyperdex_client_create("127.0.0.1", 1982);

    attr.attr = "v";
    attr.value = "Hello World!";
    attr.value_sz = strlen(attr.value);
    attr.datatype = HYPERDATATYPE_STRING;

    /* perform the "put" */
    op_id = hyperdex_client_put(client, "kv", "some key", 8,
                                &attr, 1, &op_status);
    loop_id = hyperdex_client_loop(client, -1, &loop_status);
    printf("put \"Hello World!\"\n");

    /* perform the "get" */
    op_id = hyperdex_client_get(client, "kv", "some key", 8,
                                &op_status, &attrs, &attrs_sz);
    loop_id = hyperdex_client_loop(client, -1, &loop_status);
    printf("get done\n");

    for (i = 0; i < attrs_sz; ++i)
    {
        printf("got attribute \"%s\" = \"%.*s\"\n",
               attrs[i].attr, attrs[i].value_sz, attrs[i].value);
    }

    hyperdex_client_destroy_attrs(attrs, attrs_sz);
    hyperdex_client_destroy(client);
    return EXIT_SUCCESS;
}
```

You can compile and run this example with:

```
% cc -o hello-world hello-world.c `pkg-config --cflags --libs hyperdex-client`
% ./hello-world
put "Hello World!"
get done
got attribute "v" = "Hello World!"
```

Each operation, whether it is a PUT or a GET, is broken down into a request and a response. The `hyperdex_client_put` and `hyperdex_client_get` operations each initiate the request. The application then calls `hyperdex_client_loop` to wait for the completion of the operation. The status pointer passed to put or get will be used to fill in the status of the operation, while the status pointer passed to loop will be used to report transient problems with the cluster. Until the operation's id is returned from loop, the operation is still outstanding and incomplete.

The stored value is passed via `struct hyperdex_client_attribute`, which specifies both the value of the string "Hello World!" and that this value is, in fact, a string. All HyperDex datatypes are passed to HyperDex as bytestrings via this interface.

This code omits error checking, assuming that all operations succeed. In the subsequent documentation we'll explore proper error checking, which should be in place for all non-example code.

15.4 Asynchronous Patterns

All operations are issued asynchronously, that is, the operation's start is decoupled from its completion, and it is up to the application to wait for its completion.

The C API provides a unified, event-loop-like interface for polling events for their completion. An application can issue any number of operations, and poll for their completion using the `hyperdex_client_loop` call. This provides you with several advantages unavailable in any other distributed key-value store:

- Asynchronous events free your application to perform other operations while waiting for events to complete.
- Asynchronous events re-use underlying resources, such as TCP connections to the cluster, across operations. This enables improved performance and consumes fewer resources than would be consumed by synchronous clients generating a similar workload.
- Operations will be buffered in userspace when they cannot be immediately sent to a server. This ensures that applications will never block waiting for slow servers.

Of course, applications do not need to embrace this asynchronous design pattern. It's always possible to use the library in a synchronous manner by immediately following every operation with a call to `hyperdex_client_loop`.

Applications that do embrace this asynchronous design pattern will have a certain structure. Specifically:

- Each operation must eventually be followed by a call to `hyperdex_client_loop`. This is the core of the HyperDex client. All work, including flushing userspace buffers, occurs via the call to loop.
- Pointers used for output from operations must remain valid until `hyperdex_client_loop` indicates that the operation has successfully finished. Consequently, they must not be aliased to pointers passed to other operations.

Finally, it's important to realize that calling `hyperdex_client_loop` is necessary to complete operations. An operation's outcome is not determined until the application calls `hyperdex_client_loop`. Do not simply issue `hyperdex_client_put` operations (or similar) and assume that the operations complete because there is no guarantee that they will do so.

15.5 Creating a Client

A HyperDex client is encapsulated within the incomplete `struct hyperdex_client` type. This type is created by the HyperDex client library, and should only be freed using the provided method.

```
struct hyperdex_client*
hyperdex_client_create(const char* coordinator, uint16_t port);
```

Create a new client instance. This call allocates and initializes local structures. If allocation or initialization fail, the call will return NULL and set `errno` appropriately. This call does not establish the connection to the coordinator; that will be established and maintained automatically by other calls made with this client instance.

Parameters:

`coordinator` A C-string containing IP address or hostname of the coordinator.

`port` The port number for the coordinator.

```
void
hyperdex_client_destroy(struct hyperdex_client* client);
```

Destroy a previously instantiated client, and release all associated resources. This call always succeeds.

Parameters:

`client` A previously created client instance.

15.6 Data Structures

HyperDex natively supports a variety of data structures. This section describes the available data structures and their encoding within C. HyperDex encodes as a byte string all data structures passed between the application and HyperDex. This format of this byte string varies according to its type. In this section, we'll describe the format of data structures, and provide an API for serializing to and from the prescribed format. All APIs discussed in this section are provided by `libhyperdex-client`.

15.6.1 `enum hyperdatatype`

The `enum hyperdatatype` is used to represent the type of a byte string to HyperDex. Whenever a structure accepts a byte string as a value, it will typically accept an `enum hyperdatatype` to convey the type of the string.

Primitive Data Types Primitive data types are the basic data types of HyperDex. Applications may use these primitives as the key and dimensions of hyperspaces within HyperDex.

`HYPERDATATYPE_STRING` A byte string.
`HYPERDATATYPE_INT64` A 64-bit signed integer.
`HYPERDATATYPE_FLOAT` A 64-bit floating point value.

Container Data Types Container data types contain a collection of primitive data types. Container data types cannot be used as the key or dimensions of the hyperspace.

There are three container types available within HyperDex:

Lists A list contains elements of one primitive type. The order of elements in a list is preserved, and it's possible for duplicate elements to exist.

Sets A set contains elements of one primitive type. Each element exists in the set at most once. Although the implementation enforces an order on the set for efficiency purposes, set operations are agnostic to the order of the included elements.

Maps A map contains key-value pairs of elements, where the key and value may be of different types. Each key is unique and has an associated value. Maps also offer the ability to perform most primitive operations on the values within the map.

Each of these containers may be instantiated with a primitive data type as the contained elements. In total, HyperDex supports all of the following container data types:

`HYPERDATATYPE_LIST_STRING` A list of strings.
`HYPERDATATYPE_LIST_INT64` A list of integers.
`HYPERDATATYPE_LIST_FLOAT` A list of floats.
`HYPERDATATYPE_SET_STRING` A set of strings.
`HYPERDATATYPE_SET_INT64` A set of integers.
`HYPERDATATYPE_SET_FLOAT` A set of floats.
`HYPERDATATYPE_MAP_STRING_STRING` A map from strings to strings.
`HYPERDATATYPE_MAP_STRING_INT64` A map from strings to integers.
`HYPERDATATYPE_MAP_STRING_FLOAT` A map from strings to floats.
`HYPERDATATYPE_MAP_INT64_STRING` A map from integers to strings.
`HYPERDATATYPE_MAP_INT64_INT64` A map from integers to integers.
`HYPERDATATYPE_MAP_INT64_FLOAT` A map from integers to floats.
`HYPERDATATYPE_MAP_FLOAT_STRING` A map from floats to strings.
`HYPERDATATYPE_MAP_FLOAT_INT64` A map from floats to integers.
`HYPERDATATYPE_MAP_FLOAT_FLOAT` A map from floats to floats.

The following data types are defined as well, and are generally only of interest to HyperDex developers and those who are writing client bindings:

`HYPERDATATYPE_LIST_GENERIC` A list whose element type is unspecified.
`HYPERDATATYPE_SET_GENERIC` A set whose element type is unspecified.
`HYPERDATATYPE_MAP_GENERIC` A map whose key/value types are unspecified.
`HYPERDATATYPE_MAP_STRING_KEYONLY` A map whose key is a string and whose value is unspecified.
`HYPERDATATYPE_MAP_INT64_KEYONLY` A map whose key is an integer and whose value type is unspecified.
`HYPERDATATYPE_MAP_FLOAT_KEYONLY` A map whose key is a float and whose value type is unspecified.
`HYPERDATATYPE_GARBAGE` A reserved constant never used within HyperDex.

15.6.2 Bytestring Format

The format of the data structures is defined to be the same on all platforms.

For each format, Python-like psuedocode is provided that shows example encodings.

string format A string is an 8-bit byte string. HyperDex is agnostic to the contents of the string, and it may contain any bytes, including
`x00`. By convention, the trailing `NULL` should be omitted for C-strings to ensure interoperability across languages. For example:

```
>>> encode_string('Hello\x00World!')
b'Hello\x00World!'
```

int format Integers are encoded as signed 8-byte little-endian integers. For example:

```
>>> encode_int(1)
b'\x01\x00\x00\x00\x00\x00\x00\x00'
>>> encode_int(-1)
b'\xff\xff\xff\xff\xff\xff\xff\xff'
>>> encode_int(0xdeadbeef)
b'\xef\xbe\xad\xde\x00\x00\x00\x00'
```

float format Floats are encoded as IEEE 754 binary64 values in little-endian format. For example:

```
>>> encode_double(0)
b'\x00\x00\x00\x00\x00\x00\x00\x00'
>>> encode_double(3.1415)
b'o\x12\x83\xc0\xca!\t@'
```

list(string) format Lists of strings are encoded by concatenating the encoding of each string, prefixed by an unsigned 4-byte little endian integer indicating the length of the string. For example:

```
>>> encode_list_string([])
b''
>>> encode_list_string(['hello', 'world'])
b'\x05\x00\x00\x00hello\x05\x00\x00\x00world'
```

list(int) format Lists of integers are encoded by concatenating the encoded form of each integer. For example:

```
>>> encode_list_int([])
b''
>>> encode_list_int([1, -1, 0xdeadbeef])
b'\x01\x00\x00\x00\x00\x00\x00\x00' \
b'\xff\xff\xff\xff\xff\xff\xff\xff' \
b'\xef\xbe\xad\xde\x00\x00\x00\x00'
```

list(floats) format Lists of floats are encoded by concatenating the encoded form of each float. For example:

```
>>> encode_list_float([])
b''
>>> encode_list_float([0, 3.1415])
b'\x00\x00\x00\x00\x00\x00\x00\x00' \
b'o\x12\x83\xc0\xca!\t@'
```

set(string) format Sets of strings are encoded by concatenating the encoding of each string in sorted order, where each string is prefixed by an unsigned 4-byte little endian integer indicating the length of the string. For example:

```
>>> encode_set_string([])
b''
>>> encode_set_string(['world', 'hello'])
b'\x05\x00\x00\x00hello\x05\x00\x00\x00world'
```

set(int) format Sets of integers are encoded by concatenating the encoded form of each integer in sorted order. For example:

```
>>> encode_set_int([])
b''
>>> encode_set_int([1, -1, 0xdeadbeef])
b'\xff\xff\xff\xff\xff\xff\xff\xff' \
b'\x01\x00\x00\x00\x00\x00\x00\x00' \
b'\xef\xbe\xad\xde\x00\x00\x00\x00'
```

set(float) format Sets of floats are encoded by concatenating the encoded form of each float in sorted order. For example:

```
>>> encode_set_float([])
b''
>>> encode_set_float([3.1415, 0])
b'\x00\x00\x00\x00\x00\x00\x00\x00' \
b'o\x12\x83\xc0\xca!\t@'
```

map(string, string) format Maps from strings to strings are formed by encoding the individual elements, each prefixed by an unsigned 4-byte little endian integer indicating their length. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_string_string({})
b''
>>> encode_map_string_string({'hello': 'world',
...                           'map key': 'map val',
...                           'map', 'encoding'})
b'\x05\x00\x00\x00hello\x05\x00\x00\x00world' \
b'\x03\x00\x00\x00map\x08\x00\x00\x00encoding' \
b'\x07\x00\x00\x00map key\x07\x00\x00\x00map val'
```

map(string, int) format Maps from strings to ints are formed by encoding the individual elements, where keys are prefixed by an unsigned 4-byte little endian integer indicating their length. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_string_int({})
b''
>>> encode_map_string_int({'world': -1,
...                        'hello': 1})
b'\x05\x00\x00\x00hello\x01\x00\x00\x00\x00\x00\x00\x00' \
b'\x05\x00\x00\x00world\xff\xff\xff\xff\xff\xff\xff\xff'
```

map(string, float) format Maps from strings to floats are formed by encoding the individual elements, where keys are prefixed by an unsigned 4-byte little endian integer indicating their length. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_string_float({})
b''
>>> encode_map_string_float({'zero': 0,
...                          'pi': 3.1415})
b'\x02\x00\x00\x00pio\x12\x83\xc0\xca!\t@' \
b'\x04\x00\x00\x00zero\x00\x00\x00\x00\x00\x00\x00\x00'
```

map(int, string) format Maps from ints to strings are formed by encoding the individual elements, where values are prefixed by an unsigned 4-byte little endian integer indicating their length. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_int_string({})
b''
>>> encode_map_int_string({1: 'hello',
...                        -1: 'world'})
b'\xff\xff\xff\xff\xff\xff\xff\xff\x05\x00\x00\x00world' \
b'\x01\x00\x00\x00\x00\x00\x00\x00\x05\x00\x00\x00hello'
```

map(int, int) format Maps from ints to ints are formed by encoding the individual elements. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_int_int({})
b''
>>> encode_map_int_int({1: 0xdeadbeef,
...                     -1: 0x1eaff00d})
b'\xff\xff\xff\xff\xff\xff\xff\xff\x0d\xf0\xaf\x1e\x00\x00\x00\x00' \
b'\x01\x00\x00\x00\x00\x00\x00\x00\xef\xbe\xad\xde\x00\x00\x00\x00'
```

map(int, float) format Maps from ints to floats are formed by encoding the individual elements. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_int_float({})
b''
>>> encode_map_int_float({1: 0,
...                       -1: 3.1415})
b'\xff\xff\xff\xff\xff\xff\xff\xffo\x12\x83\xc0\xca!\t@' \
b'\x01\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00'
```

map(float, string) format Maps from floats to strings are formed by encoding the individual elements, where values are prefixed by an unsigned 4-byte little endian integer indicating their length. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_float_string({})
b''
>>> encode_map_float_string({0: 'hello',
...                          3.1415: 'world'})
b'\x00\x00\x00\x00\x00\x00\x00\x00\x05\x00\x00\x00hello' \
b'o\x12\x83\xc0\xca!\t@\x05\x00\x00\x00world'
```

map(float, int) format Maps from floats to ints are formed by encoding the individual elements. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_float_int({})
b''
>>> encode_map_float_int({0: 1,
...                       3.1415: -1})
b'\x00\x00\x00\x00\x00\x00\x00\x00\x01\x00\x00\x00\x00\x00\x00\x00' \
b'o\x12\x83\xc0\xca!\t@\xff\xff\xff\xff\xff\xff\xff\xff'
```

map(float, float) format Maps from floats to floats are formed by encoding the individual elements. The pairs of elements are stored in sorted order according to the first element of the pair (the map's key). For example:

```
>>> encode_map_float_float({})
b''
>>> encode_map_float_float({0: 1,
...                         3.1415: -1})
b'\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\x00\xf0?' \
b'o\x12\x83\xc0\xca!\t@\x00\x00\x00\x00\x00\x00\xf0\xbf'
```

15.6.3 Serialization API

The serialization API supports serialization of all datatypes supported by HyperDex. Of course, feel free to manually encode data structures, especially where doing so can make use of efficient stack-allocated data structures.

struct hyperdex_ds_arena The packing routines described below may occasionally have to allocate memory into which the encoded forms of the datatypes are copied. To free the programmer from the burden of having to manually allocate and free each of these pieces of memory, the data structures API allocates all memory via an instance of `struct hyperdex_ds_arena`. Via a single call to `hyperdex_ds_arena_destroy`, all memory allocated via the arena is free'd.

`struct hyperdex_ds_arena` is intentionally defined as an incomplete type because its internals are subject to change. To create an arena, call `hyperdex_ds_arena_create`. The arena should subsequently be destroyed via `hyperdex_ds_arena_destroy`.

```
struct hyperdex_ds_arena* hyperdex_ds_arena_create();
```

Create a new arena for alocating memory. On success, this function returns a non-null pointer for the new arena. On failure, the function returns `NULL`, indicating that memory allocation failed. It is the caller's responsibility to pass this function to `hyperdex_ds_arena_destroy` when finished.

```
void hyperdex_ds_arena_destroy(struct hyperdex_ds_arena* arena);
```

Free all memory associated with `arena`. This function always succeeds.

serialize string No serialization is necessary for string data types. For convenience, the serialization API provides a copy function that copies a string into arena-allocated memory.

```
int hyperdex_ds_copy_string(struct hyperdex_ds_arena* arena, const char* str,
                            size_t str_sz, enum hyperdex_ds_returncode* status,
                            const char** value, size_t* value_sz);
```

Copy the string `str`/`str_sz` into memory allocated via `arena` and return the copy via `value` and `value_sz`. This function will fail and return -1 if there is insufficient memory available for copying the string. All pointers returned by this function remain valid until `arena` is destroyed. The client should not attempt to free the returned copy.

serialize int

```
void hyperdex_ds_pack_int(int64_t num, char* value);
```

Packs `num` into the bytes pointed to by `buf`. This function always succeeds. It is the caller's responsibility to ensure that `buf` points to at least 8 B.

```
int hyperdex_ds_copy_int(struct hyperdex_ds_arena* arena, int64_t num,
                         enum hyperdex_ds_returncode* status,
                         const char** value, size_t* value_sz);
```

Encode `num` into memory allocated via `arena` and return the value via `value` and `value_sz`. This function will fail and return -1 if there is insufficient memory available for encoding the number. All pointers returned by this function remain valid until `arena` is destroyed. The client should not attempt to free the returned copy.

serialize float

```
void hyperdex_ds_pack_float(double num, char* value);
```

Pack `num` into the bytes pointed to by `buf`. This function always succeeds. It is the caller's responsibility to ensure that `buf` points to at least 8 B.

```
int hyperdex_ds_copy_float(struct hyperdex_ds_arena* arena, double num,
                           enum hyperdex_ds_returncode* status,
                           const char** value, size_t* value_sz);
```

Encode `num` into memory allocated via `arena` and return the value via `value` and `value_sz`. This function will fail and return -1 if there is insufficient memory available for encoding the nubmer. All pointers returned by this function remain valid until `arena` is destroyed. The client should not attempt to free the returned copy.

serialize lists The below functions incrementally build lists, performing all relevant error checking to ensure that the resuling HyperDex list is well-formed. The first element appended to the list implicitly determines the type of the list. All subsequent calls that push elements of a different type will fail.

```
struct hyperdex_ds_list* hyperdex_ds_allocate_list(struct hyperdex_ds_arena* arena);
```

Create a new dynamic list. This function will fail and return `NULL` should memory allocation fail.

```
int hyperdex_ds_list_insert_string(struct hyperdex_ds_list* list,
                                   const char* str, size_t str_sz,
                                   enum hyperdex_ds_returncode* status);
```

Append the string `str`/`str_sz` to `list`. This function will fail and return -1 if memory allocation fails, or the list is not a list of strings.

```
int hyperdex_ds_list_insert_int(struct hyperdex_ds_list* list, int64_t num,
                                enum hyperdex_ds_returncode* status);
```

Append the integer `num` to `list`. This function will fail and return -1 if memory allocation fails, or the list is not a list of integers.

```
int hyperdex_ds_list_insert_float(struct hyperdex_ds_list* list, double num,
                                  enum hyperdex_ds_returncode* status);
```

Append the float `num` to `list`. This function will fail and return -1 if memory allocation fails or the list is not a list of floats.

```
int hyperdex_ds_list_finalize(struct hyperdex_ds_list*,
                              enum hyperdex_ds_returncode* status,
                              const char** value, size_t* value_sz,
                              enum hyperdatatype* datatype);
```

Finalize the list by writing its elements into a bytestring. This function returns the bytestring and the list type. It will fail and return -1 if memory allocation fails.

serialize sets The below functions incrementally build sets, performing all relevant error checking to ensure that the resuling HyperDex set is well-formed. The first element inserted into the set implicitly determines the type of the set. All subsequent calls that insert elements of different types will fail.

```
struct hyperdex_ds_set* hyperdex_ds_allocate_set(struct hyperdex_ds_arena* arena);
```

Create a new dynamic set. This function will fail and return `NULL` should memory allocation fail.

```
int hyperdex_ds_set_insert_string(struct hyperdex_ds_set* set,
                                  const char* str, size_t str_sz,
                                  enum hyperdex_ds_returncode* status);
```

Insert the string `str`/`str_sz` into `set`. This function will fail and return -1 if memory allocation fails, or the set is not a set of strings.

```
int hyperdex_ds_set_insert_int(struct hyperdex_ds_set* set, int64_t num,
                               enum hyperdex_ds_returncode* status);
```

Insert the integer `num` into `set`. This function will fail and return -1 if memory allocation fails, or the set is not a set of integers.

```
int hyperdex_ds_set_insert_float(struct hyperdex_ds_set* set, double num,
                                 enum hyperdex_ds_returncode* status);
```

Insert the float `num` into `set`. This function will fail and return -1 if memory allocation fails, or the set is not a set of floats.

```
int hyperdex_ds_set_finalize(struct hyperdex_ds_set*,
                             enum hyperdex_ds_returncode* status,
                             const char** value, size_t* value_sz,
                             enum hyperdatatype* datatype);
```

Finalize the set by writing its elements into a bytestring. This function returns the bytestring and the set type. It will fail and return -1 if memory allocation fails.

serialize maps The below functions incrementally build maps, performing all relevant error checking to ensure that the resuling HyperDex map is well-formed. The first key/value-pair inserted into the map implicitly determines the type of the map. All subsequent calls that insert elements of different types will fail.

The map is built by alternating calls to the key/value functions described below, starting with a key-based function. This keeps the number of cases linear in the number of primitive types a map may contain, rather than appending key-value pairs directly (which would require a quadratic number of calls).

```
struct hyperdex_ds_map* hyperdex_ds_allocate_map(struct hyperdex_ds_arena* arena);
```

Create a new dynamic map. This function will fail and return `NULL` should memory allocation fail.

```
int hyperdex_ds_map_insert_key_string(struct hyperdex_ds_map* map,
                                      const char* str, size_t str_sz,
                                      enum hyperdex_ds_returncode* status);
```

Set the key of the next pair to be inserted into `map` to the string specified by `str` and `str_sz`. This function will fail and return -1 if memory allocation fails, or the map does not use strings for keys.

```
int hyperdex_ds_map_insert_val_string(struct hyperdex_ds_map* map,
                                      const char* str, size_t str_sz,
                                      enum hyperdex_ds_returncode* status);
```

Set the value of the next pair to be inserted into `map` to the string specified by `str` and `str_sz`, and insert the pair. This function will fail and return -1 if memory allocation fails, or the map does not use strings for values.

```
int hyperdex_ds_map_insert_key_int(struct hyperdex_ds_map* map,
                                   int64_t num,
                                   enum hyperdex_ds_returncode* status);
```

Set the key of the next pair to be inserted into `map` to the integer specified by `num`. This function will fail and return -1 if memory allocation fails, or the map does not use integers for keys.

```
int hyperdex_ds_map_insert_val_int(struct hyperdex_ds_map* map,
                                   int64_t num,
                                   enum hyperdex_ds_returncode* status);
```

Set the value of the next pair to be inserted into `map` to the integer specified by `num`, and insert the pair. This function will fail and return -1 if memory allocation fails, or the map does not use integers for values.

```
int hyperdex_ds_map_insert_key_float(struct hyperdex_ds_map* map,
                                     double num,
                                     enum hyperdex_ds_returncode* status);
```

Set the key of the next pair to be inserted into `map` to the float specified by `num`. This function will fail and return -1 if memory allocation fails, or the map does not use floats for keys.

```
int hyperdex_ds_map_insert_val_float(struct hyperdex_ds_map* map,
                                     double num,
                                     enum hyperdex_ds_returncode* status);
```

Set the value of the next pair to be inserted into `map` to the float specified by `num`. This function will fail and return -1 if memory allocation fails, or the map does not use floats for values.

```
int hyperdex_ds_map_finalize(struct hyperdex_ds_map*,
                             enum hyperdex_ds_returncode* status,
                             const char** value, size_t* value_sz,
                             enum hyperdatatype* datatype);
```

Finalize the map by writing its key/value-pairs into a bytestring. This function returns the bytestring and the map type. It will fail and return -1 if memory allocation fails, or an uneven number of key/value calls were made.

15.6.4 Deserialization API

The deserialization API provides routines to unpack ints and floats, and iterate the elements in lists, sets, and maps. Iterators return elements one-by-one with a minimal amount of copying and allocation. All iterators are used in the same pattern. For example, to iterate a list of integers:

```
/* C Includes */
#include <stdio.h>
#include <stdlib.h>

/* HyperDex Includes */
#include <hyperdex/datastructures.h>

int
main(int argc, const char* argv[])
{
   int64_t num;
```

```
    const char* value = "\x01\x00\x00\x00\x00\x00\x00\x00"
                        "\xff\xff\xff\xff\xff\xff\xff\xff"
                        "\xef\xbe\xad\xde\x00\x00\x00\x00";
    size_t value_sz = 24;
    int status = 0;
    hyperdex_ds_iterator iter;
    hyperdex_ds_iterator_init(&iter, HYPERDATATYPE_LIST_INT64, value, value_sz);

    while ((status = hyperdex_ds_iterate_list_int_next(&iter, &num)) > 0)
    {
        printf("%ld\n", num);
    }

    if (status < 0)
    {
        printf("error: corrupt list\n");
    }

    return EXIT_SUCCESS;
}
```

Compile and run this example with:

```
$ cc -o iterate iterate.c `pkg-config --cflags --libs hyperdex-client`
$ ./iterate
1
-1
3735928559
```

The function `hyperdex_ds_iterator_init` sets up the iterator. Each container data type has a specialized iteration function. All iterators share the same initialization function.

```
void hyperdex_ds_iterator_init(struct hyperdex_ds_iterator* iter,
                               enum hyperdatatype datatype,
                               const char* value,
                               size_t value_sz);
```

Initialize an iterator for the given data type/value. This function always succeeds.

deserialize string No deserialization is necessary for string data types.

deserialize int

```
int hyperdex_ds_unpack_int(const char* buf, size_t buf_sz, int64_t* num);
```

Unpack `num` from `buf`/`buf_sz`. This function will fail and return -1 if `buf_sz` is not exactly 8 B.

deserialize float

```
int hyperdex_ds_unpack_float(const char* buf, size_t buf_sz, double* num);
```

Unpack `num` from `buf`/`buf_sz`. This function will fail and return -1 if `buf_sz` is not exactly 8 B.

deserialize lists

```
int hyperdex_ds_iterate_list_string_next(struct hyperdex_ds_iterator* iter,
                                         const char** str, size_t* str_sz);
```

Return the next string element in the list. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the list of strings is malformed. The value stored in `*str` is a pointer into the list of strings and should not be free'd by the application.

```
int hyperdex_ds_iterate_list_int_next(struct hyperdex_ds_iterator* iter, int64_t* num);
```

Return the next integer element in the list. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the list of integers is malformed.

```
int hyperdex_ds_iterate_list_float_next(struct hyperdex_ds_iterator* iter, double* num);
```

Return the next float element in the list. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the list of floats is malformed.

deserialize sets

```
int hyperdex_ds_iterate_set_string_next(struct hyperdex_ds_iterator* iter,
                                        const char** str, size_t* str_sz);
```

Return the next string element in the set. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the set of strings is malformed. The value stored in `*str` is a pointer into the set of strings and should not be free'd by the application.

```
int hyperdex_ds_iterate_set_int_next(struct hyperdex_ds_iterator* iter, int64_t* num);
```

Return the next integer element in the set. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the set of ints is malformed.

```
int hyperdex_ds_iterate_set_float_next(struct hyperdex_ds_iterator* iter, double* num);
```

Return the next float element in the set. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the set of ints is malformed.

deserialize maps

```
int hyperdex_ds_iterate_map_string_string_next(struct hyperdex_ds_iterator* iter,
                                               const char** key, size_t* key_sz,
                                               const char** val, size_t* val_sz);
```

Return the next pair of (string, string) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed. The values stored in `*key` and `*val` are pointers into the map and should not be free'd by the application.

```
int hyperdex_ds_iterate_map_string_int_next(struct hyperdex_ds_iterator* iter,
                                            const char** key, size_t* key_sz,
                                            int64_t* val);
```

Return the next pair of (string, int) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed. The value stored in `*key` is a pointer into the map and should not be free'd by the application.

```
int hyperdex_ds_iterate_map_string_float_next(struct hyperdex_ds_iterator* iter,
                                              const char** key, size_t* key_sz,
                                              double* val);
```

Return the next pair of (string, float) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed. The value stored in `*key` is a pointer into the map and should not be free'd by the application.

```
int hyperdex_ds_iterate_map_int_string_next(struct hyperdex_ds_iterator* iter,
                                            int64_t* key,
                                            const char** val, size_t* val_sz);
```

Return the next pair of (int, string) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed. The value stored in `*val` is a pointer into the map and should not be free'd by the application.

```
int hyperdex_ds_iterate_map_int_int_next(struct hyperdex_ds_iterator* iter,
                                         int64_t* key, int64_t* val);
```

Return the next pair of (int, int) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed.

```
int hyperdex_ds_iterate_map_int_float_next(struct hyperdex_ds_iterator* iter,
                                           int64_t* key, double* val);
```

Return the next pair of (int, float) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed.

```
int hyperdex_ds_iterate_map_float_string_next(struct hyperdex_ds_iterator* iter,
                                              double* key,
                                              const char** val, size_t* val_sz);
```

Return the next pair of (float, string) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed. The value stored in `*val` is a pointer into the map and should not be free'd by the application.

```
int hyperdex_ds_iterate_map_float_int_next(struct hyperdex_ds_iterator* iter,
                                           double* key, int64_t* val);
```

Return the next pair of (float, int) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed.

```
int hyperdex_ds_iterate_map_float_float_next(struct hyperdex_ds_iterator* iter,
                                             double* key, double* val);
```

Return the next pair of (float, float) in the map. This function will return 1 if an element is returned, 0 if there are no elements to return, and -1 if the map is malformed.

15.6.5 Memory Management Utilties

The data structures API provides utility functions for allocating structures from the arena, obviating the need to free them individually.

```
struct hyperdex_client_attribute*
hyperdex_ds_allocate_attribute(struct hyperdex_ds_arena* arena, size_t sz);
```

Allocate an array of `struct hyperdex_client_attribute`. On success, this function returns a non-null pointer containing `sz` elements. On failure, the function returns `NULL`, indicating that memory allocation failed. The memory will remain valid until the arena is destroyed and should not be free'd independently by the application.

```
struct hyperdex_client_attribute_check*
hyperdex_ds_allocate_attribute_check(struct hyperdex_ds_arena* arena, size_t sz);
```

Allocate an array of `struct hyperdex_client_attribute_check`. On success, this function returns a non-null pointer containing `sz` elements. On failure, the function returns `NULL`, indicating that memory allocation failed. The memory will remain valid until the arena is destroyed and should not be free'd independently by the application.

```
struct hyperdex_client_map_attribute*
hyperdex_ds_allocate_map_attribute(struct hyperdex_ds_arena* arena, size_t sz);
```

Allocate an array of `struct hyperdex_client_map_attribute`. On success, this function returns a non-null pointer containing `sz` elements. On failure, the function returns `NULL`, indicating that memory allocation failed. The memory will remain valid until the arena is destroyed and should not be free'd independently by the application.

15.7 Attributes

In HyperDex, attributes specify named values that comprise an object. For instance, in Chapter Quick Start, the phonebook space has attributes "username", "first", "last", and "phone". The C API represents such attributes using `struct hyperdex_client_attribute`. The C definition of this struct is:

```
struct hyperdex_client_attribute
{
    const char* attr; /* NULL-terminated */
    const char* value;
    size_t value_sz;
    enum hyperdatatype datatype;
};
```

This struct specifies the name, value, and data type of the attribute. The `attr` field is a NULL-terminated C-string that names the attribute affected by the value. The `value` and `value_sz` fields contain a properly formatted byte string and its size. The `datatype` field indicates the encoding of the byte string.

The interpretation of an attribute is dependent upon the operation being performed. In the case of `hyperdex_client_put`, the attributes directly convey the values to be stored; `hyperdex_client_get` returns the stored attributes. Other operations, such as `hyperdex_client_string_prepend` interpret the attribute as an argument to the operation. In the case of prepend, the attribute specifies the value to be prepended.

15.8 Map Attributes

Some HyperDex operations affect key-value pairs contained within maps. These operations use `struct hyperdex_client_map_attribute` to specify the name of the attribute affected, the key within the map, and the value associated with that key. The C definition of this struct is:

```
struct hyperdex_client_map_attribute
{
    const char* attr; /* NULL-terminated */
    const char* map_key;
    size_t map_key_sz;
    enum hyperdatatype map_key_datatype;
    const char* value;
    size_t value_sz;
    enum hyperdatatype value_datatype;
};
```

The struct specifies the name, key, and value of to be used for the operation. The `map_key_*` and `value_*` fields specify, respectively, the key and value of the element within the map. The relative fields are specified as byte strings with associated data types. The `attr` field specifies the name of the map.

15.9 Predicates

In HyperDex, a predicate is an expression about an attribute that is true or false. Predicates are specified to HyperDex using `struct hyperdex_client_attribute_check` which is defined as:

```
struct hyperdex_client_attribute_check
{
    const char* attr; /* NULL-terminated */
    const char* value;
    size_t value_sz;
    enum hyperdatatype datatype;
    enum hyperpredicate predicate;
};
```

Note that this struct closely resembles `struct hyperdex_client_attribute`, with the addition of a field named `predicate`. This field is an enum with the following values:

`HYPERPREDICATE_FAIL` Always fail.

`HYPERPREDICATE_EQUALS` Check that the existing value is equal to the one specified by `value/value_sz`.

`HYPERPREDICATE_LESS_EQUAL` Check that the existing value is less than or equal to the one specified by `value/value_sz`.

`HYPERPREDICATE_GREATER_EQUAL` Check that the existing value is greater than or equal to the one specified by `value/value_sz`.

`HYPERPREDICATE_REGEX` Check that the existing value matches the regular expression stored in as a string in `value/value_sz`.

`HYPERPREDICATE_LENGTH_EQUALS` Check that the existing container or string has a length equal to the integer stored in `value/value_sz`.

`HYPERPREDICATE_LENGTH_LESS_EQUAL` Check that the existing container or string has a length less than or equal to the integer stored in `value/value_sz`.

`HYPERPREDICATE_LENGTH_GREATER_EQUAL` Check that the existing container or string has a length greater than or equal to the integer stored in `value/value_sz`.

`HYPERPREDICATE_CONTAINS` Check that the container contains an element matching `value/value_sz`.

15.10 Error Handling

Every call in the client provides a means for reporting failure. After each call, your application should check for the error and react appropriately. Depending upon the error, your application may retry the request, or may need to take more drastic action.

Errors are typically reported via `enum hyperdex_client_returncode` defined in `hyperdex/client.h`. Values for this enum fall into three categories: values returned during normal operation, values returned to indicate anticipated errors, and values that should never be returned in practice.

The common-case returncodes are:

`HYPERDEX_CLIENT_SUCCESS` The operation was successful and no errors occurred.

`HYPERDEX_CLIENT_NOTFOUND` The operation finished because the requested object was not found.

`HYPERDEX_CLIENT_SEARCHDONE` An operation that potentially returns multiple objects. For instance, a search has finished and will no longer be returned via loop.

`HYPERDEX_CLIENT_CMPFAIL` The predicate specified as part of a conditional operation was not true.

`HYPERDEX_CLIENT_READONLY` The cluster is in read-only mode and not accepting write operations.

The following errors stem from environmental problems or problems with the application's usage of HyperDex. These errors are generally easy to remedy and are anticipated by the client library.

`HYPERDEX_CLIENT_UNKNOWNSPACE` The specified space does not exist. Ensure that a space exists before trying to manipulate its data.

`HYPERDEX_CLIENT_COORDFAIL` The connection to the coordinator has failed. The application should back off before retrying.

Note that the connection to the coordinator is not a simple TCP connection, and is redundant if the coordinator consists of multiple servers. This error indicates to the application that the redundancy has failed and that the application should back-off.

`HYPERDEX_CLIENT_SERVERERROR` A server returned a nonsensical result to the client library. Generally retrying the request should be sufficient to overcome the problem.

`HYPERDEX_CLIENT_POLLFAILED` The poll system call failed in an unexpected manner. Typically, this means that the application using the HyperDex library has mismanaged its file descriptors and improperly altered descriptors in use by the HyperDex library.

This generally indicates that there is a bug in the HyperDex client library, the application using the library, or both.

`HYPERDEX_CLIENT_OVERFLOW` An integer operation failed to complete because it would have resulted in signed overflow.

`HYPERDEX_CLIENT_RECONFIGURE` The server responsible for managing the operation failed while the operation was in-flight.

`HYPERDEX_CLIENT_TIMEOUT` The `hyperdex_client_loop` operation exceeded its timeout without completing an outstanding operation. This does not affect the status of any outstanding operation.

`HYPERDEX_CLIENT_UNKNOWNATTR` The operation references an attribute that is not part of the space. Make sure to use attributes within the space's schema.

`HYPERDEX_CLIENT_DUPEATTR` The operation references an attribute multiple times in a way that is not permitted.

`HYPERDEX_CLIENT_NONEPENDING` The `hyperdex_client_loop` call was made, but no operations were outstanding. This generally indicates that the loop call was called too many times for the operations issued.

`HYPERDEX_CLIENT_DONTUSEKEY` The operation attempted to mutate the key, which is not permitted.

`HYPERDEX_CLIENT_WRONGTYPE` The attribute or predicate was not compatible with the type in the space's schema. For example, the operation may have attempted to issue a PUT operation that writes an integer to a non-integer datatype, or tries to perform string operations on a non-string data type. Check that the operation performed is compatible with the data type of the affected attributes.

`HYPERDEX_CLIENT_NOMEM` The library failed to allocate memory.

`HYPERDEX_CLIENT_INTERRUPTED` The HyperDex library was interrupted by a signal. Read more about signals in Section 15.12.

`HYPERDEX_CLIENT_CLUSTER_JUMP` The client library started receiving configurations for a different HyperDex cluster. This can happen if a new coordinator is restarted on the same address that the client connects to. This error will not be persistent. The client library will switch to the new cluster configuration, and this error just serves as a notification to the application.

`HYPERDEX_CLIENT_OFFLINE` All servers responsible for handling the specified operation are currently offline and unavailable, whether due to failure or planned downtime.

The following errors indicate significant bugs within the client or application. In practice they should never happen and indicate bugs within HyperDex itself. They are used as one would use an `assert` statement to enforce an invariant.

`HYPERDEX_CLIENT_INTERNAL` One or more of the HyperDex client library's internal invariants have been broken. It's best to destroy and recreate the client.

`HYPERDEX_CLIENT_EXCEPTION` The C library is implemented internally using C++. C++ generated an unhandled exception that was caught at the C boundary. This indicates a bug in HyperDex, and exists only as a safeguard. Applications should never see this error.

`HYPERDEX_CLIENT_GARBAGE` This value is reserved as a well-defined value that the library will never return, and that is not used as a constant anywhere else within HyperDex.

Note that an asynchronous application should distinguish between local errors which affect one outstanding operation, and global errors that transiently affect all operations, but do not change the completion status of those operations.

Local errors are always returned via the `enum hyperdex_client_returncode*` pointer passed at the time the application initiated the operation. These errors may either result from the application returning a negative operation id, in which case the operation immediately completes, or from the result of a successful `hyperdex_client_loop` call. In either case, the error has no impact on the result of any other operation.

Global errors are always returned via the `enum hyperdex_client_returncode*` pointer passed to the most recent invocation of `hyperdex_client_loop`. These errors are not localized to any particular operation and indicate errors that are non-permanent. Example global errors include application-requested timeouts, interruptions by signals, and temporary non-connectivity with the coordinator.

15.11 Operations

15.11.1 get

Retrieve an object from `space` using `key`.

Definition:

```
int64_t hyperdex_client_get(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        enum hyperdex_client_returncode* status,
        const struct hyperdex_client_attribute** attrs, size_t* attrs_sz);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const struct hyperdex_client_attribute** attrs, size_t* attrs_sz`
  An array of attributes that comprise a returned object. The application must free the returned values with `hyperdex_client_destroy_attrs`. The pointers must remain valid until the operation completes.

15.11.2 get_partial

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
int64_t hyperdex_client_get_partial(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const char** attrnames, size_t attrnames_sz,
        enum hyperdex_client_returncode* status,
        const struct hyperdex_client_attribute** attrs, size_t* attrs_sz);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const char** attrnames, size_t attrnames_sz`
  A list of attributes to return. `attrnames` points to an array of length `attrnames_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const struct hyperdex_client_attribute** attrs, size_t* attrs_sz`
  An array of attributes that comprise a returned object. The application must free the returned values with `hyperdex_client_destroy_attrs`. The pointers must remain valid until the operation completes.

15.11.3 put

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
int64_t hyperdex_client_put(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.4 cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_put(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.5 cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_put_or_create(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.6 group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_put(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.7 put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
int64_t hyperdex_client_put_if_not_exist(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.8 del

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_del(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.9 cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_del(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.10 group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_del(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.11 atomic_add

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.12 cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.13 group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_add(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.14 atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_sub(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.15 cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_sub(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.16 group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_sub(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.17 atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_mul(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.18 cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_mul(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.19 group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_mul(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.20 atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_div(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.21 cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_div(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.22 group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_div(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.23 atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_mod(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.24 cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_mod(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.25 group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_mod(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.26 atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_and(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.27 cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_and(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.28 group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_and(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.29 atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_atomic_or(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.30 cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_or(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.31 group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_or(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.32 atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_atomic_xor(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.33 cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_xor(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.34 group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_xor(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.35 atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_atomic_min(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.36 cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_min(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.37 group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_min(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.38 atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_atomic_max(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.39 cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_atomic_max(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.40 group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_atomic_max(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.41 string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_string_prepend(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.42 cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_string_prepend(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.43 group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_string_prepend(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.44 string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_string_append(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.45 cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_string_append(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.46 group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_string_append(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.47 string_ltrim

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_string_ltrim(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.48 cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_string_ltrim(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.49 group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_string_ltrim(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.50 string_rtrim

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_string_rtrim(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.51 cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_string_rtrim(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.52 group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_string_rtrim(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.53 list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_list_lpush(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.54 cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_list_lpush(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.55 group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_list_lpush(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.56 list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_list_rpush(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.57 cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_list_rpush(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.58 group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_list_rpush(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.59 set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_set_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.60 cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_set_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.61 group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_set_add(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.62 set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_set_remove(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.63 cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_set_remove(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.64 group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_set_remove(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.65 set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_set_intersect(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.66 cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_set_intersect(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.67 group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_set_intersect(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.68 set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_set_union(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.69 cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_set_union(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.70 group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_set_union(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.71 document_rename

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_document_rename(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.72 cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_document_rename(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.73 group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_document_rename(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.74 document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_document_unset(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.75 cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_document_unset(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.76 group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_document_unset(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.77 map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.78 cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.79 group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_add(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.80 map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_remove(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.81 cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_remove(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.82 group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_remove(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_attribute* attrs, size_t attrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_attribute* attrs, size_t attrs_sz`
  The set of attributes to modify and their respective values. `attrs` points to an array of length `attrs_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.83 map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_atomic_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.84 cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_add(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.85 group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_add(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.86 map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_map_atomic_sub(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.87 cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_sub(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.88 group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_sub(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.89 map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_map_atomic_mul(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.90 cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_mul(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.91 group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_mul(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.92 map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_atomic_div(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.93 cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_div(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.94 group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_div(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.95 map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_map_atomic_mod(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.96 cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_mod(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.97 group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_mod(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.98 map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_atomic_and(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.99 cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_and(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.100 group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_and(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.101 map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_atomic_or(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.102 cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_or(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.103 group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_or(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.104 map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_atomic_xor(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.105 cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_xor(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.106 group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_xor(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.107 map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
int64_t hyperdex_client_map_string_prepend(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.108 cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_string_prepend(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.109 group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_string_prepend(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.110 map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_map_string_append(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.111 cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_string_append(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.112 group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_string_append(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.113 map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_map_atomic_min(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.114 cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_min(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.115 group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_min(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.116 map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
int64_t hyperdex_client_map_atomic_max(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.117 cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
int64_t hyperdex_client_cond_map_atomic_max(struct hyperdex_client* client,
        const char* space,
        const char* key, size_t key_sz,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const char* key, size_t key_sz`
  The key for the operation where `key` is a bytestring and `key_sz` specifies the number of bytes in `key`.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

15.11.118 group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
int64_t hyperdex_client_group_map_atomic_max(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const struct hyperdex_client_map_attribute* mapattrs, size_t mapattrs_sz`
  The set of map attributes to modify and their respective key/values. `mapattrs` points to an array of length `mapattrs_sz`. Each entry specify an attribute that is a map and a key within that map.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.11.119 search

Return all objects that match the specified `checks`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
int64_t hyperdex_client_search(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        enum hyperdex_client_returncode* status,
        const struct hyperdex_client_attribute** attrs, size_t* attrs_sz);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const struct hyperdex_client_attribute** attrs, size_t* attrs_sz`
  An array of attributes that comprise a returned object. The application must free each of the returned values with `hyperdex_client_destroy_attrs`. The pointers must remain valid until the operation completes.

15.11.120 sorted_search

Return all objects that match the specified `checks`, sorted according to `attr`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
int64_t hyperdex_client_sorted_search(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        const char* sort_by,
        uint64_t limit,
        int maxmin,
        enum hyperdex_client_returncode* status,
        const struct hyperdex_client_attribute** attrs, size_t* attrs_sz);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.
- `const char* sort_by`
  The attribute to sort by.
- `uint64_t limit`
  The number of results to return.
- `int maxmin`
  Maximize (!= 0) or minimize (== 0).

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const struct hyperdex_client_attribute** attrs, size_t* attrs_sz`
  An array of attributes that comprise a returned object. The application must free each of the returned values with `hyperdex_client_destroy_attrs`. The pointers must remain valid until the operation completes.

15.11.121 count

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
int64_t hyperdex_client_count(struct hyperdex_client* client,
        const char* space,
        const struct hyperdex_client_attribute_check* checks, size_t checks_sz,
        enum hyperdex_client_returncode* status,
        uint64_t* count);
```

Parameters:

- `struct hyperdex_client* client`
  The HyperDex client connection to use for the operation.
- `const char* space`
  The name of the space as a C-string.
- `const struct hyperdex_client_attribute_check* checks, size_t checks_sz`
  A set of predicates to check against. `checks` points to an array of length `checks_sz`.

Returns:

- `enum hyperdex_client_returncode* status`
  The status of the operation. The client library will fill in this variable before returning this operation's request id from `hyperdex_client_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `uint64_t* count`
  The number of objects which match the predicates.

15.12 Working with Signals

The HyperDex client library is signal-safe. Should a signal interrupt the client during a blocking operation, it will return `HYPERDEX_CLIENT_INTERRUPTED`.

15.13 Working with Threads

The HyperDex client library is fully reentrant. Instances of `struct hyperdex_client` and their associated state may be accessed from multiple threads, provided that the application employes its own synchronization that provides mutual exclusion.

Put simply, a multi-threaded application should protect each `struct hyperdex_client` instance with a mutex or lock to ensure correct operation.

Chapter 16

C Admin API

16.1 Operations

16.1.1 read_only

Change the cluster to and from read-only mode.

Definition:

```
int64_t hyperdex_admin_read_only(struct hyperdex_admin* admin,
        int ro,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `int ro`
  Flag indicating whether the cluster is set to read-only mode or read-write mode. Set to non-zero for read-only mode, and zero for read-write mode.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.2 wait_until_stable

Wait until the cluster is stable. The cluster is considered stable when all servers acknowledge the current configuration, and all initiated in previous configurations complete.

Definition:

```
int64_t hyperdex_admin_wait_until_stable(struct hyperdex_admin* admin,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.3 fault_tolerance

Change the fault tolerance of `space` to the specified fault tolerance threshold.

Definition:

```
int64_t hyperdex_admin_fault_tolerance(struct hyperdex_admin* admin,
        const char* space,
        uint64_t ft,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* space`
  The name of an existing space.
- `uint64_t ft`
  The fault-tolerance threshold. HyperDex will deploy at least one more server than this threshold to ensure that this many servers may simultaneously fail without introducing data loss or downtime.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.4 validate_space

Validate the provided space description.

Definition:

```
int hyperdex_admin_validate_space(struct hyperdex_admin* admin,
        const char* description,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* description`
  A complete space description.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.5 add_space

Add a new space to the cluster, using the provided space description.

Definition:

```
int64_t hyperdex_admin_add_space(struct hyperdex_admin* admin,
        const char* description,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* description`
  A complete space description.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.6 rm_space

Remove `space` from the configuration and make it inaccessible to future operations.

Definition:

```
int64_t hyperdex_admin_rm_space(struct hyperdex_admin* admin,
        const char* space,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* space`
  The name of an existing space.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.7 mv_space

Move space from name `source` to name `target`. This is a metadata operation and has the same cost as `add_space`, even for populated spaces.

Definition:

```
int64_t hyperdex_admin_mv_space(struct hyperdex_admin* admin,
        const char* source,
        const char* target,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* source`
  The name of the existing space.
- `const char* target`
  The new name for the space.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.8 list_spaces

List all spaces held by the cluster.

Definition:

```
int64_t hyperdex_admin_list_spaces(struct hyperdex_admin* admin,
        enum hyperdex_admin_returncode* status,
        const char** spaces);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const char** spaces`
  A list of spaces created within the cluster. The list is a multi-line C-string with one space per line. The C-string is valid until the next call into the admin library, and will automatically be freed by the library. It should not be changed or freed by the library user.

16.1.9 list_indices

XXX

Definition:

```
int64_t hyperdex_admin_list_indices(struct hyperdex_admin* admin,
        const char* space,
        enum hyperdex_admin_returncode* status,
        const char** indexes);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* space`
  The name of an existing space.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const char** indexes`
  XXX

16.1.10 list_subspaces

XXX

Definition:

```
int64_t hyperdex_admin_list_subspaces(struct hyperdex_admin* admin,
        const char* space,
        enum hyperdex_admin_returncode* status,
        const char** subspaces);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* space`
  The name of an existing space.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const char** subspaces`
  XXX

16.1.11 add_index

Add a new index on a HyperDex attribute or document

Definition:

```
int64_t hyperdex_admin_add_index(struct hyperdex_admin* admin,
        const char* space,
        const char* attribute,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* space`
  The name of an existing space.
- `const char* attribute`
  The name of an attribute or document.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.12 rm_index

Remove an existing index

Definition:

```
int64_t hyperdex_admin_rm_index(struct hyperdex_admin* admin,
        uint64_t idxid,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `uint64_t idxid`
  The index's unique identifier. This will be an integer that identifies the index across the lifetime of the cluster.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.13 server_register

Manually register a new server with `token` bound to `address`.

Definition:

```
int64_t hyperdex_admin_server_register(struct hyperdex_admin* admin,
        uint64_t token,
        const char* address,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `uint64_t token`
  The server's token which uniquely identifies the server. This will be logged by the server with the message prefix "generated new random token", and is available via the administration tools that can introspect the cluster state.
- `const char* address`
  An IP/port pair to which the server is bound, specified as a human-readable c-string. For instance, `127.0.0.1:2012` specifies localhost port 2012.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.14 server_online

Manually set server `token` to state `AVAILABLE`.

Definition:

```
int64_t hyperdex_admin_server_online(struct hyperdex_admin* admin,
        uint64_t token,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `uint64_t token`
  The server's token which uniquely identifies the server. This will be logged by the server with the message prefix "generated new random token", and is available via the administration tools that can introspect the cluster state.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.15 server_offline

Manually mark server `token` as `OFFLINE`.

Definition:

```
int64_t hyperdex_admin_server_offline(struct hyperdex_admin* admin,
        uint64_t token,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `uint64_t token`
  The server's token which uniquely identifies the server. This will be logged by the server with the message prefix "generated new random token", and is available via the administration tools that can introspect the cluster state.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.16 server_forget

Completely remove all state associated `token` from the cluster.

Definition:

```
int64_t hyperdex_admin_server_forget(struct hyperdex_admin* admin,
        uint64_t token,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `uint64_t token`
  The server's token which uniquely identifies the server. This will be logged by the server with the message prefix "generated new random token", and is available via the administration tools that can introspect the cluster state.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.17 server_kill

Manually change server `token` into the `KILLED` state.

Definition:

```
int64_t hyperdex_admin_server_kill(struct hyperdex_admin* admin,
        uint64_t token,
        enum hyperdex_admin_returncode* status);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `uint64_t token`
  The server's token which uniquely identifies the server. This will be logged by the server with the message prefix "generated new random token", and is available via the administration tools that can introspect the cluster state.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.

16.1.18 backup

Initiate a cluster-wide backup named `backup`. This has the same behavior as the `hyperdex backup` command discussed in Chapter 11.

Definition:

```
int64_t hyperdex_admin_backup(struct hyperdex_admin* admin,
        const char* backup,
        enum hyperdex_admin_returncode* status,
        const char** backups);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.
- `const char* backup`
  The name for the backup created by this operation.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `const char** backups`
  A list of servers that were backed up. The list is a C-string with lines of the form `"<token> <IP address> <path>"`. The C-string is valid until the next call into the admin library, and will automatically be freed by the library. It should not be changed or freed by the library user.

16.1.19 enable_perf_counters

Start collecting performance counters from all servers in the cluster.

Definition:

```
int64_t hyperdex_admin_enable_perf_counters(struct hyperdex_admin* admin,
        enum hyperdex_admin_returncode* status,
        struct hyperdex_admin_perf_counter* pc);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.

Returns:

- `enum hyperdex_admin_returncode* status`
  The status of the operation. The admin library will fill in this variable before returning this operation's request id from `hyperdex_admin_loop`. The pointer must remain valid until the operation completes, and the pointer should not be aliased to the status for any other outstanding operation.
- `struct hyperdex_admin_perf_counter* pc`
  A struct where the admin library can store returned performance counters. Memory regions returned via the struct will be valid until the next call into the admin library and are managed by the library. They should not be freed by the user.
  Only one performance counter may be enabled at a time. Consecutive calls to `enable_perf_counters` will return the same ID as the previous call, and will return performance counters via the previously-used struct. To change the structure used for returning performance counters, first disable the performance counters and then re-enable them.

16.1.20 disable_perf_counters

Stop collecting performance counters from all servers in the cluster.

Definition:

```
void hyperdex_admin_disable_perf_counters(struct hyperdex_admin* admin);
```

Parameters:

- `struct hyperdex_admin* admin`
  The HyperDex admin connection to use for the operation.

Returns: Nothing

Chapter 17

Go Client API

HyperDex provide Go bindings to the client under the import path `github.com/rescrv/HyperDex/bindings/go/client`. This library wraps the C client library and enables the use of native Go data types and uses Go's native support for concurrency to exploit the asynchronous features of HyperDex.

17.0.1 Using Go Within Your Application

All client operation are defined in the `client` package. You can access this in your program with:

```
import ("github.com/rescrv/HyperDex/bindings/go/client")
```

In your go environment, run this to fetch the module into your `GOPATH`.

```
go get github.com/rescrv/HyperDex/bindings/go/client
```

17.0.2 Hello World

The following is a minimal application that stores the value "Hello World" and then immediately retrieves the value:

```
package main

import "fmt"
import "github.com/rescrv/HyperDex/bindings/go/client"

func main() {
        c, e, errChan := client.NewClient("127.0.0.1", 1982)
        // permanent error connecting from the cluster
        if e != nil {
                // handle the error here
        }
        // handle transient errors connecting to the cluster
        go func() {
                for e := range errChan {
                        // handle the error here
                        fmt.Println("error", e)
                }
        }()
        // do the operations
        err := c.Put("kv", "some key", client.Attributes{"v": "Hello World!"})
        if err != nil {
                // handle the error here
        }
        fmt.Println("put: \"Hello World!\"")
        obj, err := c.Get("kv", "some key")
        if err != nil {
```

```
                // handle the error here
        }
        fmt.Println("got:", obj)
}
```

You can run this example with:

```
% go run hello-world.go
put: "Hello World!"
got: map[v:Hello World!]
```

In Go, every HyperDex operation appears to the application as a synchronous call. Operations will appear to block the calling go-routine and suspend execution until the operation completes. Behind the scenes, the client will use the concurrency features of Go to allow many concurrent operations using the same client library. Additionally, Go types are automatically converted to HyperDex's types under the hood.

17.0.3 Data Structures

The Go bindings automatically manage conversion of data types from Go to HyperDex types, enabling applications to be written in idiomatic Go.

17.0.3.1 Examples

This section shows examples of Go data structures that are recognized by HyperDex. The examples here are for illustration purposes and are not exhaustive.

Strings The HyperDex client recognizes Go's strings and automatically converts them to HyperDex strings. For example:

```
err := client.Put("kv", "some key", client.Attributes{"v": "some value"})
```

Integers The HyperDex client recognizes Go's integers and automatically converts them to HyperDex integers. For example:

```
err := client.Put("kv", "some key", client.Attributes{"v": 42})
err = client.Put("kv", "some key", client.Attributes{"v": int64(42)})
```

Floats The HyperDex client recognizes Go's floating point numbers and automatically converts them to HyperDex floats. For example:

```
err := client.Put("kv", "some key", client.Attributes{"v": 3.14})
```

Lists The HyperDex client recognizes lists and automatically converts them to HyperDex lists. For example:

```
err := client.Put("kv", "some key", client.Attributes{"v": client.List{"A", "B", "C"}})
```

Sets The HyperDex client recognizes sets and automatically converts them to HyperDex sets. For example:

```
err := client.Put("kv", "some key", client.Attributes{"v": client.Set{"A", "B", "C"}})
```

Maps The HyperDex client recognizes Go dictionaries and automatically converts them to HyperDex maps. For example:

```
err := client.Put("kv", "some key", client.Attributes{"v":
        client.Map{"A": "X", "B": "Y", "C": "Z"}})
```

17.0.4 Attributes

Attributes in Go are provided as struct `client.Attributes`, which has the same syntax as a Go map. As you can see in the examples above, attributes are specified in the form:

```
client.Attributes{"v1": "A", "v2": 3.14}
```

17.0.5 Map Attributes

Map attributes in Go are provided as struct `client.MapAttributes`, which maps attribute names to `client.Map` objects that specify the key-value pairs to manipulate.

```
client.MapAttributes{"v": client.Map{"A": 3.14}}
```

17.0.6 Predicates

Predicates in Go are specified as a slice of `client.Predicate`. Here's a variety of Predicates provided by Go:

```
client.Predicate{"attr", "some value", client.EQUALS}
client.Predicate{"attr", 5, client.LESS_THAN}
client.Predicate{"attr", 5, client.LESS_EQUAL}
client.Predicate{"attr", 5, client.GREATER_THAN}
client.Predicate{"attr", 5, client.GREATER_EQUAL}
client.Predicate{"attr", "^prefix", client.REGEX}
client.Predicate{"attr", 42, client.LENGTH_EQUALS}
client.Predicate{"attr", 42, client.LENGTH_LESS_EQUALS}
client.Predicate{"attr", 42, client.LENGTH_GREATER_EQUALS}
client.Predicate{"attr", "needle", client.CONTAINS}
```

17.0.7 Error Handling

All error handling within the Go bindings is done via `client.Error` returned from each operation. If an error object is returned, its status should be checked following the call. For example, we may conditionally change the value of `v` from `"foo"` to `"bar"` like this:

```
err := client.CondPut("kv", "some key",
        []client.Predicate{{"v", "foo", client.EQUALS}},
        client.Attributes{"v": "bar"})
if err != nil {
    // it worked
} else if err.Status == client.CMPFAIL {
    // the existing value was not "foo"
} else if err.Status == client.NOTFOUND {
    // there is no existing value
} else {
    // something more fatal happened
    // this is an exceptional case
    // propagate the error or try again
}
```

17.0.8 Operations

17.0.8.1 `Get`

Retrieve an object from `space` using `key`.

Definition:

```
func (client *Client) Get(spacename string, key Value) (attrs Attributes, err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.

Returns: The object (if any) as a `client.Value` and a `client.Error` object.

17.0.8.2 `GetPartial`

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
func (client *Client) GetPartial(spacename string, key Value, attributenames AttributeNames) (attrs Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributenames`
  A list of attributes to return.

Returns: The object (if any) as a `client.Value` and a `client.Error` object.

17.0.8.3 `Put`

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
func (client *Client) Put(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.4 CondPut

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondPut(spacename string, key Value, predicates []Predicate, attributes Attribute
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.5 CondPutOrCreate

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondPutOrCreate(spacename string, key Value, predicates []Predicate, attributes A
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.6 GroupPut

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupPut(spacename string, predicates []Predicate, attributes Attributes) (count
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.7 PutIfNotExist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
func (client *Client) PutIfNotExist(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.8 `Del`

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) Del(spacename string, key Value) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.9 CondDel

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondDel(spacename string, key Value, predicates []Predicate) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.10 GroupDel

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupDel(spacename string, predicates []Predicate) (count uint64, err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.11 `AtomicAdd`

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicAdd(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.12 `CondAtomicAdd`

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicAdd(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.13 GroupAtomicAdd

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicAdd(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.14 `AtomicSub`

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicSub(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.15 CondAtomicSub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicSub(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.16 GroupAtomicSub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicSub(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.17 `AtomicMul`

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicMul(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.18 CondAtomicMul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicMul(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.19 GroupAtomicMul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicMul(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.20 `AtomicDiv`

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicDiv(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.21 CondAtomicDiv

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicDiv(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.22 GroupAtomicDiv

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicDiv(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.23 `AtomicMod`

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicMod(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.24 CondAtomicMod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicMod(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.25 GroupAtomicMod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicMod(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.26 AtomicAnd

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicAnd(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.27 `CondAtomicAnd`

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicAnd(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.28 `GroupAtomicAnd`

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicAnd(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.29 AtomicOr

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
func (client *Client) AtomicOr(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Go type.
- attributes
  The attributes to modify and their respective values.

Returns: A client.Error object indicating the status of the operation.

17.0.8.30 `CondAtomicOr`

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicOr(spacename string, key Value, predicates []Predicate, attributes Attr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.31 `GroupAtomicOr`

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicOr(spacename string, predicates []Predicate, attributes Attributes) (c
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.32 AtomicXor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicXor(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.33 `CondAtomicXor`

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicXor(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.34 `GroupAtomicXor`

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicXor(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.35 AtomicMin

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicMin(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.36 `CondAtomicMin`

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicMin(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.37 `GroupAtomicMin`

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicMin(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.38 AtomicMax

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) AtomicMax(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.39 CondAtomicMax

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondAtomicMax(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.40 GroupAtomicMax

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupAtomicMax(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.41 `StringPrepend`

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) StringPrepend(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.42 CondStringPrepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondStringPrepend(spacename string, key Value, predicates []Predicate, attributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.43 GroupStringPrepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupStringPrepend(spacename string, predicates []Predicate, attributes Attribute
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.44 `StringAppend`

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) StringAppend(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.45 CondStringAppend

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondStringAppend(spacename string, key Value, predicates []Predicate, attributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.46 GroupStringAppend

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupStringAppend(spacename string, predicates []Predicate, attributes Attributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.47 `StringLtrim`

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) StringLtrim(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.48 CondStringLtrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondStringLtrim(spacename string, key Value, predicates []Predicate, attributes A
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.49 `GroupStringLtrim`

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupStringLtrim(spacename string, predicates []Predicate, attributes Attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.50 `StringRtrim`

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) StringRtrim(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.51 CondStringRtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondStringRtrim(spacename string, key Value, predicates []Predicate, attributes A
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.52 GroupStringRtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupStringRtrim(spacename string, predicates []Predicate, attributes Attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.53 ListLpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) ListLpush(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.54 CondListLpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondListLpush(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.55 GroupListLpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupListLpush(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.56 ListRpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) ListRpush(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.57 CondListRpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondListRpush(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.58 GroupListRpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupListRpush(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.59 SetAdd

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) SetAdd(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.60 `CondSetAdd`

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondSetAdd(spacename string, key Value, predicates []Predicate, attributes Attrib
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.61 GroupSetAdd

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupSetAdd(spacename string, predicates []Predicate, attributes Attributes) (cou
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.62 SetRemove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) SetRemove(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.63 CondSetRemove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondSetRemove(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.64 `GroupSetRemove`

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupSetRemove(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.65 `SetIntersect`

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) SetIntersect(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.66 `CondSetIntersect`

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondSetIntersect(spacename string, key Value, predicates []Predicate, attributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.67 GroupSetIntersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupSetIntersect(spacename string, predicates []Predicate, attributes Attributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.68 SetUnion

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) SetUnion(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.69 CondSetUnion

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondSetUnion(spacename string, key Value, predicates []Predicate, attributes Attr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.70 GroupSetUnion

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupSetUnion(spacename string, predicates []Predicate, attributes Attributes) (c
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.71 `DocumentRename`

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) DocumentRename(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.72 CondDocumentRename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondDocumentRename(spacename string, key Value, predicates []Predicate, attribute
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.73 GroupDocumentRename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupDocumentRename(spacename string, predicates []Predicate, attributes Attribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.74 `DocumentUnset`

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) DocumentUnset(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.75 CondDocumentUnset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondDocumentUnset(spacename string, key Value, predicates []Predicate, attributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.76 `GroupDocumentUnset`

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupDocumentUnset(spacename string, predicates []Predicate, attributes Attribute
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.77 `MapAdd`

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAdd(spacename string, key Value, mapattributes MapAttributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.78 `CondMapAdd`

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAdd(spacename string, key Value, predicates []Predicate, mapattributes Map
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.79 `GroupMapAdd`

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAdd(spacename string, predicates []Predicate, mapattributes MapAttributes
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.80 MapRemove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapRemove(spacename string, key Value, attributes Attributes) (err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.81 CondMapRemove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapRemove(spacename string, key Value, predicates []Predicate, attributes Att
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.82 GroupMapRemove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapRemove(spacename string, predicates []Predicate, attributes Attributes) (
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `attributes`
  The attributes to modify and their respective values.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.83 MapAtomicAdd

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicAdd(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.84 CondMapAtomicAdd

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicAdd(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.85 GroupMapAtomicAdd

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicAdd(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.86 MapAtomicSub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicSub(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.87 CondMapAtomicSub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicSub(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.88 GroupMapAtomicSub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicSub(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.89 MapAtomicMul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicMul(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.90 CondMapAtomicMul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicMul(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.91 GroupMapAtomicMul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicMul(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.92 MapAtomicDiv

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicDiv(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.93 CondMapAtomicDiv

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicDiv(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.94 GroupMapAtomicDiv

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicDiv(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.95 MapAtomicMod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicMod(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.96 CondMapAtomicMod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicMod(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.97 `GroupMapAtomicMod`

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicMod(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.98 MapAtomicAnd

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicAnd(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.99 CondMapAtomicAnd

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicAnd(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.100 GroupMapAtomicAnd

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicAnd(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.101 MapAtomicOr

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicOr(spacename string, key Value, mapattributes MapAttributes) (err *Error
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.102 CondMapAtomicOr

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicOr(spacename string, key Value, predicates []Predicate, mapattribute
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.103 GroupMapAtomicOr

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicOr(spacename string, predicates []Predicate, mapattributes MapAttri
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.104 MapAtomicXor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicXor(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.105 CondMapAtomicXor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicXor(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.106 GroupMapAtomicXor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicXor(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.107 MapStringPrepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapStringPrepend(spacename string, key Value, mapattributes MapAttributes) (err *
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.108 CondMapStringPrepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapStringPrepend(spacename string, key Value, predicates []Predicate, mapattr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.109 GroupMapStringPrepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapStringPrepend(spacename string, predicates []Predicate, mapattributes Map
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.110 MapStringAppend

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
func (client *Client) MapStringAppend(spacename string, key Value, mapattributes MapAttributes) (err *E
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Go type.
- mapattributes
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A client.Error object indicating the status of the operation.

17.0.8.111 `CondMapStringAppend`

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapStringAppend(spacename string, key Value, predicates []Predicate, mapattri
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.112 GroupMapStringAppend

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapStringAppend(spacename string, predicates []Predicate, mapattributes MapA
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.113 MapAtomicMin

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicMin(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.114 CondMapAtomicMin

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicMin(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.115 GroupMapAtomicMin

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicMin(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.116 MapAtomicMax

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
func (client *Client) MapAtomicMax(spacename string, key Value, mapattributes MapAttributes) (err *Erro
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.117 CondMapAtomicMax

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
func (client *Client) CondMapAtomicMax(spacename string, key Value, predicates []Predicate, mapattribut
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Go type.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A `client.Error` object indicating the status of the operation.

17.0.8.118 GroupMapAtomicMax

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
func (client *Client) GroupMapAtomicMax(spacename string, predicates []Predicate, mapattributes MapAttr
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `mapattributes`
  The set of maps to modify, and the respective key-value pairs in each map.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

17.0.8.119 `Search`

Return all objects that match the specified `checks`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
func (client *Client) Search(spacename string, predicates []Predicate) (attrs chan Attributes, errs cha
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.

Returns: Two channels, one for returning objects that match the search, and one for returning errors encountered during the search.

17.0.8.120 SortedSearch

Return all objects that match the specified `checks`, sorted according to `attr`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
func (client *Client) SortedSearch(spacename string, predicates []Predicate, sortby string, limit uint3
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.
- `sortby`
  The attribute to sort by.
- `limit`
  The number of results to return.
- `maxmin`
  Maximize (`'max'`) or minimize (`'min'`).

Returns: Two channels, one for returning objects that match the search, and one for returning errors encountered during the search.

17.0.8.121 `Count`

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
func (client *Client) Count(spacename string, predicates []Predicate) (count uint64, err *Error)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A slice of `client.Predicate` objects to match against.

Returns: A count of the number of objects, and a `client.Error` object indicating the status of the operation.

Chapter 18

Java API

18.1 Client Library

HyperDex provides Java bindings in the package `org.hyperdex.client`. This package wraps the HyperDex C Client library and enables the use of native Java data types.

This library was brought up-to-date following the 1.0.5 release.

18.1.1 Building the HyperDex Java Binding

The HyperDex Java Binding must be requested at configure time as it is not automatically built. You can ensure that the Java bindings are always built by providing the `--enable-java-bindings` option to `./configure` like so:

```
% ./configure --enable-client --enable-java-bindings
```

18.1.2 Using Java Within Your Application

All client operation are defined in the `org.hyperdex.client` package. You can access this in your program with:

```
import org.hyperdex.client.*;
```

18.1.3 Hello World

The following is a minimal application that stores the value "Hello World" and then immediately retrieves the value:

```
import java.util.*;
import org.hyperdex.client.*;

public class HelloWorld
{
    public static void main(String[] args) throws HyperDexClientException
    {
        Client c = new Client("127.0.0.1", 1982);
        /* put */
        Map<String, Object> attrs = new HashMap<String, Object>();
        attrs.put("v", "Hello World!");
        System.out.println("put: " + c.put("kv", "k", attrs));
        /* get */
        System.out.println("got: " + c.get("kv", "k"));
    }
}
```

You can run this example with:

```
% javac HelloWorld.java
% java -Djava.library.path=/usr/local/lib HelloWorld
put: true
got: {v=Hello World!}
```

Right away, there are several points worth noting in this example:

- Every operation is synchronous. The PUT and GET operations run to completion by default.
- Java types are automatically converted to HyperDex types. There's no need to specify information such as the length of each string, as one would do with the C API.
- We specify -Djava.library.path=/usr/local/lib. This is necessary for builds from source, but should not be necessary for Java bindings installed using binary packages.

18.1.4 Asynchronous Operations

For convenience, the Java bindings treat every operation as synchronous. This enables you to write short programs without concern for asynchronous operations. Most operations come with an asynchronous form, denoted by the `async_` prefix. For example, the above Hello World example could be rewritten in asynchronous fashion as such:

```
import java.util.*;
import org.hyperdex.client.*;

public class HelloWorldAsyncWait
{
    public static void main(String[] args) throws HyperDexClientException
    {
        Client c = new Client("127.0.0.1", 1982);
        /* put */
        Map<String, Object> attrs = new HashMap<String, Object>();
        attrs.put("v", "Hello World!");
        Deferred dp = c.async_put("kv", "k", attrs);
        System.out.println("put: " + dp.waitForIt());
        /* get */
        Deferred dg = c.async_get("kv", "k");
        System.out.println("got: " + dg.waitForIt());
    }
}
```

This enables applications to issue multiple requests simultaneously and wait for their completion in an application-specific order. It's also possible to use the `loop` method on the client object to wait for the next request to complete:

```
import java.util.*;
import org.hyperdex.client.*;

public class HelloWorldAsyncLoop
{
    public static void main(String[] args) throws HyperDexClientException
    {
        Client c = new Client("127.0.0.1", 1982);
        /* put */
        Map<String, Object> attrs = new HashMap<String, Object>();
        attrs.put("v", "Hello World!");
        Deferred dl;
```

```
        Deferred dp = c.async_put("kv", "k", attrs);
        dl = (Deferred)c.loop();
        assert dl == dp;
        System.out.println("put: " + dl.waitForIt());
        /* get */
        Deferred dg = c.async_get("kv", "k");
        dl = (Deferred)c.loop();
        assert dl == dg;
        System.out.println("got: " + dl.waitForIt());
    }
}
```

18.1.5 Data Structures

The Java bindings automatically manage conversion of data types from Java to HyperDex types, enabling applications to be written in idiomatic Java.

18.1.5.1 Examples

This section shows examples of Java data structures that are recognized by HyperDex. The examples here are for illustration purposes and are not exhaustive.

Strings The HyperDex client recognizes Java's strings and automatically converts them to HyperDex strings. For example, the following call stores a string: equivalent and have the same effect:

```
Map<String, Object> attrs = new HashMap<String, Object>();
attrs.put("v", "someattrs");
c.put("kv", "somekey", attrs);
```

Integers The HyperDex client recognizes Java's integers and automatically converts them to HyperDex integers. For example:

```
Map<String, Object> attrs = new HashMap<String, Object>();
attrs.put("v", 42);
c.put("kv", "somekey", attrs);
```

Floats The HyperDex client recognizes Java's floating point numbers and automatically converts them to HyperDex floats. For example:

```
Map<String, Object> attrs = new HashMap<String, Object>();
attrs.put("v", 3.1415);
c.put("kv", "somekey", attrs);
```

Lists The HyperDex client recognizes Java lists and automatically converts them to HyperDex lists. For example:

```
List<Object> list = new ArrayList<Object>();
list.add("a");
list.add("b");
list.add("c");
Map<String, Object> attrs = new HashMap<String, Object>();
attrs.put("v", list);
c.put("kv", "somekey", attrs);
```

Sets The HyperDex client recognizes Java sets and automatically converts them to HyperDex sets. For example:

```
Set<Object> set = new HashSet<Object>();
set.add("a");
set.add("b");
set.add("c");
Map<String, Object> attrs = new HashMap<String, Object>();
attrs.put("v", set);
c.put("kv", "somekey", attrs);
```

Maps The HyperDex client recognizes Java maps and automatically converts them to HyperDex maps. For example:

```
Map<Object, Object> map = new HashMap<Object, Object>();
map.put("k", "v");
Map<String, Object> attrs = new HashMap<String, Object>();
attrs.put("v", map);
c.put("kv", "somekey", attrs);
```

18.1.6 Attributes

Attributes in Java are specified in the form of a map from attribute names to their values. As you can see in the examples above, attributes are specified in the form:

```
Map<String, Object> attrs = new HashMap<String, Object>();
```

18.1.7 Map Attributes

Map attributes in Java are specified in the form of a nested map. The outer map key specifies the name, while the inner map key-value pair's specify the key-value pair of the map. For example:

```
Map<String, Map<Object, Object>> mapattrs = new HashMap<String, Map<Object, Object>>();
```

18.1.8 Predicates

Predicates in Java are specified in the form of a hash from attribute names to their predicates. In the simple case, the predicate is just a value to be compared against:

```
Map<String, Object> checks = new HashMap<String, Object>();
checks.put("v", "value");
```

This is the same as saying:

```
Map<String, Object> checks = new HashMap<String, Object>();
checks.put("v", new Equals("value"));
```

The Java bindings support the full range of predicates supported by HyperDex itself. For example:

```
checks.put("v", new LessEqual(5));
checks.put("v", new GreaterEqual(5));
checks.put("v", new RangeEqual(5, 10));
checks.put("v", new Regex("^s.*"));
checks.put("v", new LengthEquals(5));
checks.put("v", new LengthLessEqual(5));
checks.put("v", new LengthGreaterEqual(5));
checks.put("v", new Contains('value'));
```

18.1.9 Error Handling

All error handling within the Java bindings is done via the `try/catch` mechanism of Java. Errors will be thrown by the package and should be handled by your application. For example, if we were trying to store an integer (5) as attribute `"v"`, where `"v"` is actually a string, we'd generate an error.

```
try
{
    attrs.put("v", 5);
    System.out.println("put: " + c.put("kv", "k", attrs));
}
catch (HyperDexClientException e)
{
    System.out.println(e.status());
    System.out.println(e.symbol());
    System.out.println(e.message());
}
```

Errors of type `HyperDexClientException` will contain both a message indicating what went wrong, as well as the underlying `enum hyperdex_client_returncode`. The member `status` indicates the numeric value of this enum, while `symbol` returns the enum as a string. The above code will fail with the following output:

```
8525
HYPERDEX_CLIENT_WRONGTYPE
invalid attribute "v": attribute has the wrong type
```

18.1.10 Operations

18.1.10.1 `get`

Retrieve an object from `space` using `key`.

Definition:

```
public Map<String, Object> get(
        String spacename,
        Object key) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- The returned object, represented as a map from attributes' names to Java objects.
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.2 async_get

Retrieve an object from `space` using `key`.

Definition:

```
public Deferred async_get(
        String spacename,
        Object key) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- The returned object, represented as a map from attributes' names to Java objects.
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `get`.

18.1.10.3 get_partial

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
public Map<String, Object> get_partial(
        String spacename,
        Object key,
        List<String> attributenames) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `List<String> attributenames`
  A list of attributes to return. `attrnames` is a `List<String>`.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- The returned object, represented as a map from attributes' names to Java objects.
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.4 async_get_partial

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
public Deferred async_get_partial(
        String spacename,
        Object key,
        List<String> attributenames) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `List<String> attributenames`
  A list of attributes to return. `attrnames` is a `List<String>`.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- The returned object, represented as a map from attributes' names to Java objects.
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `get_partial`.

18.1.10.5 `put`

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
public Boolean put(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.6 async_put

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
public Deferred async_put(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `put`.

18.1.10.7 cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_put(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.8 async_cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_put(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_put`.

18.1.10.9 cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_put_or_create(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.10 async_cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_put_or_create(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_put_or_create`.

18.1.10.11 group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_put(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.12 async_group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_put(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_put`.

18.1.10.13 put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
public Boolean put_if_not_exist(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.14 async_put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
public Deferred async_put_if_not_exist(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `put_if_not_exist`.

18.1.10.15 `del`

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean del(String spacename, Object key) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.16 async_del

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_del(
        String spacename,
        Object key) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `del`.

18.1.10.17 cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_del(
        String spacename,
        Object key,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.18 async_cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_del(
        String spacename,
        Object key,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_del`.

18.1.10.19 group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_del(
        String spacename,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.20 async_group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_del(
        String spacename,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_del`.

18.1.10.21 atomic_add

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_add(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.22 async_atomic_add

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_add(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_add`.

18.1.10.23 cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.24 async_cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_add`.

18.1.10.25 group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.26 async_group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_add`.

18.1.10.27 atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_sub(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.28 async_atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_sub(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_sub`.

18.1.10.29 cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_sub(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.30 async_cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_sub(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_sub`.

18.1.10.31 group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_sub(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.32 async_group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_sub(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_sub`.

18.1.10.33 atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_mul(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.34 async_atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_mul(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_mul`.

18.1.10.35 cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_mul(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.36 async_cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_mul(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_mul`.

18.1.10.37 group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_mul(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.38 async_group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_mul(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_mul`.

18.1.10.39 atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_div(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.40 async_atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_div(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_div`.

18.1.10.41 cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_div(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.42 async_cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_div(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_div`.

18.1.10.43 group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_div(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.44 async_group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_div(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_div`.

18.1.10.45 atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_mod(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.46 async_atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_mod(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_mod`.

18.1.10.47 cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_mod(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.48 async_cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_mod(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_mod`.

18.1.10.49 group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_mod(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.50 async_group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_mod(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_mod`.

18.1.10.51 atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_and(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.52 async_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
public Deferred async_atomic_and(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- String spacename
  The name of the space as a String.
- Object key
  The key for the operation where key is a Java object.
- Map<String, Object> attributes
  The set of attributes to modify and their respective values. attrs is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a Deferred object with a waitForIt method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- True if the operation succeeded
- False if any provided predicates failed.
- null if the operation requires an existing value and none exists

On error, this function will raise a HyperDexClientException describing the error.

See also: This is the asynchronous form of atomic_and.

18.1.10.53 cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_and(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.54 async_cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_and(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_and`.

18.1.10.55 group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_and(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.56 async_group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_and(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_and`.

18.1.10.57 atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_or(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.58 async_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_or(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_or`.

18.1.10.59 cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_or(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.60 async_cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_or(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_or`.

18.1.10.61 group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_or(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.62 async_group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_or(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_or`.

18.1.10.63 atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_xor(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.64 async_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
public Deferred async_atomic_xor(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- String spacename
  The name of the space as a String.
- Object key
  The key for the operation where key is a Java object.
- Map<String, Object> attributes
  The set of attributes to modify and their respective values. attrs is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a Deferred object with a waitForIt method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- True if the operation succeeded
- False if any provided predicates failed.
- null if the operation requires an existing value and none exists

On error, this function will raise a HyperDexClientException describing the error.

See also: This is the asynchronous form of atomic_xor.

18.1.10.65 cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_xor(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.66 async_cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_xor(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_xor`.

18.1.10.67 group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_xor(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.68 async_group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_xor(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_xor`.

18.1.10.69 atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_min(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.70 async_atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_min(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_min`.

18.1.10.71 cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_min(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.72 async_cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_min(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_min`.

18.1.10.73 group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_min(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.74 async_group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_min(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_min`.

18.1.10.75 atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean atomic_max(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.76 async_atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_atomic_max(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `atomic_max`.

18.1.10.77 cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_atomic_max(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.78 async_cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_atomic_max(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_atomic_max`.

18.1.10.79 group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_atomic_max(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.80 async_group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_atomic_max(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_atomic_max`.

18.1.10.81 string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean string_prepend(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.82 async_string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_string_prepend(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `string_prepend`.

18.1.10.83 cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_string_prepend(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.84 async_cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_string_prepend(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_string_prepend`.

18.1.10.85 group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_string_prepend(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.86 async_group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_string_prepend(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_string_prepend`.

18.1.10.87 string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean string_append(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.88 async_string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_string_append(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `string_append`.

18.1.10.89 cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_string_append(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.90 async_cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_string_append(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_string_append`.

18.1.10.91 group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_string_append(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.92 async_group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_string_append(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_string_append`.

18.1.10.93 string_ltrim

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean string_ltrim(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.94 async_string_ltrim

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_string_ltrim(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `string_ltrim`.

18.1.10.95 cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_string_ltrim(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.96 async_cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_string_ltrim(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_string_ltrim`.

18.1.10.97 group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_string_ltrim(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.98 async_group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_string_ltrim(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_string_ltrim`.

18.1.10.99 string_rtrim

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean string_rtrim(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.100 async_string_rtrim

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_string_rtrim(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `string_rtrim`.

18.1.10.101 cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_string_rtrim(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.102 async_cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_string_rtrim(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_string_rtrim`.

18.1.10.103 group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_string_rtrim(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.104 async_group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_string_rtrim(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_string_rtrim`.

18.1.10.105 list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean list_lpush(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.106 async_list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_list_lpush(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `list_lpush`.

18.1.10.107 cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_list_lpush(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.108 async_cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_list_lpush(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_list_lpush`.

18.1.10.109 group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_list_lpush(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.110 async_group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_list_lpush(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_list_lpush`.

18.1.10.111 list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean list_rpush(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.112 async_list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_list_rpush(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `list_rpush`.

18.1.10.113 cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_list_rpush(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.114 async_cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_list_rpush(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_list_rpush`.

18.1.10.115 group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_list_rpush(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.116 async_group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_list_rpush(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_list_rpush`.

18.1.10.117 set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean set_add(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.118 async_set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_set_add(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `set_add`.

18.1.10.119 cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_set_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.120 async_cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_set_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_set_add`.

18.1.10.121 group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_set_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.122 async_group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_set_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_set_add`.

18.1.10.123 set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean set_remove(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.124 async_set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_set_remove(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `set_remove`.

18.1.10.125 cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_set_remove(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.126 async_cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_set_remove(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_set_remove`.

18.1.10.127 group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_set_remove(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.128 async_group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_set_remove(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_set_remove`.

18.1.10.129 set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
public Boolean set_intersect(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.130 async_set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_set_intersect(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `set_intersect`.

18.1.10.131 cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_set_intersect(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.132 async_cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_set_intersect(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_set_intersect`.

18.1.10.133 group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_set_intersect(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.134 async_group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_set_intersect(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_set_intersect`.

18.1.10.135 set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean set_union(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.136 async_set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_set_union(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `set_union`.

18.1.10.137 cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_set_union(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.138 async_cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_set_union(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_set_union`.

18.1.10.139 group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_set_union(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.140 async_group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_set_union(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_set_union`.

18.1.10.141 document_rename

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean document_rename(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.142 async_document_rename

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_document_rename(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `document_rename`.

18.1.10.143 cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_document_rename(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.144 async_cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_document_rename(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_document_rename`.

18.1.10.145 group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_document_rename(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.146 async_group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_document_rename(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_document_rename`.

18.1.10.147 `document_unset`

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean document_unset(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.148 async_document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_document_unset(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `document_unset`.

18.1.10.149 cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_document_unset(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.150 async_cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_document_unset(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_document_unset`.

18.1.10.151 group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_document_unset(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.152 async_group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_document_unset(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_document_unset`.

18.1.10.153 map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_add(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.154 async_map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_add(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_add`.

18.1.10.155 cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.156 async_cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_add`.

18.1.10.157 group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.158 async_group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_add`.

18.1.10.159 map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_remove(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.160 async_map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_remove(
        String spacename,
        Object key,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_remove`.

18.1.10.161 cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_remove(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.162 async_cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_remove(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_remove`.

18.1.10.163 group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_remove(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.164 async_group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_remove(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Object> attributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Object> attributes`
  The set of attributes to modify and their respective values. `attrs` is a map from the attributes' names to their values.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_remove`.

18.1.10.165 map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_add(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.166 async_map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_add(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_add`.

18.1.10.167 cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.168 async_cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_add(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_add`.

18.1.10.169 group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.170 async_group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_add(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_add`.

18.1.10.171 map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
public Boolean map_atomic_sub(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.172 async_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_sub(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_sub`.

18.1.10.173 cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_sub(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.174 async_cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_sub(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_sub`.

18.1.10.175 group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_sub(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.176 async_group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_sub(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_sub`.

18.1.10.177 map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_mul(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.178 async_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_mul(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_mul`.

18.1.10.179 cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_mul(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.180 async_cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_mul(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_mul`.

18.1.10.181 group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_mul(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.182 async_group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_mul(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_mul`.

18.1.10.183 map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_div(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.184 async_map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_div(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_div`.

18.1.10.185 cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_div(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.186 async_cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_div(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_div`.

18.1.10.187 group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_div(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.188 async_group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_div(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_div`.

18.1.10.189 map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_mod(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.190 async_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_mod(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_mod`.

18.1.10.191 cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_mod(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.192 async_cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_mod(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_mod`.

18.1.10.193 group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_mod(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.194 async_group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_mod(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_mod`.

18.1.10.195 map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_and(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.196 async_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_and(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_and`.

18.1.10.197 cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_and(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.198 async_cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_and(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_and`.

18.1.10.199 group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_and(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.200 async_group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_and(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_and`.

18.1.10.201 map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_or(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.202 async_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_or(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_or`.

18.1.10.203 cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_or(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.204 async_cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_or(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_or`.

18.1.10.205 group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_or(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.206 async_group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_or(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_or`.

18.1.10.207 map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_xor(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.208 async_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_xor(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_xor`.

18.1.10.209 cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_xor(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.210 async_cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_xor(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_xor`.

18.1.10.211 group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_xor(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.212 async_group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_xor(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_xor`.

18.1.10.213 map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_string_prepend(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.214 async_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_string_prepend(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_string_prepend`.

18.1.10.215 cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_string_prepend(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.216 async_cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_string_prepend(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_string_prepend`.

18.1.10.217 group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_string_prepend(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.218 async_group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_string_prepend(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_string_prepend`.

18.1.10.219 map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
public Boolean map_string_append(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.220 async_map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
public Deferred async_map_string_append(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_string_append`.

18.1.10.221 cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_string_append(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.222 async_cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_string_append(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_string_append`.

18.1.10.223 group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_string_append(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.224 async_group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_string_append(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_string_append`.

18.1.10.225 map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_min(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.226 async_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_min(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_min`.

18.1.10.227 cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_min(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.228 async_cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_min(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_min`.

18.1.10.229 group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_min(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.230 async_group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_min(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_min`.

18.1.10.231 map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Boolean map_atomic_max(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.232 async_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
public Deferred async_map_atomic_max(
        String spacename,
        Object key,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `map_atomic_max`.

18.1.10.233 cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Boolean cond_map_atomic_max(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.234 async_cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
public Deferred async_cond_map_atomic_max(
        String spacename,
        Object key,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Object key`
  The key for the operation where `key` is a Java object.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns an object indicating the success or failure of the operation. Valid values to be returned are:

- `True` if the operation succeeded
- `False` if any provided predicates failed.
- `null` if the operation requires an existing value and none exists

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `cond_map_atomic_max`.

18.1.10.235 group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Long group_map_atomic_max(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.236 async_group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
public Deferred async_group_map_atomic_max(
        String spacename,
        Map<String, Object> predicates,
        Map<String, Map<Object, Object>> mapattributes) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `Map<String, Map<Object, Object>> mapattributes`
  The set of attributes to modify and their respective values. `mapattrs` is a map from the attributes' names to inner maps which contain the key-value pairs to be modified

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `group_map_atomic_max`.

18.1.10.237 `search`

Return all objects that match the specified `checks`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
public Iterator search(String spacename, Map<String, Object> predicates)
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This asynchronous operation returns an `Iterator` object. The `Iterator` will return the resulting objects as they become available.

Errors will be returned from the Iterator, as it is possible to retreive partial results in the face of an error.

18.1.10.238 search_describe

Return a human-readable string suitable for debugging search internals. This API is only really relevant for debugging the internals of `search`.

Definition:

```
public String search_describe(
        String spacename,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This function returns a string describing the internals of the search.
On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.239 async_search_describe

Return a human-readable string suitable for debugging search internals. This API is only really relevant for debugging the internals of `search`.

Definition:

```
public Deferred async_search_describe(
        String spacename,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a string describing the internals of the search.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `search_describe`.

18.1.10.240 sorted_search

Return all objects that match the specified `checks`, sorted according to `attr`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
public Iterator sorted_search(
        String spacename,
        Map<String, Object> predicates,
        String sortby,
        int limit,
        boolean maxmin)
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.
- `String sortby`
  The attribute to sort by.
- `int limit`
  The number of results to return.
- `boolean maxmin`
  Maximize or minimize. Valid values are ``max'' and ``min''.

Returns: This asynchronous operation returns an `Iterator` object. The `Iterator` will return the resulting objects as they become available.

Errors will be returned from the Iterator, as it is possible to retreive partial results in the face of an error.

18.1.10.241 count

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
public Long count(
        String spacename,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This function returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

18.1.10.242 async_count

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
public Deferred async_count(
        String spacename,
        Map<String, Object> predicates) throws HyperDexClientException
```

Parameters:

- `String spacename`
  The name of the space as a String.
- `Map<String, Object> predicates`
  A set of predicates to check against. `checks` is a map from the attributes' names to the predicates to check.

Returns: This asynchronous operation returns a `Deferred` object with a `waitForIt` method which blocks and returns a number indicating the number of objects counted.

On error, this function will raise a `HyperDexClientException` describing the error.

See also: This is the asynchronous form of `count`.

18.1.11 Working with Signals

The HyperDex client library is signal-safe. Should a signal interrupt the client during a blocking operation, it will raise a `HyperDexClientException` with status `HYPERDEX_CLIENT_INTERRUPTED`.

18.1.12 Working with Threads

The Java package is fully reentrant. Instances of `HyperDex::Client::Client` and their associated state may be accessed from multiple threads, provided that the application employs its own synchronization that provides mutual exclusion.

Put simply, a multi-threaded application should protect each `Client` instance with a mutex or lock to ensure correct operation.

Chapter 19

Node API

19.1 Client Library

HyperDex provides Node bindings under the module `hyperdex-client`. This library wraps the HyperDex C Client library and enables use of native Javascript data types.

This module was re-introduced in HyperDex 1.2.0.

19.1.1 Building the HyperDex Node.js Binding

The HyperDex Node.js Binding must be built and installed after HyperDex is built and installed. After installing HyperDex, you can build the Node.js bindings from either source or git checkout with:

```
% cd bindings/node.js
% node rebuild
```

19.1.2 Using Node.js Within Your Application

All client operation are defined in the `hyperdex_client` module. You can access this in your program with:

```
var hyperdex_client = require('hyperdex-client');
```

19.1.3 Hello World

The following is a minimal application that stores the value "Hello World" and then immediately retrieves the value:

```
var hyperdex_client = require('hyperdex-client')
var c = new hyperdex_client.Client('127.0.0.1', 1982);

function get_callback(obj, err) {
    if (err) { console.log('error ' + err); }
    console.log('get: ' + obj);
}

function put_callback(success, err) {
    if (err) { console.log('error ' + err); }
    console.log('put: ' + success);
    c.get('kv', 'k', get_callback);
}

c.put('kv', 'k', {v: 'Hello World!'}, put_callback);
```

You can run this example with:

```
% node hello-world.js
put: true
get: [object Object]
```

Right away, there are several points worth noting in this example:

- Each operation takes a callback. While the operation is outstanding, your program is free to execute other code.
- Javascript types are automatically converted to HyperDex types. There's no need to specify information such as the length of each string, as one would do with the C API.
- There's no need to manually enter the HyperDex event loop. HyperDex will add and remove itself from the event loop as operations start and finish.

19.1.4 Asynchronous Operations

HyperDex provides native integration with the asynchronous world of Node.js. You can issue several operations concurrently, and Node.js and HyperDex will work together to complete these operations quickly and efficiently. It's easy to work with data concurrently. A common pattern is to keep a constant number of operations outstanding concurrently:

```
var hyperdex_client = require('hyperdex-client')
var c = new hyperdex_client.Client('127.0.0.1', 1982);

var NUMBER = 10000000
var CONCURRENT = 1000
var i = 0

function put (success, err) {
    if (i < THRESH) {
        c.put('kv', new String(i), {v: new String(i), put);
    }
    i += 1;
}

for (var x = 0; x < CONCURRENT; ++x) {
    put();
}
```

19.1.5 Data Structures

The Node bindings automatically manage conversion of data types from Javascript to HyperDex types, enabling applications to be written in idiomatic Javascript.

19.1.5.1 Examples

This section shows examples of Java data structures that are recognized by HyperDex. The examples here are for illustration purposes and are not exhaustive.

Strings The HyperDex client recognizes Javascript strings and buffers and automatically converts them to HyperDex strings. For example, the following two calls have the same effect:

```
c.put("kv", "somekey", {v: "somevalue"}, function (success, err) {});
c.put("kv", "somekey", {v: new Buffer("somevalue")}, function (success, err) {});
```

Integers The HyperDex client recognizes Javascript numbers and can convert them to HyperDex integers. For example:

```
c.put("kv", "somekey", {v: c.asInt(42)}, function (success, err) {});
```

Floats The HyperDex client recognizes Javascript numbers and can convert them to HyperDex floats. For example:

```
c.put("kv", "somekey", {v: c.asFloat(3.1415)}, function (success, err) {});
```

Lists The HyperDex client permits users to construct HyperDex lists from Javascript. For example:

```
c.put("kv", "somekey", {v: c.asList(['a', 'b', 'c'])}, function (success, err) {});
c.put("kv", "somekey", {v: c.asList([1, 2, 3])}, function (success, err) {});
c.put("kv", "somekey", {v: c.asList([1.0, 0.5, 0.25])}, function (success, err) {});
```

Sets The HyperDex client permits users to construct HyperDex sets from Javascript. For example:

```
c.put("kv", "somekey", {v: c.asSet(['a', 'b', 'c'])}, function (success, err) {});
c.put("kv", "somekey", {v: c.asSet([1, 2, 3])}, function (success, err) {});
c.put("kv", "somekey", {v: c.asSet([1.0, 0.5, 0.25])}, function (success, err) {});
```

Maps The HyperDex client permits users to construct HyperDex maps from Javascript. For example:

```
c.put("kv", "somekey", {v1: c.asMap([["k", "v"]])}, function(success, err) {});
c.put("kv", "somekey", {v2: c.asMap([[1, 2]])}, function(success, err) {});
c.put("kv", "somekey", {v3: c.asMap([[3.14, 0.125]])}, function(success, err) {});
c.put("kv", "somekey", {v4: c.asMap([["a", 1]])}, function(success, err) {});
```

19.1.6 Attributes

Attributes in Node are specified in the form of a Javascript object. As you can see in the examples above, attributes are specified in the form:

```
{name: "value"}
```

19.1.7 Operations

19.1.7.1 `get`

Retrieve an object from `space` using `key`.

Definition:

```
get(spacename, key, function (obj, done, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- A Javascript representation of the stored object.
- `null` if the operation is a retrieval operation and no object was found.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.2 get_partial

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
get_partial(spacename, key, attributenames, function (obj, done, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributenames`
  A list of attributes to return.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- A Javascript representation of the stored object.
- `null` if the operation is a retrieval operation and no object was found.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.3 `put`

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
put(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.4 cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_put(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.5 cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_put_or_create(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.6 group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_put(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.7 put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
put_if_not_exist(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.8 `del`

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
del(spacename, key, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.9 cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_del(spacename, key, predicates, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.10 group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_del(spacename, predicates, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.11 atomic_add

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_add(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.12 cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_add(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.13 group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_add(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.14 atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_sub(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.15 cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_sub(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.16 group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_sub(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.17 atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_mul(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.18 cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_mul(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.19 group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_mul(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.20 atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_div(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.21 cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_div(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.22 group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_div(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.23 atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_mod(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.24 cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_mod(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.25 group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_mod(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.26 atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_and(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.27 cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_and(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.28 group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_and(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.29 atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_or(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.30 cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_or(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.31 group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_or(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.32 atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_xor(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.33 cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_xor(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.34 group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_xor(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.35 atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_min(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.36 cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_min(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.37 group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_min(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.38 atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_max(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.39 cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_max(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.40 group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_max(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.41 string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_prepend(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.42 cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_prepend(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.43 group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_prepend(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.44 string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_append(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.45 cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_append(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.46 group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_append(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.47 `string_ltrim`

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_ltrim(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.48 cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_ltrim(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.49 group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_ltrim(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.50 `string_rtrim`

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_rtrim(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.51 cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_rtrim(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.52 group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_rtrim(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.53 list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
list_lpush(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.54 cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_list_lpush(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.55 group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_list_lpush(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.56 list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
list_rpush(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.57 cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_list_rpush(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.58 group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_list_rpush(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.59 set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
set_add(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.60 cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_add(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.61 group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_add(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.62 set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
set_remove(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.63 cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_remove(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.64 group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_remove(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.65 set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
set_intersect(spacename, key, attributes, function (success, err) {})
```

Parameters:

- spacename
  The name of the space as a C-string.
- key
  The key for the operation where key is a Javascript value.
- attributes
  The attributes to modify and their respective values. attrs is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- true if the operation succeeded
- false if any provided predicates failed
- null if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.66 cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_intersect(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.67 group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_intersect(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.68 set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
set_union(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.69 cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_union(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.70 group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_union(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.71 `document_rename`

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
document_rename(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.72 cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_document_rename(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.73 group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_document_rename(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.74 document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
document_unset(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.75 cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_document_unset(
        spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.76 group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_document_unset(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.77 map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_add(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.78 cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_add(spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.79 group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_add(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.80 map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_remove(spacename, key, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.81 cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_remove(spacename, key, predicates, attributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.82 group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_remove(spacename, predicates, attributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `attributes`
  The attributes to modify and their respective values. `attrs` is an object with the individual attributes as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.83 map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_add(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.84 cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_add(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.85 group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_add(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.86 map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
map_atomic_sub(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- spacename
  The name of the space as a C-string.
- key
  The key for the operation where key is a Javascript value.
- mapattributes
  The map attributes to modify and their respective key/values. mapattrs is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- true if the operation succeeded
- false if any provided predicates failed
- null if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.87 cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_sub(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.88 group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_sub(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.89 map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_mul(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.90 cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_mul(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.91 group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_mul(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.92 map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_div(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.93 cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_div(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.94 group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_div(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.95 map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_mod(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.96 cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_mod(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.97 group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_mod(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.98 map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_and(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.99 cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_and(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.100 group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_and(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.101 map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_or(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.102 cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_or(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.103 group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_or(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.104 map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_xor(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.105 cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_xor(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.106 group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_xor(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.107 map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_string_prepend(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.108 cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_string_prepend(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.109 group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_string_prepend(
        spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.110 map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
map_string_append(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- spacename
  The name of the space as a C-string.
- key
  The key for the operation where key is a Javascript value.
- mapattributes
  The map attributes to modify and their respective key/values. mapattrs is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- true if the operation succeeded
- false if any provided predicates failed
- null if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.111 cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_string_append(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.112 group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_string_append(
        spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.113 map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_min(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.114 cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_min(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.115 group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_min(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.116 map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
map_atomic_max(spacename, key, mapattributes, function (success, err) {})
```

Parameters:

- spacename
  The name of the space as a C-string.
- key
  The key for the operation where key is a Javascript value.
- mapattributes
  The map attributes to modify and their respective key/values. mapattrs is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- true if the operation succeeded
- false if any provided predicates failed
- null if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.117 cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_max(
        spacename, key, predicates, mapattributes, function (success, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `key`
  The key for the operation where `key` is a Javascript value.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- `true` if the operation succeeded
- `false` if any provided predicates failed
- `null` if the operation requires an existing value and none exist

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.118 group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_max(spacename, predicates, mapattributes, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `mapattributes`
  The map attributes to modify and their respective key/values. `mapattrs` is an object with the attributes as properties. The key/value pairs are represented as arrays.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.119 `search`

Return all objects that match the specified `checks`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
search(spacename, predicates, function (obj, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- A Javascript representation of the stored object.
- `null` if the operation is a retrieval operation and no object was found.

The second argument is a true/false value indicating whether the operation is complete. The callback will be called repeatedly until all objects are returned or encounter errors.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.120 sorted_search

Return all objects that match the specified `checks`, sorted according to `attr`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
sorted_search(spacename, predicates, sortby, limit, maxmin, function (obj, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.
- `sortby`
  The attribute to sort by.
- `limit`
  The number of results to return.
- `maxmin`
  Maximize or minimize. Valid values are ‘‘`max`’’ and ‘‘`min`’’.

Returns: This function returns via the provided callback. In the normal case, the first argument will indicate success or failure of the operation with one of the following values:

- A Javascript representation of the stored object.
- `null` if the operation is a retrieval operation and no object was found.

The second argument is a true/false value indicating whether the operation is complete. The callback will be called repeatedly until all objects are returned or encounter errors.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.7.121 `count`

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
count(spacename, predicates, function (count, err) {})
```

Parameters:

- `spacename`
  The name of the space as a C-string.
- `predicates`
  The predicates to check against. `checks` is an object with the individual predicates as properties.

Returns: This function returns via the provided callback. In the normal case, the first argument will be the total number of objects that match the predicate.

If the operation encounters any error, the error argument will be provided and will specify the error, in which case the first argument is undefined.

19.1.8 Working with Signals

The HyperDex client module is signal-safe. Should a signal interrupt the client, it will raise an exception with status `HYPERDEX_CLIENT_INTERRUPTED`.

19.1.9 Working with Events

The Node module naturally integrates with the Node.js event loop. Each instance of `Client` registers itself with the Node event loop and makes callbacks as soon as events complete on the HyperDex side.

Put simply, a Node.js application can use `Client` instances in a straight-forward fashion without worrying about threading or manual integration.

Chapter 20

Python Client API

HyperDex provides Python bindings to the client in the module `hyperdex.client`. This library wraps the HyperDex C client library and enables the use of native Python data types.

20.0.1 Building the HyperDex Python Binding

The HyperDex Python bindings must be requested at configure time as it is not automatically built. You can ensure that the Python bindings are always built by providing the `--enable-python-bindings` option to `./configure` like so:

```
% ./configure --enable-client --enable-python-bindings
```

20.0.2 Using Python Within Your Application

All client operation are defined in the `hyperdex.client` module. You can access this in your program with:

```
import hyperdex.client
```

20.0.3 Hello World

The following is a minimal application that stores the value "Hello World" and then immediately retrieves the value:

```
import hyperdex.client

c = hyperdex.client.Client("127.0.0.1", 1982)
c.put("kv", "some key", {"v": "Hello World!"})
print 'put "Hello World!"'
print 'got:', c.get("kv", "some key")
```

You can run this example with:

```
% python hello-world.py
put "Hello World!"
got:
{'v': 'Hello World!'}
```

Right away, there are several points worth noting in this example:

- Every operation is synchronous. The put and get operations run to completion by default.
- Python types are automatically converted to HyperDex types. There's no need to specify information such as the length of each string, as one would do with the C API.

20.0.4 Asynchronous Operations

For convenience, the Python bindings treat every operation as synchronous. This enables you to write short scripts without concern for asynchronous operations. Most operations come with an asynchronous form, denoted by the `async_` prefix. For example, the above Hello World example could be rewritten in asynchronous fashion as such:

```
import hyperdex.client

c = hyperdex.client.Client("127.0.0.1", 1982)
p = c.async_put("kv", "some key", {"v": "Hello World!"})
p.wait()
print 'put "Hello World!"'
g = c.async_get("kv", "some key")
print 'got:', g.wait()
```

This enables applications to issue multiple requests simultaneously and wait for their completion in an application-specific order. It's also possible to use the `loop` method on the client object to wait for the next request to complete:

```
import hyperdex.client

c = hyperdex.client.Client("127.0.0.1", 1982)
p = c.async_put("kv", "some key", {"v": "Hello World!"})
p1 = c.loop()
print 'put objects are the same:', p is p1
p.wait()
print 'put "Hello World!"'
g = c.async_get("kv", "some key")
g1 = c.loop()
print 'get objects are the same:', g is g1
print 'got:', g.wait()
```

20.0.5 Data Structures

The Python bindings automatically manage conversion of data types from Python to HyperDex types, enabling applications to be written in idiomatic Python.

20.0.5.1 Examples

This section shows examples of Python data structures that are recognized by HyperDex. The examples here are for illustration purposes and are not exhaustive.

Strings The HyperDex client recognizes Python's strings and automatically converts them to HyperDex strings. For example:

```
c.put("kv", "somekey", {"v": "somevalue"})
```

Integers The HyperDex client recognizes Python's integers and longs and automatically converts them to HyperDex integers. For example:

```
c.put("k"', "somekey", {"v": 42})
```

Floats The HyperDex client recognizes Python's floating point numbers and automatically converts them to HyperDex floats. For example:

```
c.put("kv", "somekey", {"v": 3.1415})
```

Lists The HyperDex client recognizes Python lists and automatically converts them to HyperDex lists. For example:

```
c.put("kv", "somekey", {"v1": ["a", "b", "c"]})
c.put("kv", "somekey", {"v2": [1, 2, 3]})
c.put("kv", "somekey", {"v3": [1.0, 0.5, 0.25]})
```

Sets The HyperDex client recognizes Python sets and automaticaly converts them to HyperDex sets. For example:

```
require 'set'
c.put("kv", "somekey", {"v1": set(["a", "b", "c"])})
c.put("kv", "somekey", {"v2": set([1, 2, 3])})
c.put("kv", "somekey", {"v3": set([1.0, 0.5, 0.25])})
```

Maps The HyperDex client recognizes Python dictionaries and automatically converts them to HyperDex maps. For example:

```
c.put("kv", "somekey", {"v1": {"k": "v"}})
c.put("kv", "somekey", {"v2": {1: 2}})
c.put("kv", "somekey", {"v3": {3.14: 0.125}})
c.put("kv", "somekey", {"v3": {"a": 1}})
```

20.0.6 Attributes

Attributes in Python are specified in the form of a dictionary from attribute names to their values. As you can see in the examples above, attributes are specified in the form:

```
{"name": "value"}
```

20.0.7 Map Attributes

Map attributes in Python are specified in the form of a nested dictionary. The outer diction key specifies the name, while the inner diction key-value pair specifies the key-value pair of the map. For example:

```
{"name": {"key": "value"}}
```

20.0.8 Predicates

Predicates in Python are specified in the form of a diction from attribute names to their predicates. In the simple case, the predicate is just a value to be compared against:

```
{"v": "value"}
```

This is the same as saying:

```
{"v": hyperdex.client.Equals('value')}
```

The Python bindings support the full range of predicates supported by HyperDex itself. For example:

```
{"v": hyperdex.client.LessEqual(5)}
{"v": hyperdex.client.GreaterEqual(5)}
{"v": hyperdex.client.Range(5, 10)}
{"v": hyperdex.client.Regex('^s.*')}
{"v": hyperdex.client.LengthEquals(5)}
{"v": hyperdex.client.LengthLessEqual(5)}
{"v": hyperdex.client.LengthGreaterEqual(5)}
{"v": hyperdex.client.Contains('value')}
```

20.0.9 Error Handling

All error handling within the Python bindings is done via the exception handling mechanism of Python. Errors will be raised by the library and should be handled by your application. For example, if we were trying to store an integer (5) as attribute `"v"`, where `"v"` is actually a string, we'd generate an error.

```
try:
    c.put("kv", "my_key", {"v": 5})
except HyperDexClientException as e:
    print e.status
    print e.symbol
    print e
```

Errors of type `HyperDexClientException` will contain both a message indicating what went wrong, as well as the underlying `enum hyperdex_client_returncode`. The member `status` indicates the numeric value of this enum, while `symbol` returns the enum as a string. The above code will fail with the following output:

```
8525
HYPERDEX_CLIENT_WRONGTYPE
invalid attribute "v": attribute has the wrong type
```

20.0.10 Operations

20.0.10.1 `get`

Retrieve an object from `space` using `key`.

Definition:

```
def get(self, spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.

Returns: Object if found, `None` if not found. Raises exception on error.

20.0.10.2 async_get

Retrieve an object from `space` using `key`.

Definition:

```
def async_get(self, spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.

Returns: A `Deferred` object with a `wait` method that returns the object if found, `None` if not found. Raises exception on error.

See also: This is the asynchronous form of `get`.

20.0.10.3 `get_partial`

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
def get_partial(self, spacename, key, attributenames)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributenames`
  A list of attributes to return.

Returns: Object if found, `None` if not found. Raises exception on error.

20.0.10.4 async_get_partial

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
def async_get_partial(self, spacename, key, attributenames)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributenames`
  A list of attributes to return.

Returns: A `Deferred` object with a `wait` method that returns the object if found, `None` if not found. Raises exception on error.

See also: This is the asynchronous form of `get_partial`.

20.0.10.5 `put`

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
def put(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.6 async_put

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
def async_put(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `put`.

20.0.10.7 cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_put(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.8 async_cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_put(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_put`.

20.0.10.9 cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_put_or_create(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.10 async_cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_put_or_create(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_put_or_create`.

20.0.10.11 group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_put(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.12 async_group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_put(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_put`.

20.0.10.13 put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
def put_if_not_exist(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.14 async_put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
def async_put_if_not_exist(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `put_if_not_exist`.

20.0.10.15 `del`

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def delete(self, spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.16 async_del

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_delete(self, spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `del`.

20.0.10.17 cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_del(self, spacename, key, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.18 async_cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_del(self, spacename, key, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_del`.

20.0.10.19 group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_del(self, spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.

Returns: Number of objects found. Raises exception on error.

20.0.10.20 async_group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_del(self, spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_del`.

20.0.10.21 atomic_add

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def atomic_add(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.22 async_atomic_add

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_atomic_add(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_add`.

20.0.10.23 cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_add(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.24 async_cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_add(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_add`.

20.0.10.25 group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_add(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.26 async_group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_add(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_add`.

20.0.10.27 atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def atomic_sub(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.28 async_atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_atomic_sub(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_sub.

20.0.10.29 cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_sub(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.30 async_cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_sub(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_sub`.

20.0.10.31 group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_sub(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.32 async_group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_sub(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_sub`.

20.0.10.33 atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def atomic_mul(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.34 async_atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_atomic_mul(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_mul`.

20.0.10.35 cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_mul(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.36 async_cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_mul(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_mul`.

20.0.10.37 group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_mul(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.38 async_group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_mul(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_mul`.

20.0.10.39 atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def atomic_div(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.40 async_atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_atomic_div(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_div`.

20.0.10.41 cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_div(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.42 async_cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_div(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_div`.

20.0.10.43 group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_div(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.44 async_group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_div(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_div`.

20.0.10.45 atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def atomic_mod(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.46 async_atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_atomic_mod(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_mod`.

20.0.10.47 cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_mod(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.48 async_cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_mod(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_mod`.

20.0.10.49 group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_mod(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.50 async_group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_mod(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_mod`.

20.0.10.51 atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def atomic_and(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.52 async_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_atomic_and(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_and.

20.0.10.53 cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_and(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.54 async_cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_and(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_and`.

20.0.10.55 group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_and(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.56 async_group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_and(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_and`.

20.0.10.57 atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def atomic_or(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.58 async_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_atomic_or(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_or.

20.0.10.59 cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_or(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.60 async_cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_or(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_or`.

20.0.10.61 group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_or(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.62 async_group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_or(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_or`.

20.0.10.63 atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def atomic_xor(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.64 async_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_atomic_xor(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_xor`.

20.0.10.65 cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_xor(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.66 async_cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_xor(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_xor`.

20.0.10.67 group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_xor(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.68 async_group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_xor(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_xor`.

20.0.10.69 atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def atomic_min(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.70 async_atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_atomic_min(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_min.

20.0.10.71 cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_min(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.72 async_cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_min(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_min`.

20.0.10.73 group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_min(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.74 async_group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_min(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_min`.

20.0.10.75 atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def atomic_max(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.76 async_atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_atomic_max(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_max.

20.0.10.77 cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_atomic_max(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.78 async_cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_atomic_max(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_max`.

20.0.10.79 group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_atomic_max(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.80 async_group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_atomic_max(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_max`.

20.0.10.81 string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def string_prepend(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.82 async_string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_string_prepend(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_prepend`.

20.0.10.83 cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_string_prepend(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.84 async_cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_string_prepend(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_prepend`.

20.0.10.85 group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_string_prepend(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.86 async_group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_string_prepend(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_prepend`.

20.0.10.87 string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def string_append(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.88 async_string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_string_append(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_append`.

20.0.10.89 cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_string_append(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.90 async_cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_string_append(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_append`.

20.0.10.91 group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_string_append(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.92 async_group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_string_append(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_append`.

20.0.10.93 string_ltrim

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def string_ltrim(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.94 async_string_ltrim

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_string_ltrim(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_ltrim`.

20.0.10.95 cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_string_ltrim(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.96 async_cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_string_ltrim(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_ltrim`.

20.0.10.97 group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_string_ltrim(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.98 async_group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_string_ltrim(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_ltrim`.

20.0.10.99 string_rtrim

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def string_rtrim(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.100 async_string_rtrim

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_string_rtrim(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_rtrim`.

20.0.10.101 cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_string_rtrim(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.102 async_cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_string_rtrim(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_rtrim`.

20.0.10.103 group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_string_rtrim(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.104 async_group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_string_rtrim(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_rtrim`.

20.0.10.105 list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def list_lpush(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.106 async_list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_list_lpush(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `list_lpush`.

20.0.10.107 cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_list_lpush(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.108 async_cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_list_lpush(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_list_lpush`.

20.0.10.109 group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_list_lpush(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.110 async_group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_list_lpush(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_list_lpush`.

20.0.10.111 list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def list_rpush(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.112 async_list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_list_rpush(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `list_rpush`.

20.0.10.113 cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_list_rpush(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.114 async_cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_list_rpush(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_list_rpush`.

20.0.10.115 group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_list_rpush(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.116 async_group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_list_rpush(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_list_rpush`.

20.0.10.117 set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def set_add(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.118 async_set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_set_add(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `set_add`.

20.0.10.119 cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_set_add(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.120 async_cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_set_add(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_add`.

20.0.10.121 group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_set_add(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.122 async_group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_set_add(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_add`.

20.0.10.123 set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def set_remove(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.124 async_set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_set_remove(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `set_remove`.

20.0.10.125 cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_set_remove(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.126 async_cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_set_remove(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_remove`.

20.0.10.127 group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_set_remove(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.128 async_group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_set_remove(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_remove`.

20.0.10.129 set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def set_intersect(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.130 async_set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_set_intersect(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of set_intersect.

20.0.10.131 cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_set_intersect(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.132 async_cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_set_intersect(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_intersect`.

20.0.10.133 group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_set_intersect(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.134 async_group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_set_intersect(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_intersect`.

20.0.10.135 set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def set_union(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.136 async_set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_set_union(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of set_union.

20.0.10.137 cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_set_union(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.138 async_cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_set_union(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_union`.

20.0.10.139 group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_set_union(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.140 async_group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_set_union(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_union`.

20.0.10.141 document_rename

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def document_rename(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.142 async_document_rename

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_document_rename(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `document_rename`.

20.0.10.143 cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_document_rename(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.144 async_cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_document_rename(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_document_rename`.

20.0.10.145 group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_document_rename(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.146 async_group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_document_rename(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_document_rename`.

20.0.10.147 document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def document_unset(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.148 async_document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_document_unset(self, spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- attributes
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of document_unset.

20.0.10.149 cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_document_unset(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.150 async_cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_document_unset(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_document_unset`.

20.0.10.151 group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_document_unset(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.152 async_group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_document_unset(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_document_unset`.

20.0.10.153 map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def map_add(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.154 async_map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_add(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_add`.

20.0.10.155 cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_add(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.156 async_cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_add(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_add`.

20.0.10.157 group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_add(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.158 async_group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_add(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_add`.

20.0.10.159 map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def map_remove(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.160 async_map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_remove(self, spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_remove`.

20.0.10.161 cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_remove(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.162 async_cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_remove(self, spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_remove`.

20.0.10.163 group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_remove(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

20.0.10.164 async_group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_remove(self, spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `attributes`
  A dictionary of attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_remove`.

20.0.10.165 map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_add(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.166 async_map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_atomic_add(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_add`.

20.0.10.167 cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_add(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.168 async_cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_add(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_add`.

20.0.10.169 group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_add(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.170 async_group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_add(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_add`.

20.0.10.171 map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_sub(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.172 async_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_map_atomic_sub(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_atomic_sub.

20.0.10.173 cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_sub(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.174 async_cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_sub(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_sub`.

20.0.10.175 group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_sub(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.176 async_group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_sub(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_sub`.

20.0.10.177 map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_mul(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.178 async_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_map_atomic_mul(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_atomic_mul.

20.0.10.179 cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_mul(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.180 async_cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_mul(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_mul`.

20.0.10.181 group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_mul(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.182 async_group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_mul(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_mul`.

20.0.10.183 map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_div(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.184 async_map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_atomic_div(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_div`.

20.0.10.185 cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_div(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.186 async_cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_div(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_div`.

20.0.10.187 group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_div(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.188 async_group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_div(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_div`.

20.0.10.189 map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_mod(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.190 async_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_map_atomic_mod(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_atomic_mod.

20.0.10.191 cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_mod(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.192 async_cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_mod(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_mod`.

20.0.10.193 group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_mod(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.194 async_group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_mod(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_mod`.

20.0.10.195 map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def map_atomic_and(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.196 async_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_atomic_and(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_and`.

20.0.10.197 cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_and(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.198 async_cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_and(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_and`.

20.0.10.199 group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_and(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.200 async_group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_and(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_and`.

20.0.10.201 map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def map_atomic_or(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.202 async_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_atomic_or(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_or`.

20.0.10.203 cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_or(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.204 async_cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_or(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_or`.

20.0.10.205 group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_or(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.206 async_group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_or(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_or`.

20.0.10.207 map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_xor(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.208 async_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_atomic_xor(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_xor`.

20.0.10.209 cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_xor(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.210 async_cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_xor(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_xor`.

20.0.10.211 group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_xor(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.212 async_group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_xor(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_xor`.

20.0.10.213 map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_string_prepend(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.214 async_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_map_string_prepend(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_string_prepend.

20.0.10.215 cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_string_prepend(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.216 async_cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_string_prepend(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_string_prepend`.

20.0.10.217 group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_string_prepend(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.218 async_group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_string_prepend(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_string_prepend`.

20.0.10.219 map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_string_append(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.220 async_map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_map_string_append(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_string_append.

20.0.10.221 cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_string_append(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.222 async_cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_string_append(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_string_append`.

20.0.10.223 group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_string_append(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.224 async_group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_string_append(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_string_append`.

20.0.10.225 map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_min(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.226 async_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
def async_map_atomic_min(self, spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_min`.

20.0.10.227 cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_min(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.228 async_cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_min(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_min`.

20.0.10.229 group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_min(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.230 async_group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_min(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_min`.

20.0.10.231 map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def map_atomic_max(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.232 async_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
def async_map_atomic_max(self, spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string.
- key
  The key for the operation as a Python type.
- mapattributes
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a wait method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_atomic_max.

20.0.10.233 cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def cond_map_atomic_max(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: True if the operation succeeded, False if any of the provided predicates failed. Raises an exception on error.

20.0.10.234 async_cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
def async_cond_map_atomic_max(self, spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `key`
  The key for the operation as a Python type.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded, or False if any of the provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_max`.

20.0.10.235 group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def group_map_atomic_max(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: Number of objects found. Raises exception on error.

20.0.10.236 async_group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
def async_group_map_atomic_max(self, spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `mapattributes`
  A dictionary of map attributes to modify and their respective key-value pairs.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_max`.

20.0.10.237 `search`

Return all objects that match the specified `checks`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
def search(self, spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.

Returns: An iterator that returns objects one at a time. Exceptions will be returned via the same method to enable applications to retrieve partial results in the face of errors.

20.0.10.238 sorted_search

Return all objects that match the specified `checks`, sorted according to `attr`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
def sorted_search(self, spacename, predicates, sortby, limit, maxmin)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.
- `sortby`
  The attribute to sort by.
- `limit`
  The number of results to return.
- `maxmin`
  Maximize (`'max'`) or minimize (`'min'`).

Returns: An iterator that returns objects one at a time. Exceptions will be returned via the same method to enable applications to retrieve partial results in the face of errors.

20.0.10.239 count

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
def count(self, spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.

Returns: Number of objects found. Raises exception on error.

20.0.10.240 async_count

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
def async_count(self, spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string.
- `predicates`
  A dictionary of predicates to check against.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `count`.

20.0.11 Working with Signals

The HyperDex client library is signal-safe. Should a signal interrupt the client during a blocking operation, it will raise a `HyperDexClientException` with status `HYPERDEX_CLIENT_INTERRUPTED`.

20.0.12 Working with Threads

The Python module is fully reentrant. Instances of `hyperdex.client.Client` and their associated state may be accessed from multiple threads, provided that the application employs its own synchronization that provides mutual exclusion.

Put simply, a multi-threaded application should protect each `Client` instance with a mutex or lock to ensure correct operation.

Chapter 21

Ruby API

21.1 Client Library

HyperDex provides ruby bindings under the module `HyperDex::Client`. This library wraps the HyperDex C Client library and enables use of native Ruby data types.

This library was brought up-to-date following the 1.0.rc5 release.

21.1.1 Building the HyperDex Ruby Binding

The HyperDex Ruby Binding must be requested at configure time as it is not automatically built. You can ensure that the Ruby bindings are always built by providing the `--enable-ruby-bindings` option to `./configure` like so:

```
% ./configure --enable-client --enable-ruby-bindings
```

21.1.2 Using Ruby Within Your Application

All client operation are defined in the `HyperDex::Ruby` module. You can access this in your program with:

```
require 'hyperdex'
```

21.1.3 Hello World

The following is a minimal application that stores the value "Hello World" and then immediately retrieves the value:

```
require "hyperdex"
c = HyperDex::Client::Client.new("127.0.0.1", 1982)
c.put(:kv, "some key", {:v => "Hello World!"})
puts 'put "Hello World!"'
puts 'got:', c.get(:kv, "some key")
```

You can run this example with:

```
% ruby hello-world.rb
put "Hello World!"
got:
{:v=>"Hello World!"}
```

Right away, there are several points worth noting in this example:

- Every operation is synchronous. The PUT and GET operations run to completion by default.
- Ruby types are automatically converted to HyperDex types. There's no need to specify information such as the length of each string, as one would do with the C API.
- Ruby symbols are permitted wherever a string may be used. By convention, space names and attribute names are specified using Ruby symbols, e.g. `:kv` and `:v`. Of course, you may use strings for these parameters too.

21.1.4 Asynchronous Operations

For convenience, the Ruby bindings treat every operation as synchronous. This enables you to write short scripts without concern for asynchronous operations. Most operations come with an asynchronous form, denoted by the `async_` prefix. For example, the above Hello World example could be rewritten in asynchronous fashion as such:

```
require "hyperdex"
c = HyperDex::Client::Client.new("127.0.0.1", 1982)
p = c.async_put(:kv, "some key", {:v => "Hello World!"})
p.wait()
puts 'put "Hello World!"'
g = c.async_get(:kv, "some key")
puts 'got:', g.wait()
```

This enables applications to issue multiple requests simultaneously and wait for their completion in an application-specific order. It's also possible to use the `loop` method on the client object to wait for the next request to complete:

```
require "hyperdex"
c = HyperDex::Client::Client.new("127.0.0.1", 1982)
p = c.async_put(:kv, "some key", {:v => "Hello World!"})
p1 = c.loop()
puts 'put objects are the same:', (p.equal? p1)
p.wait()
puts 'put "Hello World!"'
g = c.async_get(:kv, "some key")
g1 = c.loop()
puts 'get objects are the same:', (g.equal? g1)
puts 'got:', g.wait()
```

21.1.5 Data Structures

The Ruby bindings automatically manage conversion of data types from Ruby to HyperDex types, enabling applications to be written in idiomatic Ruby.

21.1.5.1 Examples

This section shows examples of Ruby data structures that are recognized by HyperDex. The examples here are for illustration purposes and are not exhaustive.

Strings The HyperDex client recognizes Ruby's strings and symbols and automatically converts them to HyperDex strings. For example, the following two calls are equivalent and have the same effect:

```
c.put("kv", "somekey", {"v" => "somevalue"})
c.put(:kv, :somekey, {:v => :somevalue})
```

The recommended convention is to use symbols for space and attribute names, and strings for keys and values like so:

```
c.put(:kv, "somekey", {:v => "somevalue"})
```

Integers The HyperDex client recognizes Ruby's integers, longs, and fixnums, and automatically converts them to HyperDex integers. For example:

```
c.put(:kv, "somekey", {:v => 42})
```

Floats The HyperDex client recognizes Ruby's floating point numbers and automatically converts them to HyperDex floats. For example:

```
c.put(:kv, "somekey", {:v => 3.1415})
```

Lists The HyperDex client recognizes Ruby lists and automatically converts them to HyperDex lists. For example:

```
c.put(:kv, "somekey", {:v1 => ["a", "b", "c"]})
c.put(:kv, "somekey", {:v2 => [1, 2, 3]})
c.put(:kv, "somekey", {:v3 => [1.0, 0.5, 0.25]})
```

Sets The HyperDex client recognizes Ruby sets and automaticaly converts them to HyperDex sets. For example:

```
require 'set'
c.put(:kv, "somekey", {:v1 => (Set.new ["a", "b", "c"])})
c.put(:kv, "somekey", {:v2 => (Set.new [1, 2, 3])})
c.put(:kv, "somekey", {:v3 => (Set.new [1.0, 0.5, 0.25])})
```

Note that you'll have to include the set module from the standard library.

Maps The HyperDex client recognizes Ruby hashes and automatically converts them to HyperDex maps. For example:

```
c.put(:kv, "somekey", {:v1 => {"k" => "v"}})
c.put(:kv, "somekey", {:v2 => {1 => 2}})
c.put(:kv, "somekey", {:v3 => {3.14 => 0.125}})
c.put(:kv, "somekey", {:v3 => {"a" => 1}})
```

21.1.6 Attributes

Attributes in Ruby are specified in the form of a hash from attribute names to their values. As you can see in the examples above, attributes are specified in the form:

```
{:name => "value"}
```

21.1.7 Map Attributes

Map attributes in Ruby are specified in the form of a nested hash. The outer hash key specifies the name, while the inner hash key-value pair specifies the key-value pair of the map. For example:

```
{:name => {"key" => "value"}}
```

21.1.8 Predicates

Predicates in Ruby are specified in the form of a hash from attribute names to their predicates. In the simple case, the predicate is just a value to be compared against:

```
{:v => "value"}
```

This is the same as saying:

```
{:v => HyperDex::Client::Equals.new('value')}
```

The Ruby bindings support the full range of predicates supported by HyperDex itself. For example:

```
{:v => HyperDex::Client::LessEqual.new(5)}
{:v => HyperDex::Client::GreaterEqual.new(5)}
{:v => HyperDex::Client::Range.new(5, 10)}
{:v => HyperDex::Client::Regex.new('^s.*')}
{:v => HyperDex::Client::LengthEquals.new(5)}
{:v => HyperDex::Client::LengthLessEqual.new(5)}
{:v => HyperDex::Client::LengthGreaterEqual.new(5)}
{:v => HyperDex::Client::Contains.new('value')}
```

21.1.9 Error Handling

All error handling within the Ruby bindings is done via the `begin/rescue` mechanism of Ruby. Errors will be raised by the library and should be handled by your application. For example, if we were trying to store an integer (5) as attribute `:v`, where `:v` is actually a string, we'd generate an error.

```
begin
    puts c.put(:kv, :my_key, {:v => 5})
rescue HyperDex::Client::HyperDexClientException => e
    puts e.status
    puts e.symbol
    puts e
end
```

Errors of type `HyperDexClientException` will contain both a message indicating what went wrong, as well as the underlying `enum hyperdex_client_returncode`. The member `status` indicates the numeric value of this enum, while `symbol` returns the enum as a string. The above code will fail with the following output:

```
8525
HYPERDEX_CLIENT_WRONGTYPE
invalid attribute "v": attribute has the wrong type
```

21.1.10 Operations

21.1.10.1 `get`

Retrieve an object from `space` using `key`.

Definition:

```
get(spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.

Returns: Object if found, nil if not found. Raises exception on error.

21.1.10.2 `async_get`

Retrieve an object from `space` using `key`.

Definition:

```
async_get(spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.

Returns: A Deferred object with a `wait` method that returns the object if found, nil if not found. Raises exception on error.

See also: This is the asynchronous form of `get`.

21.1.10.3 `get_partial`

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
get_partial(spacename, key, attributenames)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributenames`
  A list of attributes to return.

Returns: Object if found, nil if not found. Raises exception on error.

21.1.10.4 async_get_partial

Retrieve part of an object from `space` using `key`. Only the specified attributes of the object will be returned to the client.

Definition:

```
async_get_partial(spacename, key, attributenames)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributenames`
  A list of attributes to return.

Returns: A Deferred object with a `wait` method that returns the object if found, nil if not found. Raises exception on error.

See also: This is the asynchronous form of `get_partial`.

21.1.10.5 `put`

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
put(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.6 async_put

Store or update an object by key. The object's attributes will be set to the values specified by `attrs`. If the object exists, it will be updated and all existing values not altered by `attrs` will be preserved. If the object does not exist, a new object will be created, with its attributes initialized to their default values.

Definition:

```
async_put(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `put`.

21.1.10.7 cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_put(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.8 async_cond_put

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_put(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_put`.

21.1.10.9 cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_put_or_create(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.10 async_cond_put_or_create

Conditionally update an the object stored under `key` in `space`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged. If the object exists, this is equivalent to a `cond_put`. If the object does not exist, this is equivalent to a `put`

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_put_or_create(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_put_or_create`.

21.1.10.11 group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_put(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.12 async_group_put

Update all objects stored in `space` that match `checks`. Existing values will be overwritten with the values specified by `attrs`. Values not specified by `attrs` will remain unchanged.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_put(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_put`.

21.1.10.13 put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
put_if_not_exist(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.14 async_put_if_not_exist

Store or object under `key` in `space` if and only if the operation creates a new object. The object's attributes will be set to the values specified by `attrs`; any attributes not specified by `attrs` will be initialized to their defaults. If the object exists, the operation will fail with `CMPFAIL`.

Definition:

```
async_put_if_not_exist(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `put_if_not_exist`.

21.1.10.15 `del`

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
del(spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.16 async_del

Delete `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_del(spacename, key)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `del`.

21.1.10.17 cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_del(spacename, key, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.18 async_cond_del

Conditionally delete the object stored under `key` from `space`. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_del(spacename, key, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_del`.

21.1.10.19 group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_del(spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.

Returns: Number of objects found. Raises exception on error.

21.1.10.20 async_group_del

Delete all objects that match the specified `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_del(spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_del`.

21.1.10.21 `atomic_add`

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_add(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.22 `async_atomic_add`

Add the specified number to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_add(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_add`.

21.1.10.23 cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_add(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.24 async_cond_atomic_add

Add the specified number to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_add(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_add`.

21.1.10.25 group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_add(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.26 async_group_atomic_add

Add the specified number to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_add(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_add`.

21.1.10.27 `atomic_sub`

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_sub(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.28 async_atomic_sub

Subtract the specified number from the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_sub(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_sub`.

21.1.10.29 cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_sub(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.30 async_cond_atomic_sub

Subtract the specified number from the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_sub(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_sub`.

21.1.10.31 group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_sub(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.32 async_group_atomic_sub

Subtract the specified number from the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_sub(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_sub`.

21.1.10.33 `atomic_mul`

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_mul(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.34 async_atomic_mul

Multiply the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_mul(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_mul`.

21.1.10.35 cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_mul(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.36 async_cond_atomic_mul

Multiply the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_mul(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_mul`.

21.1.10.37 `group_atomic_mul`

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_mul(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.38 async_group_atomic_mul

Multiply the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_mul(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_mul`.

21.1.10.39 `atomic_div`

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_div(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.40 async_atomic_div

Divide the existing value by the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_div(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_div`.

21.1.10.41 cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_div(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.42 async_cond_atomic_div

Divide the existing value by the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_div(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_div`.

21.1.10.43 group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_div(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.44 async_group_atomic_div

Divide the existing value by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_div(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_div`.

21.1.10.45 `atomic_mod`

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_mod(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.46 async_atomic_mod

Store the existing value modulo the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_mod(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_mod`.

21.1.10.47 cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_mod(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.48 async_cond_atomic_mod

Store the existing value modulo the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_mod(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_mod`.

21.1.10.49 group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_mod(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.50 async_group_atomic_mod

Store the existing value modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_mod(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_mod`.

21.1.10.51 `atomic_and`

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_and(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.52 async_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
async_atomic_and(spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- attributes
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_and.

21.1.10.53 cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_and(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.54 async_cond_atomic_and

Store the bitwise AND of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_and(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_and`.

21.1.10.55 group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_and(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.56 async_group_atomic_and

Store the bitwise AND of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_and(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_and`.

21.1.10.57 `atomic_or`

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_or(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.58 async_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_or(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_or`.

21.1.10.59 cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_or(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.60 async_cond_atomic_or

Store the bitwise OR of the existing value and the specified number for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_or(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_or`.

21.1.10.61 group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_or(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.62 async_group_atomic_or

Store the bitwise OR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_or(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_or`.

21.1.10.63 `atomic_xor`

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_xor(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.64 async_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
async_atomic_xor(spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- attributes
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of atomic_xor.

21.1.10.65 cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_xor(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.66 async_cond_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_xor(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_xor`.

21.1.10.67 group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_xor(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.68 async_group_atomic_xor

Store the bitwise XOR of the existing value and the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_xor(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_xor`.

21.1.10.69 `atomic_min`

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_min(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.70 async_atomic_min

Store the minimum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_min(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_min`.

21.1.10.71 cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_min(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.72 async_cond_atomic_min

Store the minimum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_min(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_min`.

21.1.10.73 group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_min(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.74 async_group_atomic_min

Store the minimum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_min(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_min`.

21.1.10.75 atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
atomic_max(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.76 async_atomic_max

Store the maximum of the existing value and the provided value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_atomic_max(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `atomic_max`.

21.1.10.77 cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_atomic_max(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.78 async_cond_atomic_max

Store the maximum of the existing value and the provided value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_atomic_max(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_atomic_max`.

21.1.10.79 group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_atomic_max(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.80 async_group_atomic_max

Store the maximum of the existing value and the provided value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_atomic_max(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_atomic_max`.

21.1.10.81 `string_prepend`

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_prepend(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.82 async_string_prepend

Prepend the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_string_prepend(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_prepend`.

21.1.10.83 cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_prepend(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.84 async_cond_string_prepend

Prepend the specified string to the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_string_prepend(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_prepend`.

21.1.10.85 group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_prepend(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.86 async_group_string_prepend

Prepend the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_string_prepend(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_prepend`.

21.1.10.87 `string_append`

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_append(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.88 async_string_append

Append the specified string to the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_string_append(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_append`.

21.1.10.89 cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_append(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.90 async_cond_string_append

Append the specified string to the existing value for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_string_append(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_append`.

21.1.10.91 group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_append(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.92 async_group_string_append

Append the specified string to the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_string_append(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_append`.

21.1.10.93 `string_ltrim`

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_ltrim(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.94 async_string_ltrim

Remove the specified number of bytes from the start of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_string_ltrim(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_ltrim`.

21.1.10.95 cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_ltrim(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.96 async_cond_string_ltrim

Remove the specified number of bytes from the start of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_string_ltrim(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_ltrim`.

21.1.10.97 group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_ltrim(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.98 async_group_string_ltrim

Remove the specified number of bytes from the start of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_string_ltrim(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_ltrim`.

21.1.10.99 `string_rtrim`

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
string_rtrim(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.100 async_string_rtrim

Remove the specified number of bytes from the end of the string. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_string_rtrim(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `string_rtrim`.

21.1.10.101 cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_string_rtrim(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.102 async_cond_string_rtrim

Remove the specified number of bytes from the end of the string if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_string_rtrim(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_string_rtrim`.

21.1.10.103 group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_string_rtrim(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.104 async_group_string_rtrim

Remove the specified number of bytes from the end of the string for for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_string_rtrim(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_string_rtrim`.

21.1.10.105 `list_lpush`

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
list_lpush(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.106 async_list_lpush

Push the specified value onto the front of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_list_lpush(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `list_lpush`.

21.1.10.107 `cond_list_lpush`

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_list_lpush(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.108 async_cond_list_lpush

Push the specified value onto the front of the list for each attribute if and only if `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_list_lpush(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_list_lpush`.

21.1.10.109 group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_list_lpush(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.110 async_group_list_lpush

Push the specified value onto the front of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_list_lpush(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_list_lpush`.

21.1.10.111 `list_rpush`

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
list_rpush(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.112 async_list_rpush

Push the specified value onto the back of the list for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_list_rpush(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `list_rpush`.

21.1.10.113 cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_list_rpush(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.114 async_cond_list_rpush

Push the specified value onto the back of the list for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_list_rpush(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_list_rpush`.

21.1.10.115 group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_list_rpush(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.116 async_group_list_rpush

Push the specified value onto the back of the list for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_list_rpush(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_list_rpush`.

21.1.10.117 set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
set_add(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.118 async_set_add

Add the specified value to the set for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_set_add(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `set_add`.

21.1.10.119 cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_add(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.120 async_cond_set_add

Add the specified value to the set for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_set_add(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_add`.

21.1.10.121 group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_add(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.122 async_group_set_add

Add the specified value to the set for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_set_add(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_add`.

21.1.10.123 set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
set_remove(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.124 async_set_remove

Remove the specified value from the set. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_set_remove(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `set_remove`.

21.1.10.125 cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_remove(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.126 async_cond_set_remove

Remove the specified value from the set if and only if the `checks` hold on the object. If the value is not contained within the set, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_set_remove(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_remove`.

21.1.10.127 group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_remove(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.128 async_group_set_remove

Remove the specified value from the set for each object in `space` that matches `checks`. If the value is not contained within the set, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_set_remove(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_remove`.

21.1.10.129 set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
set_intersect(spacename, key, attributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- attributes
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.130 async_set_intersect

Store the intersection of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_set_intersect(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `set_intersect`.

21.1.10.131 cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_intersect(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.132 async_cond_set_intersect

Store the intersection of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_set_intersect(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_intersect`.

21.1.10.133 group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_intersect(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.134 async_group_set_intersect

Store the intersection of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_set_intersect(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_intersect`.

21.1.10.135 `set_union`

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
set_union(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.136 async_set_union

Store the union of the specified set and the existing value for each attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_set_union(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `set_union`.

21.1.10.137 cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_set_union(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.138 async_cond_set_union

Store the union of the specified set and the existing value for each attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_set_union(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_set_union`.

21.1.10.139 group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_set_union(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.140 async_group_set_union

Store the union of the specified set and the existing value for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_set_union(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_set_union`.

21.1.10.141 `document_rename`

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
document_rename(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.142 async_document_rename

Move a field within a document from one name to another. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_document_rename(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `document_rename`.

21.1.10.143 cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_document_rename(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.144 async_cond_document_rename

Move a field within a document from one name to another if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_document_rename(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_document_rename`.

21.1.10.145 group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_document_rename(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.146 async_group_document_rename

Move a field within a document from one name to another for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_document_rename(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_document_rename`.

21.1.10.147 document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
document_unset(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.148 async_document_unset

Remove a field or object from a document. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_document_unset(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `document_unset`.

21.1.10.149 cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_document_unset(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.150 async_cond_document_unset

Remove a field or object from a document if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_document_unset(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_document_unset`.

21.1.10.151 group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_document_unset(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.152 async_group_document_unset

Remove a field or object from a document for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_document_unset(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_document_unset`.

21.1.10.153 map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_add(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.154 async_map_add

Insert a key-value pair into the map specified by each map-attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_add(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_add`.

21.1.10.155 cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_add(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.156 async_cond_map_add

Insert a key-value pair into the map specified by each map-attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_add(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_add`.

21.1.10.157 group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_add(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.158 async_group_map_add

Insert a key-value pair into the map specified by each map-attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_add(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_add`.

21.1.10.159 map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_remove(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.160 async_map_remove

Remove a key-value pair from the map specified by each attribute. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_remove(spacename, key, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_remove`.

21.1.10.161 cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_remove(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.162 async_cond_map_remove

Remove a key-value pair from the map specified by each attribute if and only if `checks` hold on the object. If there is no pair with the specified key within the map, this operation will do nothing. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_remove(spacename, key, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_remove`.

21.1.10.163 group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_remove(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: Number of objects found. Raises exception on error.

21.1.10.164 async_group_map_remove

Remove a key-value pair from the map specified by each attribute for each object in `space` that matches `checks`. If there is no pair with the specified key within the map, this operation will do nothing.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_remove(spacename, predicates, attributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `attributes`
  A hash specifying attributes to modify and their respective values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_remove`.

21.1.10.165 map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_add(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.166 async_map_atomic_add

Add the specified number to the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_add(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_add`.

21.1.10.167 cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_add(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.168 async_cond_map_atomic_add

Add the specified number to the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_add(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_add`.

21.1.10.169 group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_add(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.170 async_group_map_atomic_add

Add the specified number to the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_add(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_add`.

21.1.10.171 map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
map_atomic_sub(spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- mapattributes
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.172 async_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_sub(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_sub`.

21.1.10.173 cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_sub(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.174 async_cond_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_sub(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_sub`.

21.1.10.175 group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_sub(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.176 async_group_map_atomic_sub

Subtract the specified number from the value of a key-value pair within each map for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_sub(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_sub`.

21.1.10.177 map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
map_atomic_mul(spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- mapattributes
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.178 async_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_mul(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_mul`.

21.1.10.179 cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_mul(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.180 async_cond_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_mul(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_mul`.

21.1.10.181 group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_mul(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.182 async_group_map_atomic_mul

Multiply the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_mul(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_mul`.

21.1.10.183 map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_div(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.184 async_map_atomic_div

Divide the value of each key-value pair by the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_div(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_div`.

21.1.10.185 cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_div(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.186 async_cond_map_atomic_div

Divide the value of each key-value pair by the specified number for each map if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_div(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_div`.

21.1.10.187 group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_div(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.188 async_group_map_atomic_div

Divide the value of each key-value pair by the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_div(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_div`.

21.1.10.189 map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_mod(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.190 async_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_mod(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_mod`.

21.1.10.191 cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_mod(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.192 async_cond_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_mod(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_mod`.

21.1.10.193 group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_mod(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.194 async_group_map_atomic_mod

Store the value of the key-value pair modulo the specified number for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_mod(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_mod`.

21.1.10.195 map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_and(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.196 async_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_and(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_and`.

21.1.10.197 cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_and(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.198 async_cond_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_and(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_and`.

21.1.10.199 group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_and(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.200 async_group_map_atomic_and

Store the bitwise AND of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_and(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_and`.

21.1.10.201 map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_or(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.202 async_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_or(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_or`.

21.1.10.203 cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_or(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.204 async_cond_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_or(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_or`.

21.1.10.205 group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_or(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.206 async_group_map_atomic_or

Store the bitwise OR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_or(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_or`.

21.1.10.207 `map_atomic_xor`

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_xor(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.208 async_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_xor(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_xor`.

21.1.10.209 cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_xor(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.210 async_cond_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_xor(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_xor`.

21.1.10.211 group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_xor(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.212 async_group_map_atomic_xor

Store the bitwise XOR of the value of the key-value pair and the specified number for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_xor(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_xor`.

21.1.10.213 map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_string_prepend(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.214 async_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_string_prepend(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_string_prepend`.

21.1.10.215 cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_string_prepend(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.216 async_cond_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_string_prepend(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_string_prepend`.

21.1.10.217 group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_string_prepend(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.218 async_group_map_string_prepend

Prepend the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_string_prepend(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_string_prepend`.

21.1.10.219 map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_string_append(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.220 async_map_string_append

Append the specified string to the value of the key-value pair for each map attribute. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
async_map_string_append(spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- mapattributes
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_string_append.

21.1.10.221 cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_string_append(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.222 async_cond_map_string_append

Append the specified string to the value of the key-value pair for each map attribute if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_string_append(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_string_append`.

21.1.10.223 group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_string_append(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.224 async_group_map_string_append

Append the specified string to the value of the key-value pair for each map attribute for each object in `space` that matches `checks`

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_string_append(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_string_append`.

21.1.10.225 map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_min(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.226 async_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
async_map_atomic_min(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `map_atomic_min`.

21.1.10.227 cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_min(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.228 async_cond_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_min(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_min`.

21.1.10.229 group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_min(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.230 async_group_map_atomic_min

Take the minium of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_min(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_min`.

21.1.10.231 map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

Definition:

```
map_atomic_max(spacename, key, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.232 async_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with NOTFOUND.

Definition:

```
async_map_atomic_max(spacename, key, mapattributes)
```

Parameters:

- spacename
  The name of the space as a string or symbol.
- key
  The key for the operation as a Ruby type.
- mapattributes
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a wait method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of map_atomic_max.

21.1.10.233 cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
cond_map_atomic_max(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: True if the operation succeeded. False if any provided predicates failed. Raises an exception on error.

21.1.10.234 async_cond_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair if and only if the `checks` hold on the object. This operation requires a pre-existing object in order to complete successfully. If no object exists, the operation will fail with `NOTFOUND`.

This operation will succeed if and only if the predicates specified by `checks` hold on the pre-existing object. If any of the predicates are not true for the existing object, then the operation will have no effect and fail with `CMPFAIL`.

All checks are atomic with the write. HyperDex guarantees that no other operation will come between validating the checks, and writing the new version of the object.

Definition:

```
async_cond_map_atomic_max(spacename, key, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `key`
  The key for the operation as a Ruby type.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns True if the operation succeeded or False if any provided predicates failed. Raises an exception on error.

See also: This is the asynchronous form of `cond_map_atomic_max`.

21.1.10.235 group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
group_map_atomic_max(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: Number of objects found. Raises exception on error.

21.1.10.236 async_group_map_atomic_max

Take the maximum of the specified value and existing value for each key-value pair for each object in `space` that matches `checks`.

This operation will only affect objects that match the provided `checks`. Objects that do not match `checks` will be unaffected by the group call. Each object that matches `checks` will be atomically updated with the check on the object. HyperDex guarantees that no object will be altered if the `checks` do not pass at the time of the write. Objects that are updated concurrently with the group call may or may not be updated; however, regardless of any other concurrent operations, the preceding guarantee will always hold.

Definition:

```
async_group_map_atomic_max(spacename, predicates, mapattributes)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `mapattributes`
  A hash specifying map attributes to modify and their respective key/values.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `group_map_atomic_max`.

21.1.10.237 `search`

Return all objects that match the specified `checks`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
search(spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.

Returns: An iterator that returns objects one at a time. Exceptions will be returned via the same path to enable applications to retrieve partial results in the face of errors.

21.1.10.238 sorted_search

Return all objects that match the specified `checks`, sorted according to `attr`. This operation behaves as an iterator and may return multiple objects from the single call.

Definition:

```
sorted_search(spacename, predicates, sortby, limit, maxmin)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.
- `sortby`
  The attribute to sort by.
- `limit`
  The number of results to return.
- `maxmin`
  Maximize (`'max'`) or minimize (`'min'`).

Returns: An iterator that returns objects one at a time. Exceptions will be returned via the same path to enable applications to retrieve partial results in the face of errors.

21.1.10.239 `count`

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
count(spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.

Returns: Number of objects found. Raises exception on error.

21.1.10.240 async_count

Count the number of objects that match the specified `checks`. This will return the number of objects counted by the search. If an error occurs during the count, the count may reflect a partial count. The real count will be higher than the returned value.

A count is the server-side equivalent of performing a search, and counting the number of objects that exist. For that reason, its efficiency closely follows that of search, except it will not transfer any objects over the network.

Definition:

```
async_count(spacename, predicates)
```

Parameters:

- `spacename`
  The name of the space as a string or symbol.
- `predicates`
  A hash of predicates to check against.

Returns: A Deferred object with a `wait` method that returns the number of objects found. Raises exception on error.

See also: This is the asynchronous form of `count`.

21.1.11 Working with Signals

The HyperDex client library is signal-safe. Should a signal interrupt the client during a blocking operation, it will raise a `HyperDexClientException` with status `HYPERDEX_CLIENT_INTERRUPTED`.

21.1.12 Working with Threads

The Ruby module is fully reentrant. Instances of `HyperDex::Client::Client` and their associated state may be accessed from multiple threads, provided that the application employs its own synchronization that provides mutual exclusion.

Put simply, a multi-threaded application should protect each `Client` instance with a mutex or lock to ensure correct operation.

Index

add_index
  C API, 216
add_space
  C API, 210
async_atomic_add
  Java API, 376
  Python API, 748
  Ruby API, 993
async_atomic_and
  Java API, 406
  Python API, 778
  Ruby API, 1023
async_atomic_div
  Java API, 394
  Python API, 766
  Ruby API, 1011
async_atomic_max
  Java API, 430
  Python API, 802
  Ruby API, 1047
async_atomic_min
  Java API, 424
  Python API, 796
  Ruby API, 1041
async_atomic_mod
  Java API, 400
  Python API, 772
  Ruby API, 1017
async_atomic_mul
  Java API, 388
  Python API, 760
  Ruby API, 1005
async_atomic_or
  Java API, 412
  Python API, 784
  Ruby API, 1029
async_atomic_sub
  Java API, 382
  Python API, 754
  Ruby API, 999
async_atomic_xor
  Java API, 418
  Python API, 790
  Ruby API, 1035
async_cond_atomic_add
  Java API, 378
  Python API, 750
  Ruby API, 995
async_cond_atomic_and
  Java API, 408
  Python API, 780
  Ruby API, 1025
async_cond_atomic_div
  Java API, 396
  Python API, 768
  Ruby API, 1013
async_cond_atomic_max
  Java API, 432
  Python API, 804
  Ruby API, 1049
async_cond_atomic_min
  Java API, 426
  Python API, 798
  Ruby API, 1043
async_cond_atomic_mod
  Java API, 402
  Python API, 774
  Ruby API, 1019
async_cond_atomic_mul
  Java API, 390
  Python API, 762
  Ruby API, 1007
async_cond_atomic_or
  Java API, 414
  Python API, 786
  Ruby API, 1031
async_cond_atomic_sub
  Java API, 384
  Python API, 756
  Ruby API, 1001
async_cond_atomic_xor
  Java API, 420
  Python API, 792
  Ruby API, 1037
async_cond_del
  Java API, 372
  Python API, 744
  Ruby API, 989
async_cond_document_rename
  Java API, 498

Python API, 870
Ruby API, 1115
async_cond_document_unset
Java API, 504
Python API, 876
Ruby API, 1121
async_cond_list_lpush
Java API, 462
Python API, 834
Ruby API, 1079
async_cond_list_rpush
Java API, 468
Python API, 840
Ruby API, 1085
async_cond_map_add
Java API, 510
Python API, 882
Ruby API, 1127
async_cond_map_atomic_add
Java API, 522
Python API, 894
Ruby API, 1139
async_cond_map_atomic_and
Java API, 552
Python API, 924
Ruby API, 1169
async_cond_map_atomic_div
Java API, 540
Python API, 912
Ruby API, 1157
async_cond_map_atomic_max
Java API, 588
Python API, 960
Ruby API, 1205
async_cond_map_atomic_min
Java API, 582
Python API, 954
Ruby API, 1199
async_cond_map_atomic_mod
Java API, 546
Python API, 918
Ruby API, 1163
async_cond_map_atomic_mul
Java API, 534
Python API, 906
Ruby API, 1151
async_cond_map_atomic_or
Java API, 558
Python API, 930
Ruby API, 1175
async_cond_map_atomic_sub
Java API, 528
Python API, 900
Ruby API, 1145
async_cond_map_atomic_xor
Java API, 564
Python API, 936
Ruby API, 1181
async_cond_map_remove
Java API, 516
Python API, 888
Ruby API, 1133
async_cond_map_string_append
Java API, 576
Python API, 948
Ruby API, 1193
async_cond_map_string_prepend
Java API, 570
Python API, 942
Ruby API, 1187
async_cond_put
Java API, 362
Python API, 734
Ruby API, 979
async_cond_put_or_create
Java API, 364
Python API, 736
Ruby API, 981
async_cond_set_add
Java API, 474
Python API, 846
Ruby API, 1091
async_cond_set_intersect
Java API, 486
Python API, 858
Ruby API, 1103
async_cond_set_remove
Java API, 480
Python API, 852
Ruby API, 1097
async_cond_set_union
Java API, 492
Python API, 864
Ruby API, 1109
async_cond_string_append
Java API, 444
Python API, 816
Ruby API, 1061
async_cond_string_ltrim
Java API, 450
Python API, 822
Ruby API, 1067
async_cond_string_prepend
Java API, 438
Python API, 810
Ruby API, 1055
async_cond_string_rtrim
Java API, 456

Python API, 828
Ruby API, 1073
async_count
Java API, 596
Python API, 966
Ruby API, 1211
async_del
Java API, 370
Python API, 742
Ruby API, 987
async_document_rename
Java API, 496
Python API, 868
Ruby API, 1113
async_document_unset
Java API, 502
Python API, 874
Ruby API, 1119
async_get
Java API, 356
Python API, 728
Ruby API, 973
async_get_partial
Java API, 358
Python API, 730
Ruby API, 975
async_group_atomic_add
Java API, 380
Python API, 752
Ruby API, 997
async_group_atomic_and
Java API, 410
Python API, 782
Ruby API, 1027
async_group_atomic_div
Java API, 398
Python API, 770
Ruby API, 1015
async_group_atomic_max
Java API, 434
Python API, 806
Ruby API, 1051
async_group_atomic_min
Java API, 428
Python API, 800
Ruby API, 1045
async_group_atomic_mod
Java API, 404
Python API, 776
Ruby API, 1021
async_group_atomic_mul
Java API, 392
Python API, 764
Ruby API, 1009
async_group_atomic_or
Java API, 416
Python API, 788
Ruby API, 1033
async_group_atomic_sub
Java API, 386
Python API, 758
Ruby API, 1003
async_group_atomic_xor
Java API, 422
Python API, 794
Ruby API, 1039
async_group_del
Java API, 374
Python API, 746
Ruby API, 991
async_group_document_rename
Java API, 500
Python API, 872
Ruby API, 1117
async_group_document_unset
Java API, 506
Python API, 878
Ruby API, 1123
async_group_list_lpush
Java API, 464
Python API, 836
Ruby API, 1081
async_group_list_rpush
Java API, 470
Python API, 842
Ruby API, 1087
async_group_map_add
Java API, 512
Python API, 884
Ruby API, 1129
async_group_map_atomic_add
Java API, 524
Python API, 896
Ruby API, 1141
async_group_map_atomic_and
Java API, 554
Python API, 926
Ruby API, 1171
async_group_map_atomic_div
Java API, 542
Python API, 914
Ruby API, 1159
async_group_map_atomic_max
Java API, 590
Python API, 962
Ruby API, 1207
async_group_map_atomic_min
Java API, 584

Python API, 956
Ruby API, 1201
async_group_map_atomic_mod
Java API, 548
Python API, 920
Ruby API, 1165
async_group_map_atomic_mul
Java API, 536
Python API, 908
Ruby API, 1153
async_group_map_atomic_or
Java API, 560
Python API, 932
Ruby API, 1177
async_group_map_atomic_sub
Java API, 530
Python API, 902
Ruby API, 1147
async_group_map_atomic_xor
Java API, 566
Python API, 938
Ruby API, 1183
async_group_map_remove
Java API, 518
Python API, 890
Ruby API, 1135
async_group_map_string_append
Java API, 578
Python API, 950
Ruby API, 1195
async_group_map_string_prepend
Java API, 572
Python API, 944
Ruby API, 1189
async_group_put
Java API, 366
Python API, 738
Ruby API, 983
async_group_set_add
Java API, 476
Python API, 848
Ruby API, 1093
async_group_set_intersect
Java API, 488
Python API, 860
Ruby API, 1105
async_group_set_remove
Java API, 482
Python API, 854
Ruby API, 1099
async_group_set_union
Java API, 494
Python API, 866
Ruby API, 1111
async_group_string_append
Java API, 446
Python API, 818
Ruby API, 1063
async_group_string_ltrim
Java API, 452
Python API, 824
Ruby API, 1069
async_group_string_prepend
Java API, 440
Python API, 812
Ruby API, 1057
async_group_string_rtrim
Java API, 458
Python API, 830
Ruby API, 1075
async_list_lpush
Java API, 460
Python API, 832
Ruby API, 1077
async_list_rpush
Java API, 466
Python API, 838
Ruby API, 1083
async_map_add
Java API, 508
Python API, 880
Ruby API, 1125
async_map_atomic_add
Java API, 520
Python API, 892
Ruby API, 1137
async_map_atomic_and
Java API, 550
Python API, 922
Ruby API, 1167
async_map_atomic_div
Java API, 538
Python API, 910
Ruby API, 1155
async_map_atomic_max
Java API, 586
Python API, 958
Ruby API, 1203
async_map_atomic_min
Java API, 580
Python API, 952
Ruby API, 1197
async_map_atomic_mod
Java API, 544
Python API, 916
Ruby API, 1161
async_map_atomic_mul
Java API, 532

Python API, 904
Ruby API, 1149
async_map_atomic_or
Java API, 556
Python API, 928
Ruby API, 1173
async_map_atomic_sub
Java API, 526
Python API, 898
Ruby API, 1143
async_map_atomic_xor
Java API, 562
Python API, 934
Ruby API, 1179
async_map_remove
Java API, 514
Python API, 886
Ruby API, 1131
async_map_string_append
Java API, 574
Python API, 946
Ruby API, 1191
async_map_string_prepend
Java API, 568
Python API, 940
Ruby API, 1185
async_put
Java API, 360
Python API, 732
Ruby API, 977
async_put_if_not_exist
Java API, 368
Python API, 740
Ruby API, 985
async_search_describe
Java API, 593
async_set_add
Java API, 472
Python API, 844
Ruby API, 1089
async_set_intersect
Java API, 484
Python API, 856
Ruby API, 1101
async_set_remove
Java API, 478
Python API, 850
Ruby API, 1095
async_set_union
Java API, 490
Python API, 862
Ruby API, 1107
async_string_append
Java API, 442
Python API, 814
Ruby API, 1059
async_string_ltrim
Java API, 448
Python API, 820
Ruby API, 1065
async_string_prepend
Java API, 436
Python API, 808
Ruby API, 1053
async_string_rtrim
Java API, 454
Python API, 826
Ruby API, 1071
atomic_add
C API, 93
Java API, 375
Node.js API, 611
Python API, 747
Ruby API, 992
atomic_and
C API, 108
Java API, 405
Node.js API, 626
Python API, 777
Ruby API, 1022
atomic_div
C API, 102
Java API, 393
Node.js API, 620
Python API, 765
Ruby API, 1010
atomic_max
C API, 120
Java API, 429
Node.js API, 638
Python API, 801
Ruby API, 1046
atomic_min
C API, 117
Java API, 423
Node.js API, 635
Python API, 795
Ruby API, 1040
atomic_mod
C API, 105
Java API, 399
Node.js API, 623
Python API, 771
Ruby API, 1016
atomic_mul
C API, 99
Java API, 387
Node.js API, 617

Python API, 759
Ruby API, 1004
atomic_or
C API, 111
Java API, 411
Node.js API, 629
Python API, 783
Ruby API, 1028
atomic_sub
C API, 96
Java API, 381
Node.js API, 614
Python API, 753
Ruby API, 998
atomic_xor
C API, 114
Java API, 417
Node.js API, 632
Python API, 789
Ruby API, 1034
AtomicAdd
Go API, 239
AtomicAnd
Go API, 254
AtomicDiv
Go API, 248
AtomicMax
Go API, 266
AtomicMin
Go API, 263
AtomicMod
Go API, 251
AtomicMul
Go API, 245
AtomicOr
Go API, 257
AtomicSub
Go API, 242
AtomicXor
Go API, 260

backup
C API, 223

cond_atomic_add
C API, 94
Java API, 377
Node.js API, 612
Python API, 749
Ruby API, 994
cond_atomic_and
C API, 109
Java API, 407
Node.js API, 627
Python API, 779
Ruby API, 1024
cond_atomic_div
C API, 103
Java API, 395
Node.js API, 621
Python API, 767
Ruby API, 1012
cond_atomic_max
C API, 121
Java API, 431
Node.js API, 639
Python API, 803
Ruby API, 1048
cond_atomic_min
C API, 118
Java API, 425
Node.js API, 636
Python API, 797
Ruby API, 1042
cond_atomic_mod
C API, 106
Java API, 401
Node.js API, 624
Python API, 773
Ruby API, 1018
cond_atomic_mul
C API, 100
Java API, 389
Node.js API, 618
Python API, 761
Ruby API, 1006
cond_atomic_or
C API, 112
Java API, 413
Node.js API, 630
Python API, 785
Ruby API, 1030
cond_atomic_sub
C API, 97
Java API, 383
Node.js API, 615
Python API, 755
Ruby API, 1000
cond_atomic_xor
C API, 115
Java API, 419
Node.js API, 633
Python API, 791
Ruby API, 1036
cond_del
C API, 91
Java API, 371
Node.js API, 609

Python API, 743
Ruby API, 988
cond_document_rename
C API, 154
Java API, 497
Node.js API, 672
Python API, 869
Ruby API, 1114
cond_document_unset
C API, 157
Java API, 503
Node.js API, 675
Python API, 875
Ruby API, 1120
cond_list_lpush
C API, 136
Java API, 461
Node.js API, 654
Python API, 833
Ruby API, 1078
cond_list_rpush
C API, 139
Java API, 467
Node.js API, 657
Python API, 839
Ruby API, 1084
cond_map_add
C API, 160
Java API, 509
Node.js API, 678
Python API, 881
Ruby API, 1126
cond_map_atomic_add
C API, 166
Java API, 521
Node.js API, 684
Python API, 893
Ruby API, 1138
cond_map_atomic_and
C API, 181
Java API, 551
Node.js API, 699
Python API, 923
Ruby API, 1168
cond_map_atomic_div
C API, 175
Java API, 539
Node.js API, 693
Python API, 911
Ruby API, 1156
cond_map_atomic_max
C API, 199
Java API, 587
Node.js API, 717
Python API, 959
Ruby API, 1204
cond_map_atomic_min
C API, 196
Java API, 581
Node.js API, 714
Python API, 953
Ruby API, 1198
cond_map_atomic_mod
C API, 178
Java API, 545
Node.js API, 696
Python API, 917
Ruby API, 1162
cond_map_atomic_mul
C API, 172
Java API, 533
Node.js API, 690
Python API, 905
Ruby API, 1150
cond_map_atomic_or
C API, 184
Java API, 557
Node.js API, 702
Python API, 929
Ruby API, 1174
cond_map_atomic_sub
C API, 169
Java API, 527
Node.js API, 687
Python API, 899
Ruby API, 1144
cond_map_atomic_xor
C API, 187
Java API, 563
Node.js API, 705
Python API, 935
Ruby API, 1180
cond_map_remove
C API, 163
Java API, 515
Node.js API, 681
Python API, 887
Ruby API, 1132
cond_map_string_append
C API, 193
Java API, 575
Node.js API, 711
Python API, 947
Ruby API, 1192
cond_map_string_prepend
C API, 190
Java API, 569
Node.js API, 708

Python API, 941
Ruby API, 1186
cond_put
C API, 86
Java API, 361
Node.js API, 604
Python API, 733
Ruby API, 978
cond_put_or_create
C API, 87
Java API, 363
Node.js API, 605
Python API, 735
Ruby API, 980
cond_set_add
C API, 142
Java API, 473
Node.js API, 660
Python API, 845
Ruby API, 1090
cond_set_intersect
C API, 148
Java API, 485
Node.js API, 666
Python API, 857
Ruby API, 1102
cond_set_remove
C API, 145
Java API, 479
Node.js API, 663
Python API, 851
Ruby API, 1096
cond_set_union
C API, 151
Java API, 491
Node.js API, 669
Python API, 863
Ruby API, 1108
cond_string_append
C API, 127
Java API, 443
Node.js API, 645
Python API, 815
Ruby API, 1060
cond_string_ltrim
C API, 130
Java API, 449
Node.js API, 648
Python API, 821
Ruby API, 1066
cond_string_prepend
C API, 124
Java API, 437
Node.js API, 642
Python API, 809
Ruby API, 1054
cond_string_rtrim
C API, 133
Java API, 455
Node.js API, 651
Python API, 827
Ruby API, 1072
CondAtomicAdd
Go API, 240
CondAtomicAnd
Go API, 255
CondAtomicDiv
Go API, 249
CondAtomicMax
Go API, 267
CondAtomicMin
Go API, 264
CondAtomicMod
Go API, 252
CondAtomicMul
Go API, 246
CondAtomicOr
Go API, 258
CondAtomicSub
Go API, 243
CondAtomicXor
Go API, 261
CondDel
Go API, 237
CondDocumentRename
Go API, 300
CondDocumentUnset
Go API, 303
CondListLpush
Go API, 282
CondListRpush
Go API, 285
CondMapAdd
Go API, 306
CondMapAtomicAdd
Go API, 312
CondMapAtomicAnd
Go API, 327
CondMapAtomicDiv
Go API, 321
CondMapAtomicMax
Go API, 345
CondMapAtomicMin
Go API, 342
CondMapAtomicMod
Go API, 324
CondMapAtomicMul
Go API, 318

CondMapAtomicOr
Go API, 330
CondMapAtomicSub
Go API, 315
CondMapAtomicXor
Go API, 333
CondMapRemove
Go API, 309
CondMapStringAppend
Go API, 339
CondMapStringPrepend
Go API, 336
CondPut
Go API, 232
CondPutOrCreate
Go API, 233
CondSetAdd
Go API, 288
CondSetIntersect
Go API, 294
CondSetRemove
Go API, 291
CondSetUnion
Go API, 297
CondStringAppend
Go API, 273
CondStringLtrim
Go API, 276
CondStringPrepend
Go API, 270
CondStringRtrim
Go API, 279
Count
Go API, 349
count
C API, 203
Java API, 595
Node.js API, 721
Python API, 965
Ruby API, 1210

Del
Go API, 236
del
C API, 90
Java API, 369
Node.js API, 608
Python API, 741
Ruby API, 986
disable_perf_counters
C API, 225
document_rename
C API, 153
Java API, 495
Node.js API, 671
Python API, 867
Ruby API, 1112
document_unset
C API, 156
Java API, 501
Node.js API, 674
Python API, 873
Ruby API, 1118
DocumentRename
Go API, 299
DocumentUnset
Go API, 302

enable_perf_counters
C API, 224

fault_tolerance
C API, 208

Get
Go API, 229
get
C API, 83
Java API, 355
Node.js API, 601
Python API, 727
Ruby API, 972
get_partial
C API, 84
Java API, 357
Node.js API, 602
Python API, 729
Ruby API, 974
GetPartial
Go API, 230
group_atomic_add
C API, 95
Java API, 379
Node.js API, 613
Python API, 751
Ruby API, 996
group_atomic_and
C API, 110
Java API, 409
Node.js API, 628
Python API, 781
Ruby API, 1026
group_atomic_div
C API, 104
Java API, 397
Node.js API, 622
Python API, 769
Ruby API, 1014
group_atomic_max

C API, 122
Java API, 433
Node.js API, 640
Python API, 805
Ruby API, 1050
group_atomic_min
C API, 119
Java API, 427
Node.js API, 637
Python API, 799
Ruby API, 1044
group_atomic_mod
C API, 107
Java API, 403
Node.js API, 625
Python API, 775
Ruby API, 1020
group_atomic_mul
C API, 101
Java API, 391
Node.js API, 619
Python API, 763
Ruby API, 1008
group_atomic_or
C API, 113
Java API, 415
Node.js API, 631
Python API, 787
Ruby API, 1032
group_atomic_sub
C API, 98
Java API, 385
Node.js API, 616
Python API, 757
Ruby API, 1002
group_atomic_xor
C API, 116
Java API, 421
Node.js API, 634
Python API, 793
Ruby API, 1038
group_del
C API, 92
Java API, 373
Node.js API, 610
Python API, 745
Ruby API, 990
group_document_rename
C API, 155
Java API, 499
Node.js API, 673
Python API, 871
Ruby API, 1116
group_document_unset
C API, 158
Java API, 505
Node.js API, 676
Python API, 877
Ruby API, 1122
group_list_lpush
C API, 137
Java API, 463
Node.js API, 655
Python API, 835
Ruby API, 1080
group_list_rpush
C API, 140
Java API, 469
Node.js API, 658
Python API, 841
Ruby API, 1086
group_map_add
C API, 161
Java API, 511
Node.js API, 679
Python API, 883
Ruby API, 1128
group_map_atomic_add
C API, 167
Java API, 523
Node.js API, 685
Python API, 895
Ruby API, 1140
group_map_atomic_and
C API, 182
Java API, 553
Node.js API, 700
Python API, 925
Ruby API, 1170
group_map_atomic_div
C API, 176
Java API, 541
Node.js API, 694
Python API, 913
Ruby API, 1158
group_map_atomic_max
C API, 200
Java API, 589
Node.js API, 718
Python API, 961
Ruby API, 1206
group_map_atomic_min
C API, 197
Java API, 583
Node.js API, 715
Python API, 955
Ruby API, 1200
group_map_atomic_mod

C API, 179
Java API, 547
Node.js API, 697
Python API, 919
Ruby API, 1164
group_map_atomic_mul
C API, 173
Java API, 535
Node.js API, 691
Python API, 907
Ruby API, 1152
group_map_atomic_or
C API, 185
Java API, 559
Node.js API, 703
Python API, 931
Ruby API, 1176
group_map_atomic_sub
C API, 170
Java API, 529
Node.js API, 688
Python API, 901
Ruby API, 1146
group_map_atomic_xor
C API, 188
Java API, 565
Node.js API, 706
Python API, 937
Ruby API, 1182
group_map_remove
C API, 164
Java API, 517
Node.js API, 682
Python API, 889
Ruby API, 1134
group_map_string_append
C API, 194
Java API, 577
Node.js API, 712
Python API, 949
Ruby API, 1194
group_map_string_prepend
C API, 191
Java API, 571
Node.js API, 709
Python API, 943
Ruby API, 1188
group_put
C API, 88
Java API, 365
Node.js API, 606
Python API, 737
Ruby API, 982
group_set_add
C API, 143
Java API, 475
Node.js API, 661
Python API, 847
Ruby API, 1092
group_set_intersect
C API, 149
Java API, 487
Node.js API, 667
Python API, 859
Ruby API, 1104
group_set_remove
C API, 146
Java API, 481
Node.js API, 664
Python API, 853
Ruby API, 1098
group_set_union
C API, 152
Java API, 493
Node.js API, 670
Python API, 865
Ruby API, 1110
group_string_append
C API, 128
Java API, 445
Node.js API, 646
Python API, 817
Ruby API, 1062
group_string_ltrim
C API, 131
Java API, 451
Node.js API, 649
Python API, 823
Ruby API, 1068
group_string_prepend
C API, 125
Java API, 439
Node.js API, 643
Python API, 811
Ruby API, 1056
group_string_rtrim
C API, 134
Java API, 457
Node.js API, 652
Python API, 829
Ruby API, 1074
GroupAtomicAdd
Go API, 241
GroupAtomicAnd
Go API, 256
GroupAtomicDiv
Go API, 250
GroupAtomicMax

Go API, 268
GroupAtomicMin
Go API, 265
GroupAtomicMod
Go API, 253
GroupAtomicMul
Go API, 247
GroupAtomicOr
Go API, 259
GroupAtomicSub
Go API, 244
GroupAtomicXor
Go API, 262
GroupDel
Go API, 238
GroupDocumentRename
Go API, 301
GroupDocumentUnset
Go API, 304
GroupListLpush
Go API, 283
GroupListRpush
Go API, 286
GroupMapAdd
Go API, 307
GroupMapAtomicAdd
Go API, 313
GroupMapAtomicAnd
Go API, 328
GroupMapAtomicDiv
Go API, 322
GroupMapAtomicMax
Go API, 346
GroupMapAtomicMin
Go API, 343
GroupMapAtomicMod
Go API, 325
GroupMapAtomicMul
Go API, 319
GroupMapAtomicOr
Go API, 331
GroupMapAtomicSub
Go API, 316
GroupMapAtomicXor
Go API, 334
GroupMapRemove
Go API, 310
GroupMapStringAppend
Go API, 340
GroupMapStringPrepend
Go API, 337
GroupPut
Go API, 234
GroupSetAdd
Go API, 289
GroupSetIntersect
Go API, 295
GroupSetRemove
Go API, 292
GroupSetUnion
Go API, 298
GroupStringAppend
Go API, 274
GroupStringLtrim
Go API, 277
GroupStringPrepend
Go API, 271
GroupStringRtrim
Go API, 280

list_indices
C API, 214
list_lpush
C API, 135
Java API, 459
Node.js API, 653
Python API, 831
Ruby API, 1076
list_rpush
C API, 138
Java API, 465
Node.js API, 656
Python API, 837
Ruby API, 1082
list_spaces
C API, 213
list_subspaces
C API, 215
ListLpush
Go API, 281
ListRpush
Go API, 284

map_add
C API, 159
Java API, 507
Node.js API, 677
Python API, 879
Ruby API, 1124
map_atomic_add
C API, 165
Java API, 519
Node.js API, 683
Python API, 891
Ruby API, 1136
map_atomic_and
C API, 180
Java API, 549

Node.js API, 698
Python API, 921
Ruby API, 1166
map_atomic_div
C API, 174
Java API, 537
Node.js API, 692
Python API, 909
Ruby API, 1154
map_atomic_max
C API, 198
Java API, 585
Node.js API, 716
Python API, 957
Ruby API, 1202
map_atomic_min
C API, 195
Java API, 579
Node.js API, 713
Python API, 951
Ruby API, 1196
map_atomic_mod
C API, 177
Java API, 543
Node.js API, 695
Python API, 915
Ruby API, 1160
map_atomic_mul
C API, 171
Java API, 531
Node.js API, 689
Python API, 903
Ruby API, 1148
map_atomic_or
C API, 183
Java API, 555
Node.js API, 701
Python API, 927
Ruby API, 1172
map_atomic_sub
C API, 168
Java API, 525
Node.js API, 686
Python API, 897
Ruby API, 1142
map_atomic_xor
C API, 186
Java API, 561
Node.js API, 704
Python API, 933
Ruby API, 1178
map_remove
C API, 162
Java API, 513
Node.js API, 680
Python API, 885
Ruby API, 1130
map_string_append
C API, 192
Java API, 573
Node.js API, 710
Python API, 945
Ruby API, 1190
map_string_prepend
C API, 189
Java API, 567
Node.js API, 707
Python API, 939
Ruby API, 1184
MapAdd
Go API, 305
MapAtomicAdd
Go API, 311
MapAtomicAnd
Go API, 326
MapAtomicDiv
Go API, 320
MapAtomicMax
Go API, 344
MapAtomicMin
Go API, 341
MapAtomicMod
Go API, 323
MapAtomicMul
Go API, 317
MapAtomicOr
Go API, 329
MapAtomicSub
Go API, 314
MapAtomicXor
Go API, 332
MapRemove
Go API, 308
MapStringAppend
Go API, 338
MapStringPrepend
Go API, 335
mv_space
C API, 212

Put
Go API, 231
put
C API, 85
Java API, 359
Node.js API, 603
Python API, 731
Ruby API, 976

put_if_not_exist
C API, 89
Java API, 367
Node.js API, 607
Python API, 739
Ruby API, 984
PutIfNotExist
Go API, 235

read_only
C API, 206
rm_index
C API, 217
rm_space
C API, 211

Search
Go API, 347
search
C API, 201
Java API, 591
Node.js API, 719
Python API, 963
Ruby API, 1208
search_describe
Java API, 592
server_forget
C API, 221
server_kill
C API, 222
server_offline
C API, 220
server_online
C API, 219
server_register
C API, 218
set_add
C API, 141
Java API, 471
Node.js API, 659
Python API, 843
Ruby API, 1088
set_intersect
C API, 147
Java API, 483
Node.js API, 665
Python API, 855
Ruby API, 1100
set_remove
C API, 144
Java API, 477
Node.js API, 662
Python API, 849
Ruby API, 1094
set_union
C API, 150
Java API, 489
Node.js API, 668
Python API, 861
Ruby API, 1106
SetAdd
Go API, 287
SetIntersect
Go API, 293
SetRemove
Go API, 290
SetUnion
Go API, 296
sorted_search
C API, 202
Java API, 594
Node.js API, 720
Python API, 964
Ruby API, 1209
SortedSearch
Go API, 348
string_append
C API, 126
Java API, 441
Node.js API, 644
Python API, 813
Ruby API, 1058
string_ltrim
C API, 129
Java API, 447
Node.js API, 647
Python API, 819
Ruby API, 1064
string_prepend
C API, 123
Java API, 435
Node.js API, 641
Python API, 807
Ruby API, 1052
string_rtrim
C API, 132
Java API, 453
Node.js API, 650
Python API, 825
Ruby API, 1070
StringAppend
Go API, 272
StringLtrim
Go API, 275
StringPrepend
Go API, 269
StringRtrim
Go API, 278

validate_space
C API, 209

wait_until_stable
C API, 207